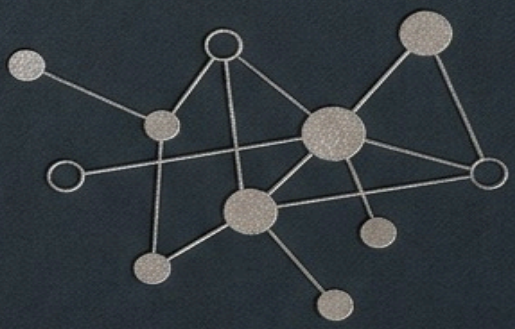
Audit Analytics with CaatMaster

A practitioner's guide to AI-powered computer-assisted audit workflows





Ugochukwu Onyeka

COMPLETE EDITION · NINE PARTS, THIRTY-THREE CHAPTERS, FOUR APPENDICES

Audit Analytics with CaatMaster

A practitioner's guide to AI-powered computer-assisted audit workflows.

Testing for error, omission, and fraud, using computer-assisted audit techniques, financial statement analytics, and AI-assisted review — without asking the reader to write a line of code, except where Appendix D shows how to.

UGOCHUKWU ONYEKA

Parts I–IX: Complete 33 Chapters Companion to CaatMaster

Audit Analytics with CaatMaster

Copyright © 2026 Ugochukwu Onyeka. All rights reserved.

Companion book to the CaatMaster desktop application. No part of this book may be reproduced or distributed without permission, except for brief quotations in a review.

CaatMaster itself is licensed separately, under the Apache License, Version 2.0 — see the project repository for details.

Contents

PART I — FOUNDATIONS

1 What This Book Is About 7

2 Getting to Know CaatMaster 11

3 Getting Your Data In Correctly 17

4 Deterministic Math, AI Where It Belongs 23

PART II — CLASSIC CAATS

5 Data Profiling and Descriptive Statistics 27

6 Completeness and Integrity 32

7 Cross-Tabulation and Grouping 37

8 Benford's Law 41

9 Ratio and Trend Analysis 47

10 Outlier and Anomaly Detection 54

11 Sampling and Stratification 60

PART III — INTERNAL CONTROLS

12 Segregation of Duties and Access Review 70

13 Mapping Findings to COSO 77

14 Deterministic Rule Libraries 82

PART IV — IFRS ASSERTIONS

15 Revenue Recognition (IFRS 15) 87

16 Expected Credit Losses (IFRS 9) 92

17 Leases (IFRS 16) 97

18 Impairment (IAS 36) 101

19 Provisions and Inventory (IAS 37, IAS 2) 106

20 Tax and Currency (IAS 12, IAS 21) 110

PART V — SOURCE DATA

21 Ratio Analysis Done Right 115

22 Reading XBRL Directly 122

PART VI — AI, USED RESPONSIBLY

23 The Grounded AI Pattern 128
24 AI-Assisted Text and Document Analysis 132
25 Numeric Disclosure Reconciliation 137
26 Conversational Analysis 141

PART VII — RESULTS

27 Visualizing Findings 146
28 Building the Audit Report 150
29 Reusable Audit Programs 152
30 Managing Projects and Sessions 155

PART VIII — JUDGMENT

31 Every Flag Is a Question 158
32 Governance, Security, and Limits 161

PART IX — DIRECT DATA ACCESS

33 The SQL Query Workflow 165

APPENDICES

A Operation Reference 171
B Dataset Reference 184
C Default Thresholds Reference 187
D A Quick Reference for Polars and SQL 190

PART I

Foundations

Chapter 1

What This Book Is About

Financial statements can be wrong for reasons that have nothing to do with dishonesty. A clerk enters an invoice twice because two people happened to process the same batch of mail on the same afternoon. A finance team continues to apply a depreciation rate that was correct for one class of equipment to a newly acquired class that depreciates differently. A revenue recognition policy that was appropriate under a prior accounting standard is left unchanged after the standard was replaced. None of these situations involves an intention to deceive, and each of them can materially misstate a set of financial statements.

Fraud does happen, and this book addresses it directly, in the chapters on segregation of duties, Benford's Law, and journal entry testing, among others. But an audit methodology built primarily around catching dishonest people misses most of what actually goes wrong in financial reporting. In most engagements, the majority of findings trace back to error, to control weaknesses that permit error to go undetected, and to the misapplication of judgment on accounting estimates that are inherently difficult to get exactly right. A well designed set of audit analytics has to surface all three causes, and it has to do so without prejudging which one it has found.

This distinction is central enough to the way this book is written that it recurs as a deliberate discipline in every technique chapter. Each chapter that introduces an analytical method closes with a section titled Interpreting the Result, and that section asks the same three questions in the same order every time: what benign, procedural explanation would produce this pattern; what error or lapse in judgment would produce it; and what would have to be true for the pattern to reflect deliberate manipulation. A flagged transaction is the beginning of an inquiry, not the end of one. A reader who works through the technique chapters while skipping this framing will still learn to operate CaatMaster competently. That reader will not learn to use it the way an experienced audit senior does, which is to treat every exception as a question addressed to a specific person, most of whom can answer it in a single sentence.

What CaatMaster Is

CaatMaster is a desktop application for computer-assisted audit techniques, built for auditors, internal control specialists, and compliance officers who need to examine a complete population of transactions rather than a sample of them. It combines three kinds of capability that are usually found in separate tools: general-purpose data transformation and cleaning, of the kind found in a spreadsheet or a business intelligence product; a library of audit-specific analytical tests, ranging from long-established techniques such as Benford's Law and gap analysis to statistical estimates required by specific accounting standards such as IFRS 9 and IAS 36; and a set of AI-assisted features for working with narrative text and unstructured documents, designed under a strict rule described in Chapter 4 that the AI is never asked to count, sum, or estimate a figure that Polars, the underlying data engine, can compute exactly.

Every operation applied to a dataset in CaatMaster is recorded as a named, ordered step. This record, called the Applied Steps pipeline, can be reviewed, reordered, disabled, or reapplied to a refreshed extract of the same data without repeating the analysis by hand. The purpose of this design is reproducibility. An audit finding that cannot be regenerated from the same inputs by a different reviewer is a weak finding, regardless of how interesting it looked the first time it appeared. Chapter 2 describes this pipeline and the rest of the application's structure in detail.

Who Should Read This Book

This book is written for three overlapping audiences. External and internal auditors will use it as a guide to running substantive tests and control evaluations inside CaatMaster, and as a reference for the accounting and statistical reasoning behind each test, not only the sequence of buttons that runs it. Risk and compliance officers will use it to design recurring, self-administered reviews ahead of a formal audit. Engagement leaders and audit methodology staff will use the later chapters, particularly those on audit programs and reporting, to standardize procedures across a team so that the same test performed by two different people produces the same result.

The book assumes a working knowledge of financial statements and of the audit process generally. It does not assume prior experience with a specific CAAT tool, with Python, or with statistics beyond what a professional accounting qualification already covers. Where a technique depends on a statistical concept that a reader might not have encountered, such as the interquartile range or a z-score, that concept is explained at the point where it is first used rather than gathered into a separate mathematics appendix.

How This Book Is Organized

The nine parts of this book move from general capability toward specific application. Part I, which this chapter opens, covers the application itself: its structure, how to bring data into it without corrupting the very numbers under examination, and the design philosophy that separates deterministic calculation from AI assistance. Part II covers the classic techniques common to almost every CAAT tool: profiling, completeness testing, cross-tabulation, Benford's Law, ratio analysis, anomaly detection, and sampling. Part III addresses internal control testing under the frameworks most commonly referenced in a financial statement audit, the Sarbanes-Oxley Act and the COSO Internal Control Framework. Part IV takes up the estimate-heavy areas of International Financial Reporting Standards where a recalculation, rather than a simple exception test, is the appropriate audit response: revenue recognition, expected credit losses, leases, impairment, provisions, inventory, tax, and foreign currency.

Part V returns to a problem introduced in Chapter 3 and treats it at full length: getting genuinely reliable figures out of a client's own reporting, whether that reporting arrives as a spreadsheet or as a structured XBRL filing. Part VI covers the AI-assisted features in detail, beginning with the architectural pattern that keeps them from inventing figures. Part VII covers what happens after an analysis is complete: visualizing results, building a report, saving a procedure for reuse next year, and managing the underlying projects and datasets. Part VIII closes with two chapters on the limits of everything that came before it, on professional skepticism as an operating discipline rather than a slogan, and on the data governance obligations that a tool like this one creates for the firm that deploys it. Part IX closes the book with a single chapter on querying data directly with SQL, for the relational questions — self-joins, cross-dataset joins, and per-group ranking — that the Applied Steps pipeline's single-operation model doesn't fit as naturally.

How to Read This Book

Readers who are new to computer-assisted audit techniques generally are encouraged to read Parts I and II in sequence before working on a live engagement, since later chapters assume familiarity with the Applied Steps pipeline and with the general shape of a CaatMaster analysis. Readers who already have experience with a CAAT tool such as ACL or IDEA may move directly to Part III or Part IV and treat Parts I and II as a reference, consulted when a specific mechanism, such as the numeric parsing behavior covered in Chapter 3, needs clarification.

Every technique chapter in Parts II through V follows the same internal structure: an explanation of what the technique detects and why it matters, instructions for running it inside

CaatMaster, an explanation of the calculation performed underneath the interface, a worked case study with realistic figures, and an Interpreting the Result section that applies the three-question discipline described earlier in this chapter. A reader in a hurry before a specific procedure can go directly to the how-to-run-it section of the relevant chapter. A reader building genuine competence in audit analytics, rather than familiarity with one tool's menus, should read the surrounding sections as well, since the underlying reasoning is what continues to apply after the software changes.

> A NOTE ON TERMINOLOGY
>
> Throughout this book, the word finding refers to any pattern surfaced by an analytical procedure, without regard to its eventual cause. The word exception is used interchangeably with finding. Neither word implies fraud, error, or wrongdoing of any kind. Where a specific cause is meant, this book says so directly.

Chapter 2

Getting to Know CaatMaster

A CAAT tool is judged, in practice, less by the sophistication of any single test it offers than by how easily an auditor can trust the trail leading to that test's result. This chapter describes the structural features of CaatMaster that exist to support that trust: the project and dataset model, the Applied Steps pipeline, and the arrangement of views that an auditor moves between during a typical engagement.

Projects and Datasets

Work in CaatMaster is organized into projects. A project corresponds, in most cases, to a single audit engagement or a single recurring compliance review, and it holds every dataset, transformation, and saved analysis associated with that engagement in one file. A dataset is a single table of data brought into the project, whether that table originated as a comma-separated export from a general ledger system, a worksheet from an Excel workbook, or the output of a query against a client's database.

A project can hold any number of datasets, and datasets within a project can be joined, appended, or compared against one another. This matters for tests that depend on more than one source at once. A three-way match between a purchase order, a goods received note, and an invoice, for example, requires three separate datasets to be joined on a common key before the comparison can run; Chapter 6 works through this exact procedure. Because the join itself is recorded as a step, the three-way match can be rerun automatically the following quarter against refreshed extracts of the same three tables.

The Applied Steps Pipeline

Every transformation applied to a dataset, from a simple column rename to a statistical calculation with a dozen parameters, is recorded as a step in that dataset's Applied Steps pipeline. The pipeline is visible at all times as an ordered list alongside the data itself. A step can be temporarily disabled without being deleted, which allows an auditor to compare a dataset's state before and after a given transformation without losing either version. A step can also be reordered, subject to the transformations later in the pipeline still being valid against the

changed sequence, and the entire pipeline can be reapplied in one operation to a freshly imported version of the same source file.

This last capability is what makes a procedure genuinely reusable rather than merely repeatable by manual reconstruction. An audit program built once, during the current year's engagement, against this year's trial balance extract can be saved and, in the following year, pointed at next year's extract with the same column names, producing the same tests without the auditor rebuilding a single step. Chapter 29 covers the mechanics of saving and reusing an audit program in this way.

> WHY THIS MATTERS FOR REVIEW
>
> A second reviewer examining a finding does not need to take the first preparer's word for how a number was produced. The Applied Steps pipeline for the dataset that produced it is a literal, ordered record of every transformation involved, from the original import to the final flag. This is the same principle that underlies a well-documented spreadsheet formula, applied at the level of an entire analysis rather than a single cell.

Moving Around the Application

CaatMaster presents its functionality through a set of views, reached from the sidebar, each responsible for one stage of the work. The Data view displays the current state of a dataset in a table and is where a dataset is first imported. The Transform view exposes the Applied Steps pipeline described above and is where the great majority of the transformations and tests covered in this book are actually run, since every statistical, audit, financial, and machine-learning operation opens as a dialog on top of it, in one of the three ways described below. The SQL Query workspace, covered in full in Chapter 33, provides a direct SQL interface against every dataset currently open in the project, useful when a question is more naturally expressed as a join or an aggregation than as a sequence of individual operations. The Visualizations view builds charts from any dataset in the project. The Reports view assembles findings into a document for distribution, covered in Chapter 28. The Projects view manages the engagements themselves, and the Dashboard view gives a summary of recent activity across the current project.

A single dataset can be worked on from more than one view without conflict, because every view operates against the same underlying Applied Steps pipeline for that dataset. A transformation

applied from the Transform view is immediately reflected in the Data view's table, and a chart built in the Visualizations view against a dataset updates automatically if an upstream step in that dataset's pipeline is later modified.

Three Ways to Reach an Operation

Every technique chapter in this book states, precisely, where the operation it covers is found, because CaatMaster genuinely offers three different starting points, and they are not interchangeable. Getting this distinction wrong is the single most common way an instruction in a CAAT tool's documentation fails to match what a reader sees on their own screen, so it is worth establishing clearly here rather than leaving it to be inferred chapter by chapter.

The first path is the sidebar itself. A small number of features, Profile Dataset, Verify Data, Variables Sampling, and Audit Programs under the Analyze heading, along with Join Datasets, Append Datasets, Compare Datasets, and Fuzzy Match Datasets under Combine, are reached only from the sidebar. Most of these are standalone tools rather than steps in the Applied Steps pipeline; Profile Dataset and Verify Data, for instance, produce a read-only report and never modify the underlying dataset.

The second path is the top menu bar: File, Transform, Audit, Statistics, Reports, AI, ML, and View. Each of these menus holds a curated, thematically grouped subset of the full operation catalog, promoted to a permanent, one-click location because it is used frequently enough to deserve one. The Audit menu's Audit Checks submenu, for example, includes Benford's Law Analysis, Outlier Detection, and Round Number Analysis, all covered in this book, but does not include every audit-related operation CaatMaster offers.

The third path is the one that actually reaches everything: right-clicking any column header in the Transform view's data grid. This opens a menu of quick, one-click actions for that specific column, Sort Ascending, Sort Descending, Remove, and Visualize, followed by a complete, category-organized listing of every single operation in the application, including a number of Statistics and Machine Learning operations, such as Quick Stats and Group Summary, that have no shortcut anywhere in the top menu bar or the sidebar. A reviewer who cannot find an operation in either of the first two locations should always try this third path before concluding the feature does not exist. Appendix A lists every operation this menu contains, organized by the same categories the menu itself uses, together with where else, if anywhere, each one can also be reached.

Working Across Multiple Datasets

Several of the techniques in this book, including the three-way match mentioned earlier and the segregation of duties analysis in Chapter 12, depend on bringing together data that arrives as separate extracts: a purchase order table, a goods received table, an invoice table, a user access log. CaatMaster handles this through the Join Datasets and Append Datasets operations, both of which are recorded as ordinary steps in the resulting dataset's pipeline in the same way as any single-table transformation.

A join produces a new dataset combining columns from two existing ones, matched on a key that need not share the same name in both source tables. An append stacks two datasets with compatible columns into one, useful for combining several months of transaction extracts into a single population before running a test across the full period. Both operations are ordinary, replayable steps, which means a monthly append-and-test procedure, once built, runs identically every month without being rebuilt from source files each time.

The Working Dataset Used in This Book

Every technique chapter from this point forward is demonstrated against two small datasets, so that the instructions can be followed step by step rather than taken on faith. Both datasets belong to a fictional company, Meridian Office Supply Co., invented for this book and not based on any real business. The first, used throughout Part II, is an extract of thirty-four accounts payable transactions covering January and February of 2025, spanning eight vendors, six departments, and seven expense accounts. The table below shows the first ten records; the complete file carries the same ten columns for all thirty-four.

ID	Date	Vendor	Department	Account	Amount	Approver
20001	2025-01-03	Anderson Paper Co	Operations	Office Supplies	842.50	T. Alvarez
20002	2025-01-04	Cascade IT Solutions	IT	IT Equipment	1,290.00	R. Chen
20003	2025-01-06	BrightPath Logistics	Operations	Maintenance	675.25	T. Alvarez
20004	2025-01-07	Fenwick Legal Group	Legal	Professional Services	4,900.00	M. Okafor
20005	2025-01-08	Evergreen Facilities	Facilities	Utilities	3,150.75	S. Patel
20006	2025-01-09	GreenLeaf Catering	Marketing	Travel & Entertainment	410.00	*(blank)*

ID	Date	Vendor	Department	Account	Amount	Approver
20007	2025-01-10	Harborview Consulting	Finance	Consulting Fees	5,000.00	R. Chen
20008	2025-01-13	Anderson Paper Co	Operations	Office Supplies	842.50	T. Alvarez
20009	2025-01-14	Delta Office Interiors	Facilities	Maintenance	1,225.00	S. Patel
20010	2025-01-15	Cascade IT Solutions	IT	IT Equipment	9,900.00	R. Chen

Figure 2.1 — The first ten rows of ap_transactions.csv, the working file used throughout Part II.

The second file, introduced in Chapter 9, is a four-quarter financial summary for the same company, used for ratio and trend analysis. Both files are intentionally small. A real engagement population runs to thousands or millions of rows, and CaatMaster is built for exactly that scale, but a book cannot print a million-row table, and a reader cannot verify a calculation against a population too large to inspect. Where a technique behaves differently at production scale than it does against these thirty-four rows, that difference is called out explicitly in the relevant chapter, most notably in Chapter 8, where Benford's Law requires a considerably larger population than this working file provides.

The instructions in every following chapter assume this file has already been imported as a dataset named AP Transactions, following the steps described in Chapter 3. The screenshot below shows the Transform view immediately after that import, with the Applied Steps panel on the right still empty, since no transformation has yet been applied.

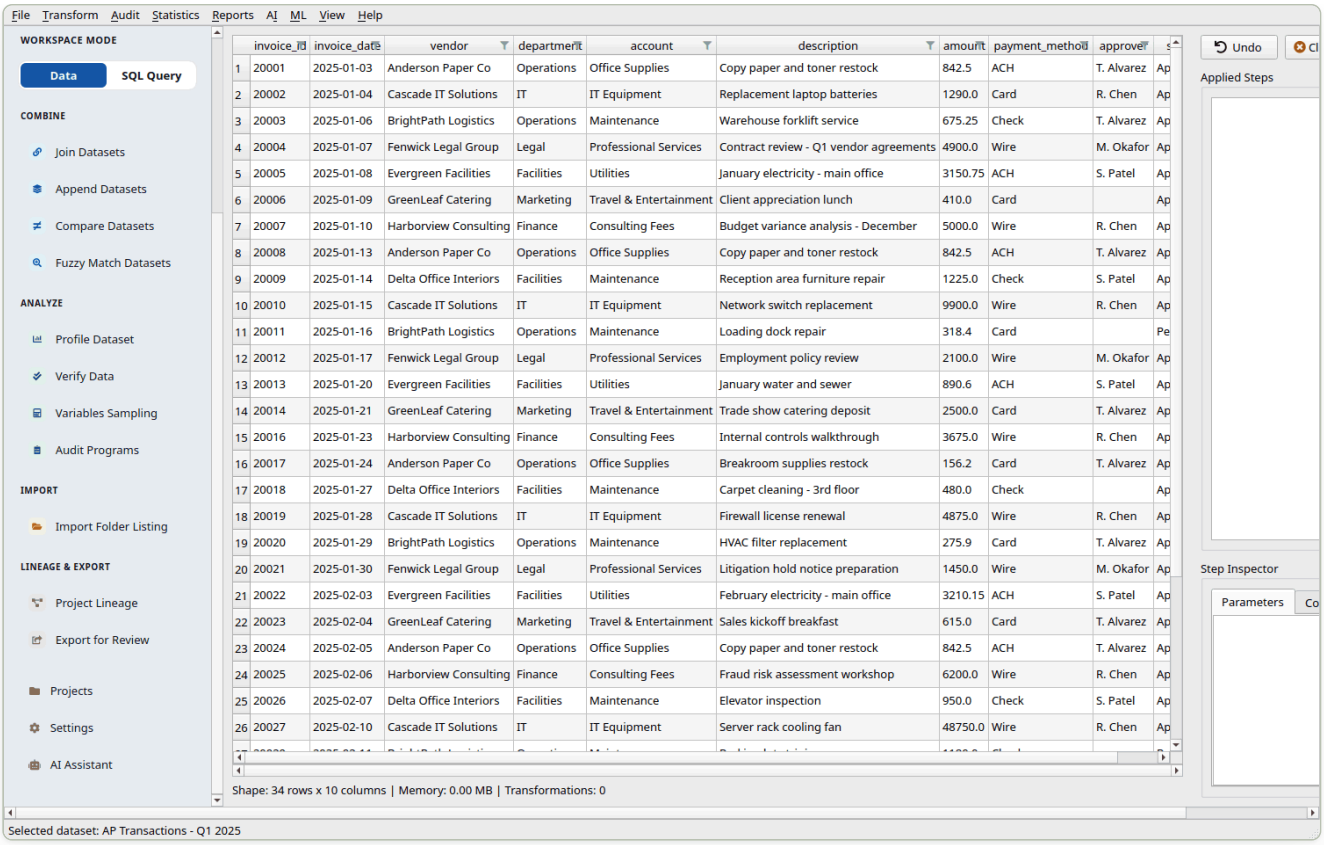


Figure 2.2 — The Transform view immediately after importing ap_transactions.csv. The sidebar on the left groups every available operation by category (Combine, Analyze, Import, Lineage & Export); the grid in the center shows the data itself; the Applied Steps panel on the right will record each transformation as it is added.

What this chapter's structure supports, and what is worth carrying forward into every later chapter, is a question that precedes any specific test: can the number in front of the reviewer be traced, step by step, back to the file it came from. In CaatMaster, the answer is always yes, and the Applied Steps pipeline is where that trace lives.

Chapter 3

Getting Your Data In Correctly

No analytical technique in this book can produce a correct result from a number that was misread on the way into the application. This sounds obvious enough to skip past, and it is skipped past often enough in practice that misread figures are among the most common causes of a wrong audit conclusion produced by software rather than by a person. This chapter covers the specific ways a financial figure can be misread during import, and the features CaatMaster provides to prevent it.

The Problem with Ordinary Numbers

A number as it appears in an accounting export rarely looks like a number as a computer understands one. A trial balance extracted from an enterprise resource planning system typically presents amounts with a thousands separator, so that one million two hundred thirty-four thousand five hundred sixty-seven dollars appears as `1,234,567` rather than `1234567`. A negative amount, following long-standing accounting convention, is frequently shown in parentheses rather than with a leading minus sign, so that a credit balance of eight hundred thousand dollars appears as `(800,000)`. A figure copied from a report generator may carry a currency symbol, a percent sign, or trailing whitespace.

A general-purpose data tool that reads a column of such values without accounting for these conventions has two possible failure modes, and both are worse than an outright error message. The first is that the import fails to recognize the column as numeric at all, and every subsequent calculation against it silently returns nothing, without necessarily producing a visible error if the calling code tolerates missing values. The second, more dangerous failure, is that the import partially succeeds: some rows in the column parse correctly and others silently become null, so that a sum, an average, or a ratio computed from the column is quietly built from an incomplete population, with no indication in the resulting figure that anything was dropped.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose an internal audit team imports a subsidiary's trial balance into a general-purpose analytics tool ahead of a quarterly review. The Amount column, as exported from the subsidiary's accounting system, uses a thousands separator throughout and represents every credit balance in parentheses. The import tool recognizes the column as text, since none of its values match a bare numeric pattern, and the team's workflow calls for casting the column to a number before running any ratio tests. The cast succeeds for values without a comma, such as small expense line items, and fails silently for every larger balance, which is to say for nearly every asset and liability account that matters. The resulting current ratio, computed from a current assets total that is missing most of its constituent accounts, comes out below one, and the team could spend a full day investigating an apparent liquidity problem that does not exist before tracing the discrepancy back to the import step.
>
> A reader whose own import process casts a text column to a number in a single step, without a validated intermediate parse, is exposed to exactly this failure mode, whether or not it has yet produced a wrong ratio that anyone noticed.

How CaatMaster Parses a Number

CaatMaster resolves a value formatted this way by recognizing, rather than merely stripping, the specific conventions that a financial figure uses. A recognized currency symbol or a small set of common ISO currency codes at the start of the value is removed. A parenthesized value, with or without a preceding currency symbol, is treated as negative and its sign is applied after the surrounding punctuation is removed. A trailing minus sign, occasionally produced by older accounting systems, is moved to the front of the value before parsing. Thousands separators are removed once the value has been confirmed, by pattern, to actually be a number rather than a piece of text that happens to contain digits.

That last qualification matters as much as the parsing rules themselves. A parser that strips every character that is not a digit, a decimal point, or a minus sign will, in the course of correctly handling `1,234,567`, also convert a quarter label such as `Q1` into the number one, and a fiscal year reference such as `FY2023` into the number two thousand twenty-three. Both of these are real values that appear routinely in period columns and reference numbers

throughout a financial dataset, and misreading either one as a numeric account balance would be a worse outcome than failing to parse it at all. CaatMaster's numeric parser validates that what remains after removing a recognized currency prefix and thousands separators is actually a clean, properly formed number before accepting it, and returns a null value, exactly as a failed parse should, for anything that is not.

As written in the source file	Parses to
1,234,567	1234567
(800,000)	-800000
$(50,000)	-50000
USD 1,500	1500
Q1	null — not a financial figure
FY2023	null — not a financial figure
INV-2024-001	null — not a financial figure

Figure 3.1 — Representative inputs to CaatMaster's numeric parser and their results.

Running an Import

A dataset enters CaatMaster through the Import Data command, which accepts comma-separated files, Excel workbooks of either the legacy or current format, and, for financial statement material specifically, the Financial Import Wizard, which applies additional heuristics suited to a trial balance, a general ledger extract, or a chart of accounts. Where a source workbook contains more than one worksheet, the import prompts for a selection rather than guessing which sheet holds the intended data.

For a financial statement or trial balance import specifically, running the file through the Financial Import Wizard rather than a plain import is worth the additional step. The wizard applies the numeric parsing described above automatically to every column it identifies as likely to be financial, proposes a mapping from the source file's account labels to the standard concepts used throughout the rest of the application, such as revenue, total assets, or current liabilities, and flags any column where a meaningful share of values failed to parse cleanly, so that the failure is visible before it propagates into a ratio calculation rather than after.

1. From the sidebar, under Import, choose `Import Folder Listing` or the equivalent Import Data command for a single file, and select `ap_transactions.csv`, the working file introduced in Chapter 2.
2. CaatMaster reads the file, infers a data type for each of the ten columns, and opens the newly created dataset directly in the Transform view.
3. Give the dataset a clear name, such as `AP Transactions`, so it can be referred to unambiguously later, including from any dialog that lets a step reference a second dataset.

Cleaning What the Import Cannot Fix Alone

Some data quality issues are not a matter of numeric formatting and require a deliberate cleaning step regardless of how the import itself is handled. A column header that spans two rows in the source spreadsheet, a column that mixes a date format from one region with a date format from another partway down its length, or a row that is actually a subtotal rather than a transaction, are all common enough in real accounting exports that CaatMaster provides a dedicated Cleaning Wizard to surface and resolve them before an analytical test is run against the affected dataset.

The Cleaning Wizard inspects a freshly imported dataset and proposes a set of corrective steps, each shown with a preview of its effect before it is accepted. As with every other transformation in the application, an accepted cleaning step becomes an ordinary entry in the dataset's Applied Steps pipeline, and is therefore visible to a second reviewer in exactly the same way a manually applied transformation would be. Every proposed step is tied to an exact, checkable rule rather than a guess; where a judgment call is genuinely required, such as deciding which columns are identifiers rather than values, the wizard proposes the step unchecked, and applies it only if the reviewer actively selects it.

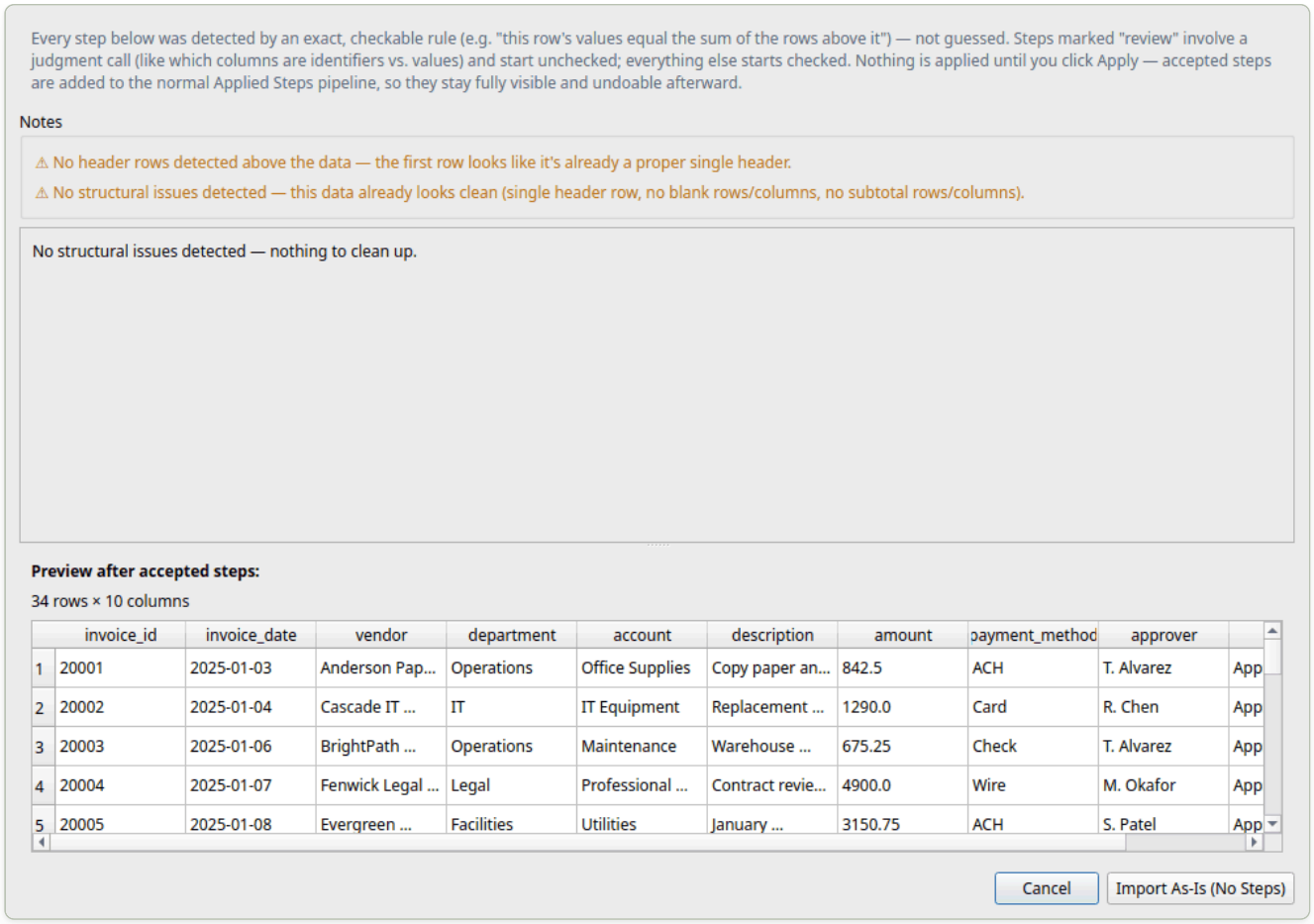


Figure 3.2 — The Cleaning Wizard run against ap_transactions.csv. Because this working file already carries a single clean header row and no blank or subtotal rows, the wizard reports nothing to correct; a messier export would instead list each proposed correction here, with a preview of its effect, before anything is applied.

Interpreting the Result

WHEN AN IMPORT PRODUCES FEWER VALID ROWS THAN EXPECTED

1. **Benign explanation.** The source file may genuinely contain blank or non-financial rows, such as section headers or spacing rows carried over from a formatted spreadsheet, which should not have parsed as numeric values in the first place.

2. **Error or judgment lapse.** The source system may have exported a column using a numeric convention, such as a regional decimal comma, that the import step was not configured to expect, causing otherwise valid figures to fail parsing.

3. **What would indicate something more serious.** A pattern in which specific accounts or specific value ranges consistently fail to parse, rather than a random scattering of blank rows, warrants a direct question to whoever prepared the export about whether the extract itself was filtered or altered before it reached the audit team.

Chapter 4

Deterministic Math, AI Where It Belongs

Artificial intelligence, and specifically the large language models behind CaatMaster's AI-assisted features, is good at particular kinds of task and poor at others, in ways that are easy to state clearly and important to keep in view throughout the rest of this book. A language model is good at reading unstructured text and extracting the plain-English content of it: identifying what a paragraph in a note to the financial statements is asserting, translating a compliance policy written in prose into a checkable rule, or drafting a description of a finding in professional audit language. A language model is poor at arithmetic performed silently inside its own reasoning, and it is poor in a specific way that matters for an audit tool: it will very often produce an answer that is plausible, confidently stated, and wrong, with no signal to the reader that anything is amiss.

CaatMaster is built around a single governing rule that follows from this distinction: a calculation that Polars, the deterministic data engine underneath the application, can perform exactly is never delegated to the AI, regardless of how convenient it might be to simply ask a language model for the answer. The AI is used only for the narrower class of task it is actually reliable at, and even there, its output is checked against real data before it is presented as a finding.

The Two Layers of the Application

Every calculation in this book that produces a number, whether it is a Benford deviation, a lease liability, or an expected credit loss under IFRS 9, is computed by ordinary, reproducible arithmetic against the complete dataset, using the Polars library. Run the same test twice against the same data and the result is identical to the last decimal place, every time, on every machine. None of these calculations involve a language model at any stage, and none of them can be affected by a change in AI provider, a change in a model's training, or a network outage.

A separate layer of the application, described at length in Part VI, uses a language model for tasks that genuinely require reading and producing natural language: summarizing a long narrative disclosure, classifying free-text entries into categories, extracting a list of compliance rules from a written policy document, or drafting the descriptive prose of an audit finding. This

layer is deliberately kept away from any task that involves counting, summing, or estimating a quantity, because that is exactly the class of task at which a language model is unreliable in a way that is difficult to detect after the fact.

> PRINCIPLE
>
> If a number in a CaatMaster report came from an AI call, something in the design has gone wrong. Every number in this book's worked examples, and every number CaatMaster itself produces, is traceable to a deterministic calculation against the full population of data, never to a language model's own arithmetic.

The Translate-Then-Compute Pattern

Where an AI-assisted feature does need to interact with real figures, CaatMaster applies a consistent pattern, used throughout the compliance rule extraction feature covered in Chapter 14 and the numeric disclosure reconciliation feature covered in Chapter 25. The language model is asked to translate a piece of unstructured input, such as a sentence of policy text or a paragraph of narrative disclosure, into a structured, checkable statement: a filter condition, or a labeled figure with the sentence it came from. That structured statement is then evaluated against the complete dataset using ordinary code, producing an exact count, an exact dollar amount, and a specific list of the rows involved. Only after this real computation has happened is the language model asked to draft the prose description of the finding, using the real numbers it is given rather than numbers it might otherwise be tempted to estimate.

This division of labor means the language model is never in a position to simply state that forty percent of transactions violate a policy. It is asked, instead, to say what the policy requires in a form that can be tested, and the forty percent figure, if that turns out to be the correct figure, comes from actually testing every row.

What This Means When You Read a Finding

A finding produced entirely by a deterministic test, such as a duplicate payment or a Benford deviation, carries no qualification about its source, because none is needed. A finding whose descriptive text was drafted with AI assistance is marked as such wherever it appears in the application, and the underlying figures behind that description remain fully traceable to the

same Applied Steps pipeline described in Chapter 2. An auditor reviewing such a finding should read the drafted prose as a starting point for the workpaper narrative, subject to the same editorial judgment any drafted text would receive, while treating the count, the amount, and the evidence rows as already verified, because they were computed the same way any other number in this book is computed.

This distinction is worth carrying forward as a standing question to ask of any audit software claiming an AI-assisted feature, CaatMaster included: does a specific number in the output trace back to a real calculation against the complete population, or was it produced, at some point in the chain, by asking a language model what the answer probably is. Where the second is true, the number deserves exactly the scrutiny a rough estimate deserves, no more and no less. Where the first is true, as it is throughout CaatMaster's analytical library, the number can be relied upon the same way any other output of a well-tested calculation can be relied upon, and the auditor's attention belongs on interpreting what the number means, not on verifying that it is real.

PART II

Classic CAATs

Chapter 5

Data Profiling and Descriptive Statistics

Before any specific audit test is chosen, a reviewer needs a general sense of what the data actually contains. How many rows are there, and how many of them are exact duplicates of one another. How complete is each column, and how many distinct values does it hold. Are there numeric columns whose values spread out in a way that already suggests further inquiry, before a single formal test has been run against them. This general orientation is called profiling, and it belongs at the start of nearly every engagement, not because it produces findings on its own, but because it tells the reviewer where to look next.

Running the Profile Dataset Command

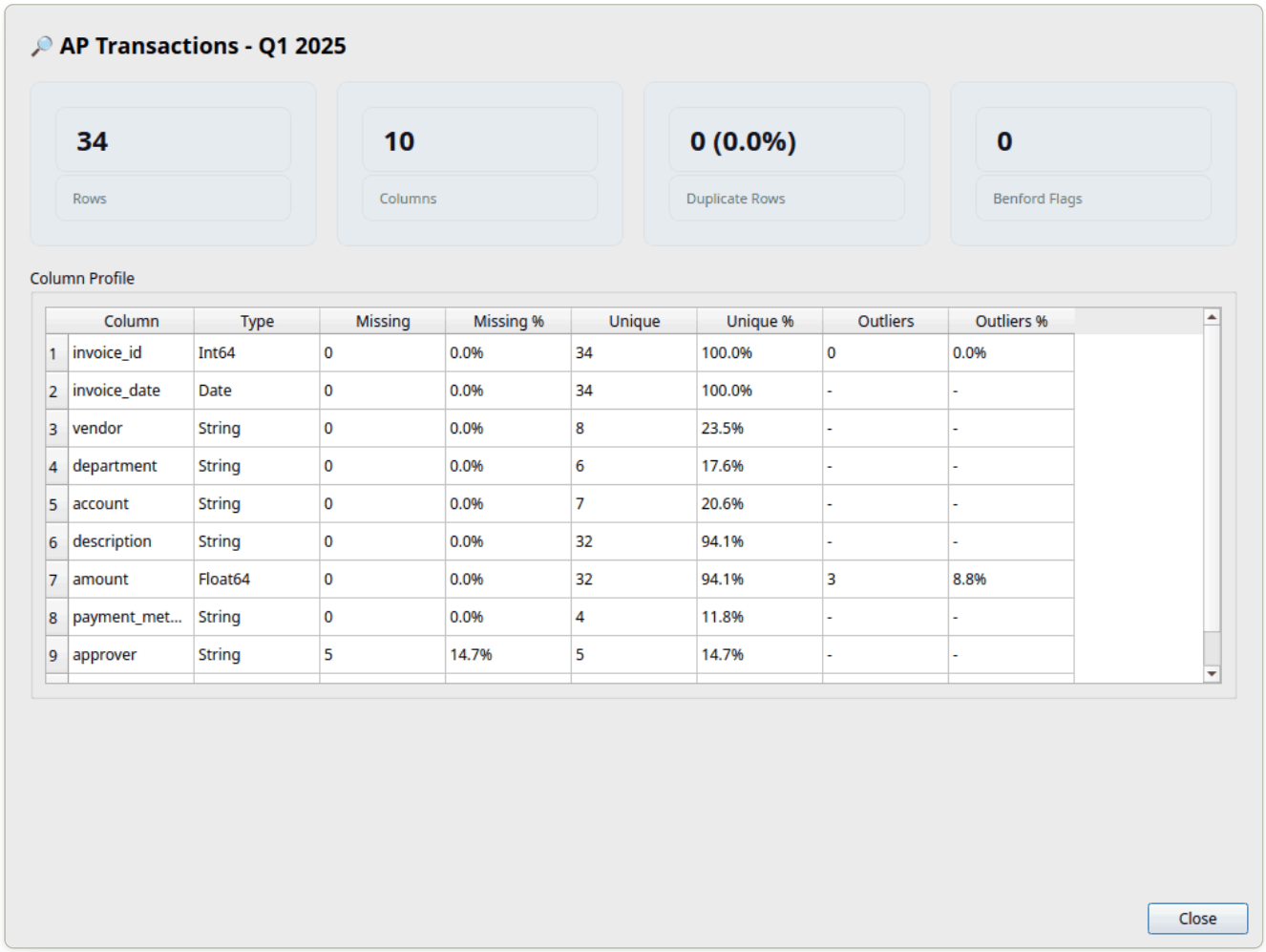
CaatMaster provides an automatic profiling command, reached from the sidebar as Profile Dataset, that runs the moment it is invoked against the currently open dataset, without requiring any parameters to be set in advance. The profile examines every column, up to a limit of fifty columns for a very wide dataset, and reports, for each one, its declared data type, the count and percentage of missing values, and the count and percentage of distinct values. For any column recognized as numeric, the profile additionally computes the interquartile range and reports how many of that column’s values fall outside 1.5 times the interquartile range beyond the first and third quartiles, the same fence used throughout this book’s outlier detection chapter.

The profile goes further than a per-column summary in two respects that make it genuinely useful as a starting point rather than a mechanical listing. First, for every numeric column with at least one hundred non-zero values and enough spread in magnitude to be a plausible amount field, rather than an identifier or a code, the profile computes Mark Nigrini’s mean absolute deviation statistic against Benford’s Law, the same conformity measure used in professional forensic accounting practice, and reports any column whose deviation exceeds the conventional threshold of 0.012, which Nigrini’s own published guidance treats as the boundary between acceptable and marginal conformity. Second, the profile computes the pairwise correlation between every two numeric columns and reports any pair whose correlation, in absolute value, reaches 0.8 or higher, which frequently surfaces the fact that two supposedly independent

columns are actually derived from one another, or that a column believed to carry new information is largely redundant with one already understood.

Every figure the profile reports is computed directly from the data at the moment the command is run. None of it is estimated, inferred, or drawn from a general model of what a typical dataset looks like.

1. With the AP Transactions dataset open, select it in the sidebar dataset list if it is not already the active dataset.
2. Under Analyze in the sidebar, click `Profile Dataset`. No parameters are required; the command runs immediately against the dataset's current state.
3. Review the summary tiles at the top of the resulting dialog, then scroll the Column Profile table for any row whose Missing, Unique, or Outliers percentage stands out.



	Column	Type	Missing	Missing %	Unique	Unique %	Outliers	Outliers %
1	invoice_id	Int64	0	0.0%	34	100.0%	0	0.0%
2	invoice_date	Date	0	0.0%	34	100.0%	-	-
3	vendor	String	0	0.0%	8	23.5%	-	-
4	department	String	0	0.0%	6	17.6%	-	-
5	account	String	0	0.0%	7	20.6%	-	-
6	description	String	0	0.0%	32	94.1%	-	-
7	amount	Float64	0	0.0%	32	94.1%	3	8.8%
8	payment_met...	String	0	0.0%	4	11.8%	-	-
9	approver	String	5	14.7%	5	14.7%	-	-

Figure 5.1 — Profile Dataset run against ap_transactions.csv. The amount column shows three outliers (8.8 percent) under the interquartile fence; the approver column shows five missing values (14.7 percent). Benford Flags reads zero, not because the amount column conforms to Benford's Law, but because thirty-four values fall well short of the one-hundred-value minimum described in Chapter 8 — the profile correctly declines to score a column it cannot evaluate reliably, rather than reporting a false conformity result.

Field Statistics: A Deeper Descriptive Layer

Where the automatic profile gives a fast overview, the Field Statistics operation, found in the Transform view under the statistics category, produces a considerably more detailed report and, unlike the sidebar profile command, becomes a recorded step in the dataset's Applied Steps pipeline, which means it can be saved and reproduced against a later extract of the same data. For every selected column, or every column in the dataset if none are specifically selected, Field Statistics reports the count, the missing count and percentage, and the number of distinct values, matching the automatic profile. For numeric columns, it goes considerably further, reporting the minimum, maximum, range, and sum, the mean and median, the sample standard deviation and variance, skewness and kurtosis where the column has enough values to

compute them meaningfully, the fifth, twenty-fifth, fiftieth, seventy-fifth, and ninety-fifth percentiles, and the coefficient of variation, which expresses the standard deviation as a proportion of the mean and allows the spread of columns measured in different units to be compared on a common footing. Every column, numeric or not, additionally reports its most frequently occurring value and how often it occurs, alongside the least frequently occurring value.

Skewness and kurtosis deserve a brief explanation for a reader who has not encountered them before. Skewness measures whether a distribution leans further in one direction than the other around its center; a strongly positive skew in an expense account, for example, suggests a small number of unusually large transactions pulling the distribution's tail to the right, which is itself worth investigating. Kurtosis measures how much of a distribution's variance comes from occasional extreme values relative to frequent, modest ones; a distribution with high kurtosis warns a reviewer that summary statistics such as the mean are being disproportionately influenced by a small number of large transactions rather than describing the typical case.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a reviewer opens a newly imported accounts payable extract of eighteen thousand rows and runs Profile Dataset before deciding which tests to prioritize. The profile reports that the Vendor ID column is ninety-nine point four percent complete, which is unremarkable, but that the Approval Date column is only sixty-one percent complete, meaning close to seven thousand payments carry no recorded approval date at all. The profile also reports that the Payment Amount column shows a Benford deviation of 0.021, above the 0.012 conformity threshold, and that Payment Amount and Invoice Amount correlate at 0.97, which is expected for two columns that should nearly always match, but is worth confirming rather than assuming.
>
> None of these three observations is, by itself, a finding severe enough to report. Together, they tell the reviewer where the engagement's substantive testing should concentrate: a completeness test on Approval Date, described in the next chapter, and a closer Benford's Law analysis on Payment Amount, described in Chapter 8, are now the natural next steps rather than an arbitrary starting point chosen without evidence.

Interpreting the Result

WHEN A PROFILE SURFACES AN UNUSUAL PATTERN

1. **Benign explanation.** A high missing-value percentage in a specific column often reflects a field that a source system only began populating partway through the period under review, or a field that is genuinely optional for a subset of legitimate transaction types.

2. **Error or judgment lapse.** A near-perfect correlation between two columns that ought to carry independent information can indicate that one column was populated by copying the other during data entry, rather than being independently recorded, which weakens any control that assumes the two values were determined separately.

3. **What would indicate something more serious.** A missing-value pattern concentrated in transactions above a specific dollar threshold, rather than spread evenly across the population, suggests the missing field was deliberately left blank to avoid triggering a review step tied to that field, and warrants a direct conversation with the process owner.

Chapter 6

Completeness and Integrity

A ratio, a trend, or a statistical test built on top of a column that is silently incomplete produces a number that is technically calculated correctly and substantively wrong, because it describes only the subset of the population that happened to have a value recorded. Completeness and integrity testing exists to establish, before any other analysis proceeds, whether a column actually contains what it is supposed to contain, across the whole population, and to quantify exactly how far short of that standard it falls where it does not.

The Six Verification Checks

CaatMaster's Verify Data command applies one or more named checks to selected columns and returns a report, one row per check, rather than modifying the underlying dataset in any way. Six check types are available. A numeric check confirms that every non-null value in a column genuinely parses as a number, using the same validated parsing logic described in Chapter 3, and reports how many values fail to do so. A date check performs the equivalent test for date values. A no-nulls check simply counts how many null values a column contains, which is the most direct completeness test available. A unique check counts how many values in a column occur more than once, appropriate for any column that is supposed to serve as an identifier, such as an invoice number or an employee ID. A positive check counts how many values in a numeric column are zero or negative, useful for columns such as quantity or unit price that should never fall to zero or below under normal business circumstances. An in-range check counts how many values fall outside an auditor-specified minimum and maximum, useful for a column such as a discount percentage that has a defined valid range regardless of what the data happens to contain.

Each check reports a fail count, a plain status of pass, fail, or skipped, and a short description of what was found. A check is marked skipped, rather than failed, when the requested check cannot meaningfully apply to the column's actual data type, such as a positive-value check requested against a text column; this distinction matters because a skipped check should prompt the reviewer to reconsider whether the check was configured correctly, while a failed check is a positive finding in its own right.

Running a Verification

The Verify Data dialog allows any number of checks to be assembled against any combination of columns in a single pass, producing one consolidated report rather than requiring each check to be run and reviewed separately. A typical completeness pass against a general ledger extract, for example, would combine a no-nulls check against the account code, the posting date, and the amount columns, a unique check against the journal entry number, and a positive check against the amount column if the ledger convention in use records debits and credits as separate signed columns rather than a single unsigned amount with a type indicator.

1. Under Analyze in the sidebar, click `Verify Data` to open the dialog against the active dataset.
2. In the Add Expectation row, set Column to `approver` and Check to `no_nulls`, then click `+ Add`.
3. Repeat for Column `invoice_id` with Check `unique`, and again for Column `amount` with Check `positive`. All three now appear in the Expectations To Check table.
4. Click `Run Verification`. The Results table below populates with one row per check.

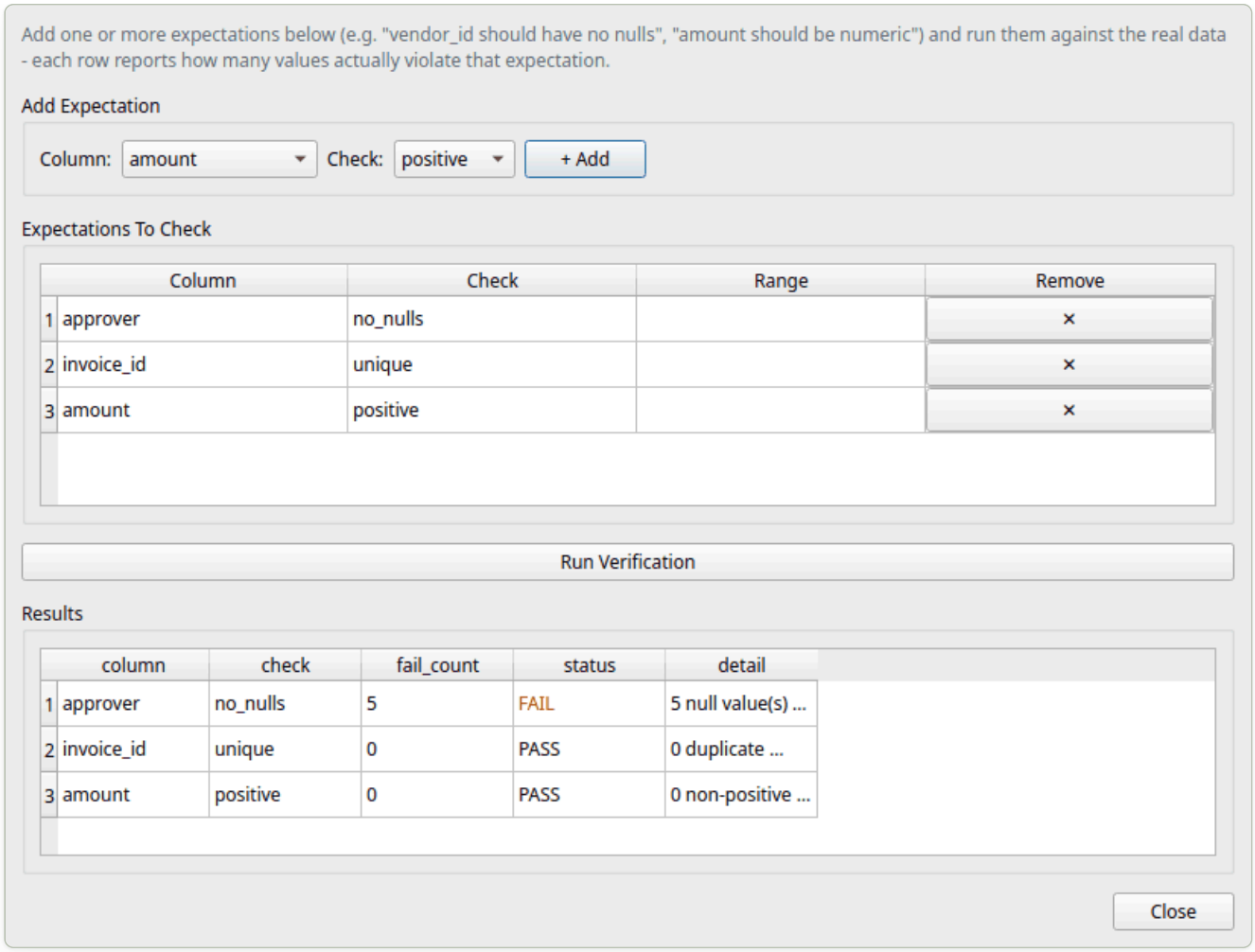


Figure 6.1 — Verify Data run against ap_transactions.csv with three checks configured. The approver column fails its no-nulls check with five violations, matching the five blank approver cells visible in Figure 2.1; the other two checks pass cleanly.

The result matches what a direct inspection of the working file would show: five of the thirty-four records carry no approver, a fourteen point seven percent gap. Every invoice ID is unique and every amount is strictly positive, so those two checks pass without qualification.

A SCENARIO WORTH RECOGNIZING

Consider a payroll extract of four thousand two hundred employee records submitted ahead of a compensation review. A no-nulls check against the Employee ID, Base Salary, and Department columns reports zero failures on all three, which might reasonably be read as a clean population. A unique check against Employee ID, however, reports eleven duplicate occurrences, meaning eleven employee identifiers appear more than once in the file. On inspection, each duplicated identifier corresponds to an employee who transferred between departments mid-period and was exported once under each department's payroll run, rather than once under a single consolidated record.

A reviewer who accepted the no-nulls result alone and moved directly to a compensation-by-department analysis would have counted each of these eleven employees twice, inflating the department headcount and, depending on how the analysis aggregated pay, potentially double-counting their compensation as well. The uniqueness check is what surfaces a completeness problem that a nulls check, on its own, cannot detect, since none of the affected fields were actually missing a value.

Interpreting the Result

WHEN A VERIFICATION CHECK REPORTS FAILURES

1. **Benign explanation.** A field may legitimately be blank for a specific transaction type, such as a discount percentage on an invoice that carries no discount, and a no-nulls failure on that field may simply reflect an overly broad check rather than a real data quality problem.
2. **Error or judgment lapse.** A recurring duplicate identifier, as in the payroll example above, more often reflects a limitation in how a source system exports data under a specific business event, such as a mid-period transfer, than any intent to misstate the underlying records.
3. **What would indicate something more serious.** A pattern of missing or duplicated values concentrated among a specific approver, vendor, or account, rather than distributed in a way consistent with the stated benign explanation, warrants direct follow-up with whoever controls that segment of the process.

Chapter 7

Cross-Tabulation and Grouping

Many of the questions an audit actually needs answered are not about a single column in isolation, but about how two attributes relate to one another across a population. Do refunds cluster among a small number of employees. Does a particular approval status appear disproportionately often for a particular vendor. A cross-tabulation, sometimes called a contingency table, answers exactly this kind of question by counting, or aggregating, how often each combination of two categorical values occurs together.

Running a Cross-Tabulation

The Cross-Tabulation operation, found in the Transform view under the statistics category, takes a row variable and a column variable, both categorical, and produces a table with one row per distinct value of the row variable and one column per distinct value of the column variable. By default, each cell in the resulting table counts how many records share that particular combination of row and column values. An optional value column and aggregation function, chosen from sum, mean, minimum, or maximum, replace the count with an aggregate of a numeric column instead, which turns the same structural device into a tool for comparing, for example, total transaction value by employee and by transaction type, rather than merely the number of transactions.

An optional percentage display recasts each cell as a percentage rather than a raw count or aggregate, and this option accepts a choice of denominator: a percentage of the table's grand total, a percentage within each row, or a percentage within each column. This choice changes the question the table actually answers, and choosing incorrectly is one of the more common ways a cross-tabulation is misread. A percentage of the row total answers what share of a given employee's transactions fall into each category; a percentage of the column total answers what share of a given category's transactions belong to each employee. The two are not the same, and confusing them can lead a reviewer to describe a finding in a way that overstates or understates its actual significance.

Cross-Tabulation has no button in the sidebar, but it is reachable two ways: from the top menu bar's Statistics menu, or the way most of CaatMaster's deeper analytical operations can also be

reached, by right-clicking directly on a column header in the data grid. This chapter uses the right-click path, since it is the one that generalizes to the largest number of operations covered in this book.

1. In the Transform view, right-click the header of any column in the AP Transactions dataset. A menu opens with quick actions at the top (Sort, Remove, Visualize) followed by every operation in the application, grouped into submenus that mirror the top menu bar's categories, including a `Statistics` and a `Financial` submenu; a handful of operations under each, covered individually across this book, are additionally promoted to their own top menu bar entries, but most of the right-click catalog still has no other path.
2. Choose `Statistics → Cross-Tabulation...`. Which column was right-clicked does not matter; the dialog that opens lets any column be chosen for any role.
3. Set Row Variable to `department` and Column Variable to `status`.
4. Set Value Column to `amount` and Aggregation Function to `sum`, so each cell totals dollar value rather than counting transactions.
5. Click `Generate Crosstab`.

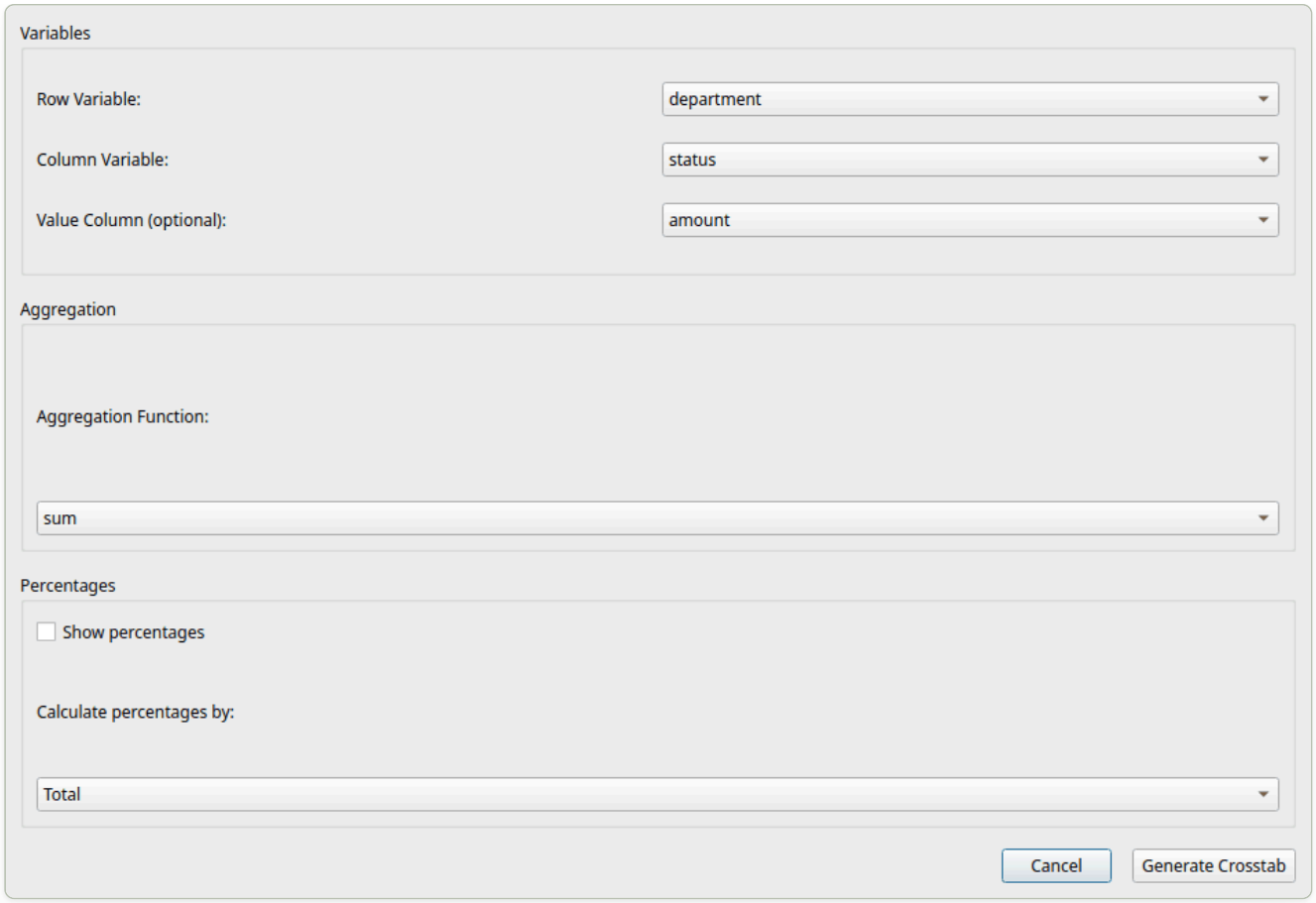


Figure 7.1 — Cross-Tabulation configured to sum approved, pending, and rejected amounts by department.

Department	Approved	Pending	Rejected
Operations	3,849.60	1,498.40	0.00
IT	66,675.00	0.00	0.00
Legal	11,770.00	0.00	0.00
Facilities	11,496.80	0.00	0.00
Marketing	3,525.00	0.00	290.00
Finance	24,875.00	0.00	0.00

Figure 7.2 — The resulting cross-tabulation. IT carries the largest approved total, driven by the network switch and server rack invoices flagged as outliers in Chapter 10.

A SCENARIO WORTH RECOGNIZING

Suppose an accounts payable team wants to understand whether payment approvals are concentrated among too few individuals for the volume of transactions being approved. A cross-tabulation with Approver as the row variable and Payment Method as the column variable, aggregating on Payment Amount with a sum function, shows that one approver, out of fourteen active approvers, accounts for sixty-two percent of all wire transfer value for the period, expressed as a percentage of the column total.

Read as a percentage of that approver's own row instead, the same underlying data might show that wire transfers make up only eighteen percent of that individual's total approval volume, meaning the concentration is specific to the wire transfer payment method rather than reflecting an approver who dominates approvals in general. Both readings are correct; they answer different questions, and a workpaper narrative needs to state clearly which one is being reported, since a reader who assumes the other interpretation could draw a materially different conclusion about the severity of the concentration.

Interpreting the Result

WHEN A CROSS-TABULATION SHOWS CONCENTRATION IN ONE CELL

1. **Benign explanation.** A single approver or employee may legitimately handle a disproportionate share of one transaction type because of role assignment, seniority, or regional responsibility, rather than any control weakness.
2. **Error or judgment lapse.** Concentration can also result from a workflow tool that defaults every transaction of a given type to a single approver's queue, regardless of that approver's intended authority, which is a control design issue rather than an individual's action.
3. **What would indicate something more serious.** Concentration that coincides with the same individual having originated or requested the underlying transactions, rather than merely approved them, raises a segregation of duties question addressed directly in Chapter 12.

Chapter 8

Benford's Law

Benford's Law describes an observation, first published by the physicist Frank Benford in 1938 and later brought into forensic accounting practice by Mark Nigrini, that in many naturally occurring collections of numbers, the leading digit is not uniformly distributed across the values one through nine. The digit one appears as the leading digit roughly thirty percent of the time, the digit nine appears as the leading digit less than five percent of the time, and the frequencies of the digits in between decline steadily according to a logarithmic formula. This pattern holds for collections of numbers that span several orders of magnitude and arise from an unconstrained natural process, such as transaction amounts across a large general ledger, and tends to break down for numbers that are artificially constrained, such as prices deliberately set just under a round figure, or numbers that are assigned rather than measured, such as invoice numbers or telephone extensions.

The value of Benford's Law to an auditor lies in its use as a screening test. A dataset of transaction amounts that conforms closely to the expected distribution provides mild comfort that the values were not extensively fabricated by a person choosing numbers that felt plausible, since people are demonstrably poor at manufacturing numbers with the correct leading-digit frequencies. A dataset that deviates substantially from the expected distribution does not, on its own, prove fabrication; it indicates that the population is worth a closer look, using the more targeted techniques described elsewhere in this book.

Two Ways CaatMaster Applies This Test

CaatMaster applies Benford's Law in two distinct places, at two distinct levels of granularity, and the difference between them matters for how each is used. The automatic Profile Dataset command, described in Chapter 5, applies a whole-column conformity test: it computes Nigrini's mean absolute deviation statistic for a candidate amount column and reports whether that single summary number exceeds the 0.012 threshold treated in Nigrini's published guidance as the boundary of acceptable conformity. This is a fast, automatic screen that answers one question: does this column, taken as a whole, look like the kind of population Benford's Law would predict.

The dedicated Benford's Law Analysis operation, found in the Transform view, answers a more granular question and is the tool to reach for once the whole-column screen has indicated that a closer look is warranted. It computes, for every row, the value's leading digit or, optionally, its leading two digits, adds that digit as a new column, and then computes two further columns: the frequency with which that specific leading digit actually occurs across the whole dataset, and the absolute difference between that actual frequency and the frequency Benford's Law predicts for it. Because this deviation is attached to every row individually, a reviewer can sort or filter the dataset by deviation and go directly to the transactions carrying the most overrepresented leading digits, rather than working only with a single column-level verdict.

Running a Benford's Law Analysis

Selecting the operation requires only a numeric column to analyze. An option to analyze the leading two digits rather than the leading one digit narrows the test considerably, since the two-digit version compares against ninety possible values, from ten through ninety-nine, rather than nine, and is appropriate once a population is large enough, generally several thousand values or more, to populate each of those ninety buckets meaningfully. A further option restricts the digit extraction to a value's significant digits only, which matters for a column recorded to a fixed number of decimal places, where the digits immediately after a decimal point would otherwise be treated as leading digits in a way that does not reflect the underlying magnitude of the transaction.

1. From the top menu bar, choose `Audit → Audit Checks → Benford's Law Analysis...`. The same dialog is also reachable by right-clicking any column header and choosing `Audit → Benford's Law Analysis...` from the full operation menu described in Chapter 7; the top menu bar simply promotes this operation, along with the rest of the Audit Checks group, to a permanent, one-click location since it is used so frequently.
2. Set Column to Analyze to `amount` and leave Leading Digits at its default of `1 (First digit only)`.
3. Click `Run Analysis`.

Run against the thirty-four-row AP Transactions working file, this last step does not produce a chart. It produces a warning.

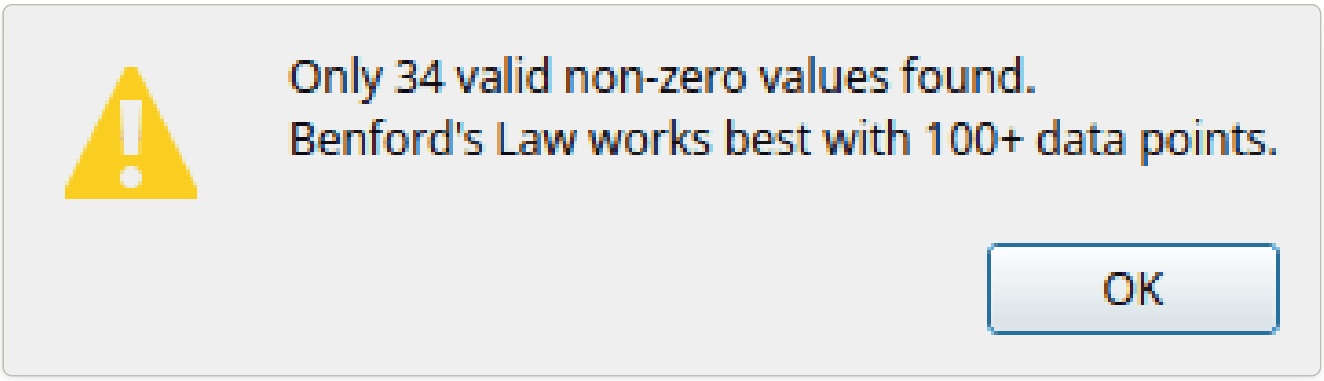


Figure 8.1 — CaatMaster declining to run a Benford’s Law analysis against a population of thirty-four values. This is an enforced floor, not a soft recommendation: the dialog will not produce a chart or a deviation figure below one hundred valid, non-zero values, regardless of what the reviewer requests.

To show what a completed analysis looks like, the screenshot below runs the same dialog against a larger, separately generated population of five hundred transaction amounts built to resemble the kind of spread a real accounts payable ledger would show. This population exists only to illustrate the technique at a workable scale and is not part of the working file introduced in Chapter 2.

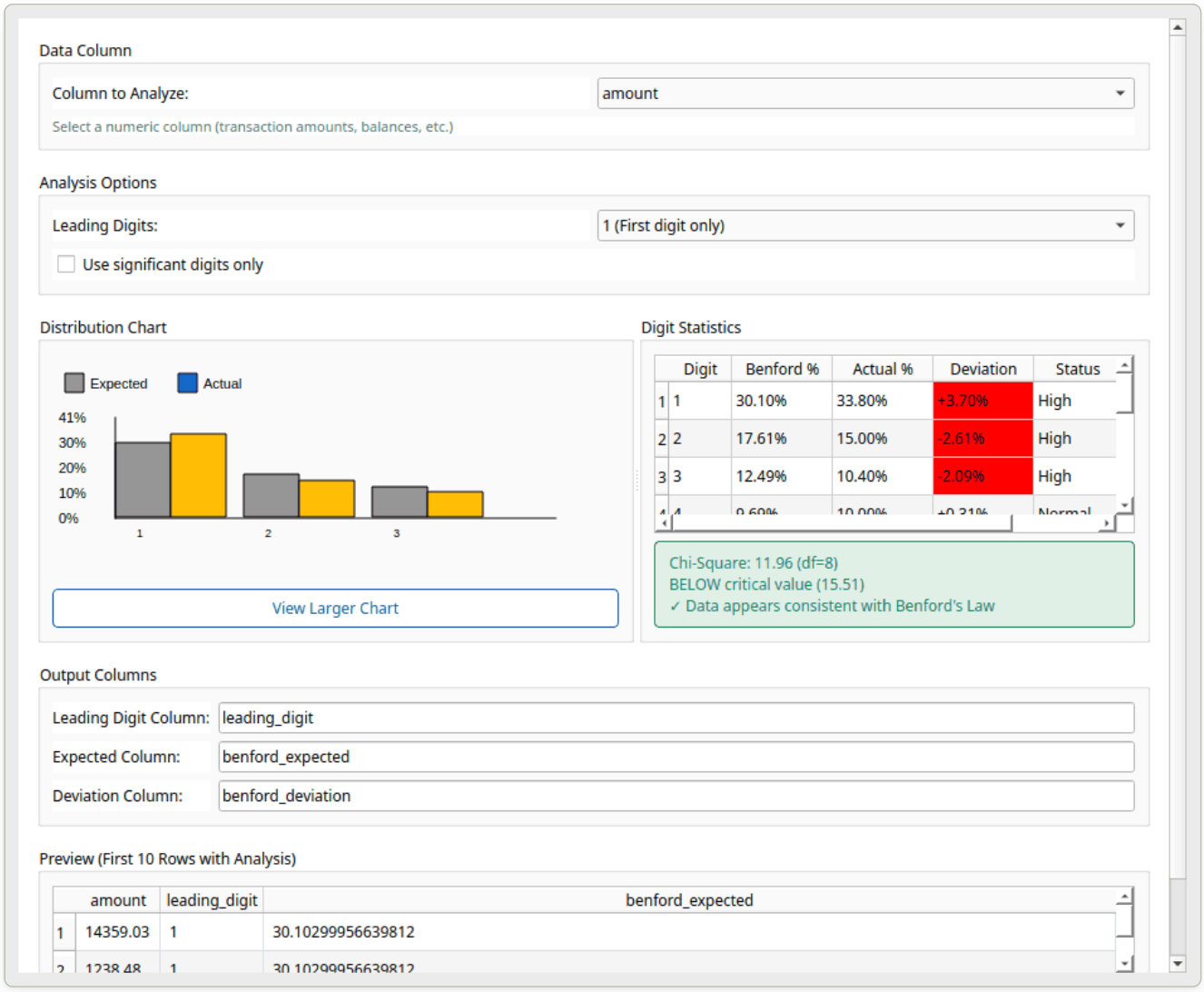


Figure 8.2 — A completed Benford's Law analysis. Beyond the per-digit deviation table, CaatMaster also runs a chi-square goodness-of-fit test and reports whether the total deviation across all nine digits falls below the critical value for statistical significance — in this run, a chi-square statistic of 11.96 against eight degrees of freedom, below the 15.51 critical value, indicating the population is statistically consistent with Benford's Law overall even though individual digits, such as digit 1 at plus 3.70 percentage points, show a visible local deviation.

A SCENARIO WORTH RECOGNIZING

Suppose a procurement audit runs the two-digit Benford analysis against a purchase order value column of nine thousand records. The result shows the leading digits forty-nine and ninety-nine occurring far more often than Benford's Law predicts, at frequencies roughly three times the expected rate, while every other leading two-digit combination sits close to its expected frequency. A reviewer filtering the dataset to purchase orders beginning with these two digit combinations finds that the great majority of them fall just under two internal approval thresholds set at five thousand dollars and ten thousand dollars, meaning the actual values cluster near four thousand nine hundred and nine thousand nine hundred.

This is precisely the pattern Benford's Law is best at surfacing: values that are not randomly generated but deliberately positioned just beneath a control threshold, whether to avoid a required second approval or simply because a requester rounded down from a mental estimate of the threshold without any intent to circumvent it. The Benford test cannot distinguish between these two explanations on its own; it can only point to where the concentration lies.

Limits of the Technique

A leading-digit test is not appropriate for every numeric column, and applying it indiscriminately produces false alarms that waste review time. Columns that are assigned rather than measured, such as invoice numbers, employee identifiers, or check numbers, do not follow Benford's Law and should not be tested against it. Columns with an artificially narrow range, such as a discount percentage that can only take values between zero and twenty, likewise fall outside the conditions under which the law is expected to hold. Nigrini's own published guidance treats several hundred values as the practical minimum for a reliable result; CaatMaster enforces a stricter floor directly in the dialog, refusing to run the dedicated Benford's Law Analysis operation at all below one hundred valid, non-zero values, as Figure 8.1 shows. A reviewer working with a smaller population is not without options: the population can be widened, for instance by extending the period under review, or the test can be deferred until enough transactions have accumulated to apply it meaningfully.

Interpreting the Result

WHEN A COLUMN DEVIATES FROM THE EXPECTED DISTRIBUTION

1. **Benign explanation.** A column that mixes several genuinely different populations, such as invoice amounts from two business units with very different average transaction sizes, can deviate from Benford's Law even though neither population, considered on its own, is unusual.

2. **Error or judgment lapse.** Values clustered just under a threshold, as in the case above, more often reflect informal rounding by requesters familiar with an approval limit than deliberate structuring, particularly where the clustering is broad and involves many different individuals.

3. **What would indicate something more serious.** Clustering concentrated among a small number of requesters or approvers, rather than spread broadly across the population, together with values that fall just barely under the threshold rather than comfortably under it, is the pattern most consistent with deliberate avoidance of a control and warrants direct inquiry.

Chapter 9

Ratio and Trend Analysis

A single financial figure, taken alone, rarely tells an auditor very much. Revenue of forty-two million dollars is neither large nor small without a point of comparison. A ratio supplies that point of comparison by relating one figure to another in a way that is meaningful regardless of the absolute size of the business being examined, and a trend supplies a second kind of comparison by relating a figure to its own past values over time. Chapter 3 addressed the precondition both techniques depend on: a ratio or a trend calculated from a badly parsed numerator or denominator is worse than no ratio at all, because it carries the appearance of precision without the substance of it.

The Working Financial Dataset

This chapter introduces the second recurring dataset used in this book, alongside the accounts payable extract from Chapter 2. The file, quarterly_financials.csv, holds four rows, one per quarter of Meridian Office Supply Co.'s 2024 fiscal year, already arranged in the wide, one-row-per-period layout that CaatMaster's ratio calculation expects, with a separate column for each concept: revenue, cost of goods sold, gross profit, net income, EBITDA, current assets, current liabilities, cash, inventory, total assets, total liabilities, total equity, and interest expense. A statement exported in the more common long format, one row per account per period, is brought into this shape first using the Reshape Statement to Ratio Inputs operation, or through the mapping step built into the Financial Import Wizard from Chapter 3.

The Four Families of Ratio

CaatMaster's Financial Ratios feature groups its calculations into four families, labeled in the dialog itself as Liquidity, Solvency, Profitability, and Efficiency, matching the categories used throughout financial statement analysis generally. Liquidity ratios, comprising the current ratio, the quick ratio, and the cash ratio, measure a company's capacity to meet short-term obligations as they come due, each one progressively stricter about which current assets it counts as available to do so. Solvency ratios, comprising the debt-to-equity ratio, the debt ratio, and interest coverage, measure how heavily a company relies on borrowed funds relative to its own capital, and how comfortably its earnings cover the interest expense that borrowing

creates. Profitability ratios, comprising net profit margin, gross margin, return on assets, return on equity, and EBITDA margin, measure how effectively a company converts revenue and assets into earnings. Efficiency ratios, represented in the current release by asset turnover, measure how effectively a company uses its asset base to generate revenue.

Every ratio in every family is calculated with the same safeguard described in Chapter 3: a denominator of exactly zero produces a genuinely missing result rather than an unbounded or undefined number silently propagating into every calculation and chart that later depends on it. A reviewer examining a ratio report and finding a blank value where a ratio would normally appear should read that blank as an honest signal that the denominator was zero for that period or entity, not as a defect in the calculation.

One formula is worth stating precisely rather than left to assumption, since it departs from the version printed in most introductory textbooks. Interest coverage in CaatMaster is calculated as net income plus interest expense, divided by interest expense, rather than the more common earnings-before-interest-and-tax divided by interest expense. Because net income has already had tax deducted from it, this version of the ratio runs somewhat lower than the textbook formula would produce from the same underlying figures. A reviewer citing this figure in a workpaper should describe it as calculated, not assume it matches a number pulled from a different source without checking the formula behind it.

Running a Ratio Analysis

Calculating a ratio in CaatMaster begins with mapping the standard financial concepts each ratio depends on, such as current assets, current liabilities, or net income, to the actual column names present in the imported dataset. Where a financial statement was brought in through the Financial Import Wizard described in Chapter 3, or through the XBRL import pipeline described in Chapter 22, this mapping is proposed automatically based on the recognized concept and can be adjusted before any ratio is calculated.

1. With the quarterly_financials dataset open in the Transform view, right-click any column header and choose `Financial → Financial Ratios...`. The same dialog is also reachable from the top menu bar's `Financial` menu; Financial Ratios has no button in the sidebar.

2. Click `Auto-Detect Income`, `Auto-Detect Balance`, and `Auto-Detect Cash` in turn, so CaatMaster proposes a mapping from each recognized concept to the matching column name. Review the proposed mapping for each statement section before continuing.

3. Under Select Ratios to Calculate, leave the default selection of twelve ratios checked across all four families.

4. Click `Calculate Ratios`. CaatMaster appends one new column per selected ratio to the dataset, leaving the original reported figures untouched alongside the calculated results.

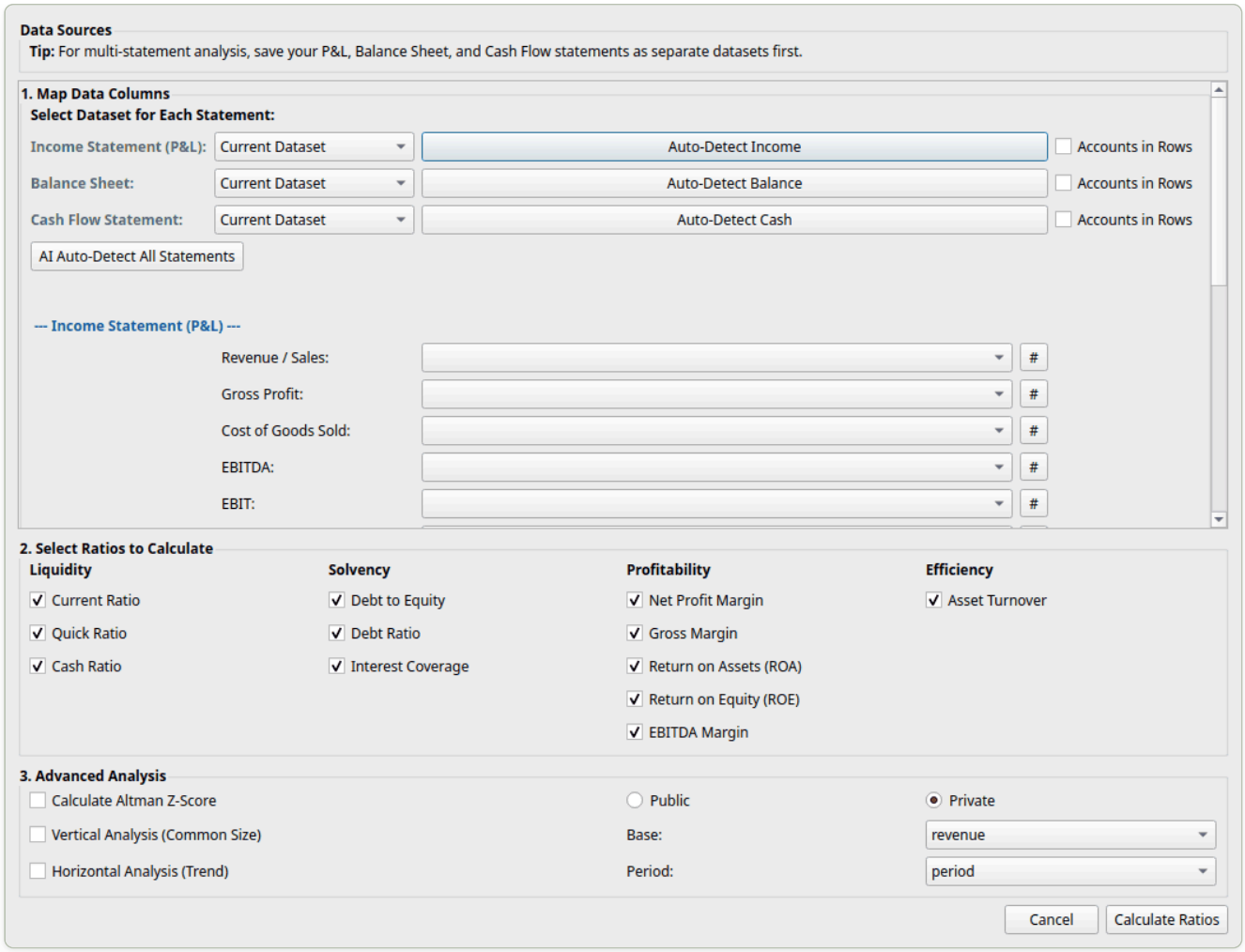


Figure 9.1 — The Financial Ratios dialog. Section 1 maps statement line items to standard concepts; Section 2 selects which ratios to calculate, grouped exactly as described above; Section 3 offers three further analyses covered later in this chapter.

Ratio	Q1	Q2	Q3	Q4
Current ratio	1.600	1.553	1.488	1.420
Quick ratio	1.000	0.921	0.817	0.727
Debt to equity	1.164	1.175	1.210	1.245
Interest coverage	6.278	5.757	4.789	4.283
Net profit margin	9.0%	8.1%	6.8%	5.9%
Gross margin	40.0%	39.4%	38.0%	37.4%
Return on assets	6.6%	5.9%	4.7%	4.2%
Return on equity	14.2%	12.8%	10.4%	9.4%
EBITDA margin	14.3%	13.7%	12.7%	11.6%
Asset turnover	0.724	0.725	0.698	0.706

Figure 9.2 — Every ratio CaatMaster calculates from quarterly_financials.csv, computed directly by the actual application code against the working file, not estimated for illustration.

Three Further Analyses

The Financial Ratios dialog offers three additional analyses beyond the twelve core ratios. Vertical analysis, sometimes called common-size analysis, restates every line item as a percentage of a chosen base figure, typically revenue for an income statement or total assets for a balance sheet, which allows statements of very different sizes to be compared directly. Horizontal analysis restates each line item as a percentage change from a chosen base period, which is a more formal, statement-wide counterpart to the period-by-period Trend Analysis described later in this chapter.

The Altman Z-Score is a bankruptcy-risk model, developed by Edward Altman in 1968 and still in wide professional use, that combines five weighted ratios, working capital to total assets, retained earnings to total assets, earnings before interest and tax to total assets, equity to total liabilities, and revenue to total assets, into a single score, with separate coefficient sets and interpretation thresholds depending on whether the company being assessed is publicly traded. Two aspects of CaatMaster's implementation deserve a direct note of caution. If retained earnings is not separately mapped, the calculation substitutes one-half of total equity as a rough estimate, rather than leaving the ratio blank. If a distinct earnings-before-interest-and-tax figure is not mapped, the calculation substitutes EBITDA in its place, which is consistently higher than EBIT and therefore biases the score toward the safer end of its scale. A reviewer relying on the Z-Score for a genuine going-concern assessment should confirm that both retained earnings and EBIT, not EBITDA, were mapped explicitly, rather than accept the fallback values without checking.

Trend Analysis Across Periods

Trend Analysis takes a date column and a value column and aggregates the value by calendar period, using day, week, month, quarter, or year as the grouping unit, with a choice of sum, mean, count, minimum, or maximum as the aggregation applied within each period. This produces a compact time series suitable for charting through the Visualizations view covered in Chapter 27, and is the appropriate first step whenever a question concerns change over time

rather than a single point-in-time comparison, such as whether a particular expense category has been growing faster than revenue over successive quarters.

Grouping the Inventory column of quarterly_financials.csv by quarter and summing within each period shows inventory rising from 210,000 in the first quarter to 305,000 in the fourth, an increase of forty-five percent, while the same grouping applied to Revenue shows growth of only five percent over the same four quarters. This is precisely the pattern behind the widening gap between the current ratio and the quick ratio in Figure 9.2: inventory, not a broad decline across every current asset, is what is pulling working capital down.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a working capital review extends the same kind of analysis shown in Figure 9.2 across eight quarters rather than four, for a distributor considerably larger than Meridian Office Supply Co. The current ratio, calculated the same way, declines steadily from 1.85 to 1.12 over that period, while the quick ratio declines much more sharply, from 1.20 to 0.58. Because the quick ratio excludes inventory from current assets while the current ratio includes it, the gap widening between the two ratios points specifically at inventory as the driver of the decline, rather than a broad deterioration across every current asset category, exactly as in the four-quarter working example above but at a scale severe enough to warrant a formal finding.
>
> A trend analysis of the Inventory column alone, grouped by quarter, confirms that inventory has grown forty-one percent over the same eight quarters while revenue, aggregated over the same periods, grew only six percent. Inventory accumulating well ahead of the sales activity needed to justify it is a classic indicator worth investigating further, whether the underlying cause turns out to be a genuine demand slowdown that management has not yet reflected in a valuation allowance, a data entry issue overstating quantities on hand, or slow-moving stock that should have been written down under the analysis covered in Chapter 19.

Interpreting the Result

WHEN A RATIO OR A TREND MOVES OUTSIDE ITS EXPECTED RANGE

1. **Benign explanation.** A ratio can move because of a genuine, disclosed business change, such as a planned inventory build ahead of a known seasonal peak, that has nothing to do with reporting quality.
2. **Error or judgment lapse.** A sudden, isolated jump in a ratio between two adjacent periods, rather than a gradual trend, often traces back to a misclassification between two account categories, such as a current asset booked as non-current, rather than a change in the underlying business.
3. **What would indicate something more serious.** A ratio that moves gradually in a direction management has strong incentive to avoid disclosing, such as a debt covenant ratio drifting toward its contractual limit over several consecutive periods without any corresponding management commentary, warrants direct inquiry into why the trend has not been addressed.

Chapter 10

Outlier and Anomaly Detection

An outlier is a value that sits meaningfully apart from the rest of its population, and detecting one is only useful once a reviewer has settled on a defensible definition of how far apart is meaningful. CaatMaster offers two statistically grounded definitions for a quick, single-column screen, a dedicated round-number test aimed at a specific and well-documented manipulation pattern, and a set of trainable machine learning models for anomaly detection that considers more than one column of context at a time.

Two Statistical Definitions of Unusual

The interquartile range method defines the bulk of a population as the range between its first and third quartiles, and flags any value falling more than one and a half times that range below the first quartile or above the third as an outlier, the same convention, sometimes called Tukey's fences, used in the box plots taught in most introductory statistics courses. This method has the advantage of being resistant to the outliers themselves distorting the boundary, since quartiles are not pulled by extreme values the way a mean is.

The z-score method instead measures how many standard deviations a value sits from the population mean, and flags a value once that count exceeds a chosen threshold, three standard deviations in CaatMaster's default configuration. This method is more familiar to most reviewers but is itself sensitive to extreme values, because a small number of very large outliers can inflate the standard deviation enough to mask their own significance, and a population with a strongly skewed distribution, common in financial data where amounts cannot be negative and often span several orders of magnitude, tends to overstate how many values legitimately belong in the tail. In practice, the interquartile range method is the more conservative and generally preferable choice for financial transaction data, and the z-score method is better suited to data that is closer to normally distributed by nature, such as a physical measurement or a well-behaved ratio.

Running Outlier Detection

Audit Outlier Detection, found in the Transform view under the audit category, requires a column and a choice of method, and adds a boolean flag column together with a column recording how far each flagged value deviates from the applicable boundary, expressed in units of the interquartile range for the interquartile range method or in standard deviations for the z-score method. Because both flag and deviation columns become ordinary columns in the dataset, they can be sorted, filtered, or combined with other criteria using any of the general-purpose transformations available elsewhere in the application.

1. From the top menu bar, choose `Audit → Audit Checks → Outlier Detection...` against the AP Transactions dataset. As with Benford's Law, the same dialog is also reachable by right-clicking a column header and choosing Audit → Outlier Detection.
2. Set Column to `amount` and Method to `iqr`.
3. Leave Flag Column and Deviation Column at their defaults, `is_outlier` and `outlier_deviation`.
4. Click `OK`. The two new columns appear immediately in the data grid.

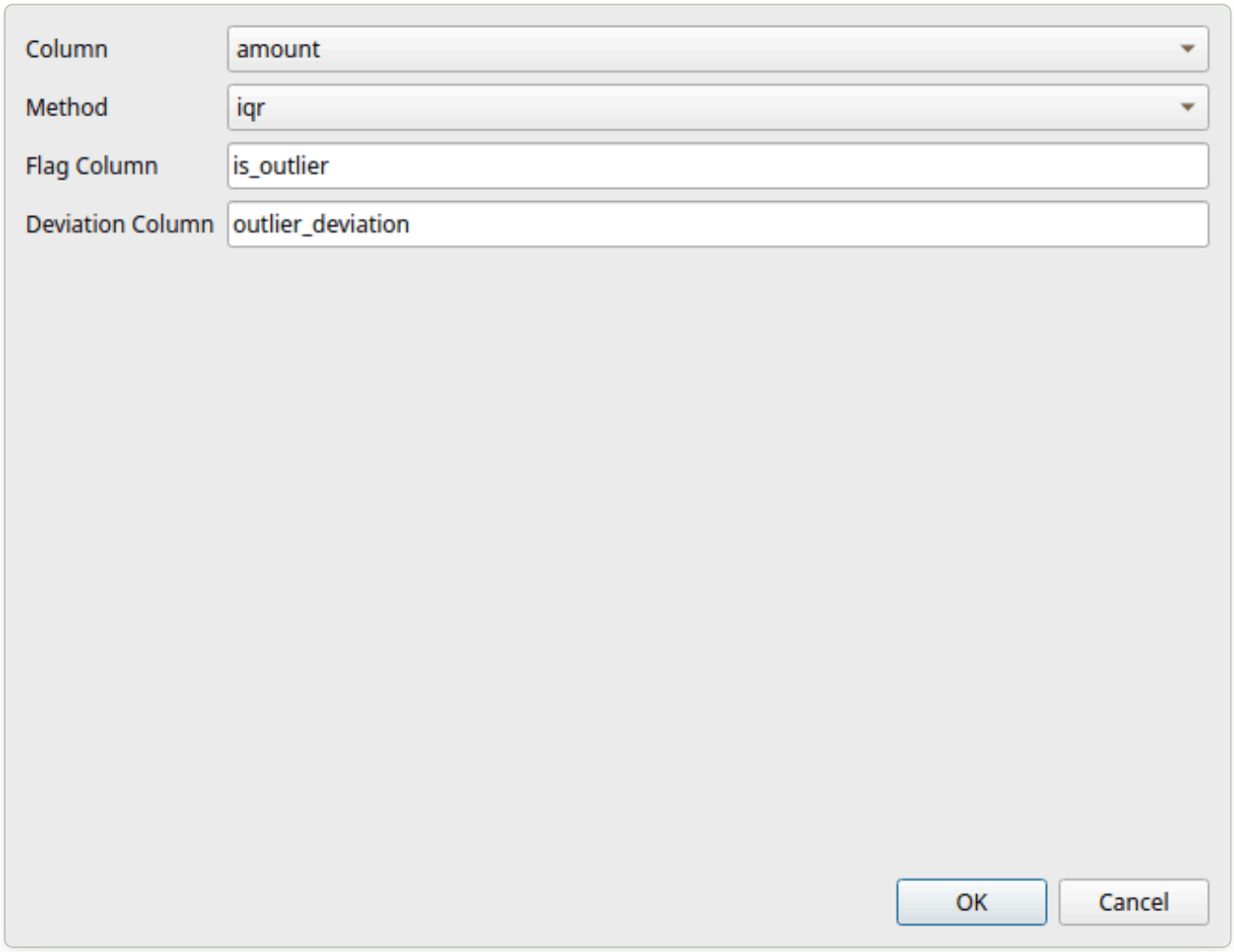


Figure 10.1 — Audit Outlier Detection configured for the interquartile range method against the amount column.

Invoice	Vendor	Amount	Deviation (in IQR)
20010	Cascade IT Solutions	9,900.00	2.70
20034	Harborview Consulting	10,000.00	2.80
20027	Cascade IT Solutions	48,750.00	41.55

Figure 10.2 — The three rows flagged by the interquartile range method against ap_transactions.csv. The server rack cooling fan invoice deviates more than forty-one interquartile ranges beyond the upper fence, an order of magnitude past the other two.

Round Number Detection

A cluster of unusually round transaction amounts is a long-recognized indicator worth checking for in its own right, since genuine transaction amounts, arrived at through pricing calculations, tax computations, and negotiated terms, tend to include cents and non-round dollar figures, while amounts entered by estimation, or fabricated without reference to an underlying calculation, disproportionately land on round figures. Round Number Detection scores each value in a column by the proportion of its digits that are trailing zeros, so that a value of five thousand, three of whose four digits are trailing zeros, scores 0.75, while a value of one hundred fifty scores approximately 0.33, and any value carrying a fractional cent amount scores zero regardless of how round its whole-dollar portion might look. A configurable threshold, set to 0.9 by default, determines how round a value must be before it is flagged.

Round Number Detection is reached the same way as the two operations above it in this chapter: Audit → Audit Checks → Round Number Analysis on the top menu bar, or the equivalent entry in a column header's right-click menu. This default threshold is calibrated with large balances in mind and should be lowered for a population of smaller transaction amounts. By construction, a four-digit whole-dollar amount can score at most 0.75, since its leading digit can never be a trailing zero, and a five-digit amount can score at most 0.8; neither could ever reach the default 0.9 threshold regardless of how round it looks to a human reviewer. Run against ap_transactions.csv with the threshold lowered to 0.7, Round Number Detection flags two records: invoice 20007, five thousand dollars even, scoring 0.75, and invoice 20034, ten thousand dollars even, scoring 0.8. Both belong to the same vendor, Harborview Consulting, which on its own is not yet a finding, but is exactly the kind of pattern worth a closer look at that vendor's invoicing history.

Machine Learning Anomaly Models

The top menu bar's ML menu includes an Anomaly Detection submenu offering Isolation Forest, Z-Score, IQR, and LOF as one-click options. All four now run the genuine algorithm their name promises: Isolation Forest and LOF call directly into the scikit-learn library's own IsolationForest and LocalOutlierFactor implementations against the single chosen column, rather than an internal stand-in. An earlier version of this submenu scored both by distance from the column mean, a method closer to the z-score technique described earlier in this chapter than to true isolation-based or density-based detection, which a reviewer selecting Isolation Forest by name had no way to know from the interface alone; that gap is closed, and every value these two entries produce, the anomaly flag and its underlying score, now comes from the real algorithm.

What still separates this submenu from the trainable models described next is scope, not authenticity. Each entry here considers exactly one column at a time: LOF, for instance, judges a value anomalous by comparing the local density of its own neighborhood against its neighbors' neighborhoods within that single column, which is genuine density-based reasoning but necessarily blind to any pattern that only shows up across several columns at once. The genuine, multivariate versions live in a separate part of the application, reached from the top menu bar through ML → Model Tools → Train Model…, which opens a dedicated training dialog rather than adding a single Applied Step. This workflow supports training an anomaly detection model across several columns at once, using the same scikit-learn library's established implementations of Isolation Forest, Local Outlier Factor, and One-Class Support Vector Machine. Unlike the single-column entries in the ML menu's Anomaly Detection submenu, these models can surface a transaction that looks unremarkable in every individual field but unusual in the combination of fields taken together, such as a transaction whose amount, time of day, and approver are each ordinary on their own but rarely occur together. A trained model can be saved and applied again to a later extract of comparable data, which makes this the appropriate technique for a recurring monitoring program rather than a one-time review.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a travel and expense review applies interquartile range outlier detection to a Claim Amount column of six thousand two hundred expense claims and flags one hundred forty of them as statistical outliers. Reviewing these flagged claims individually, the majority turn out to be legitimate, larger claims tied to international travel, easily explained by higher costs in the visited location. Separately, the same reviewer trains an Isolation Forest model across four columns at once, Claim Amount, Day of Week, Time Submitted, and Employee Department, and this model flags a much smaller set of nineteen claims, none of which were flagged by the single-column outlier test, because no individual field value was unusual. What these nineteen claims share is an unusual combination: each was submitted on a weekend, for an amount just under the two-hundred-dollar receipt-exemption threshold, by an employee in a department with no recorded weekend business activity.
>
> The two techniques answered different questions and found different things, which is the reason CaatMaster offers both rather than treating one as a replacement for the other.

Interpreting the Result

WHEN A VALUE OR A RECORD IS FLAGGED AS ANOMALOUS

1. **Benign explanation.** A statistically unusual value very often corresponds to a genuinely unusual, but entirely legitimate, business event, such as a one-time bulk purchase or an executive's international travel claim.
2. **Error or judgment lapse.** A round-number cluster can result from a policy of entering estimated amounts pending a final invoice, later corrected, rather than any attempt to misstate a balance permanently.
3. **What would indicate something more serious.** A pattern of anomalies clustered around a single individual, timing window, or combination of attributes chosen specifically to avoid a documented control threshold, as in the weekend claims example above, warrants direct inquiry rather than dismissal as statistical noise.

Chapter 11

Sampling and Stratification

Most of the techniques covered so far in this book examine an entire population at once, which is one of the principal advantages a computer-assisted audit technique holds over traditional manual testing. Sampling remains necessary all the same, whether because a specific test, such as a physical inspection or a third-party confirmation, cannot practically be performed on every item in a population, or because a statistically valid projection of a sample's results is itself the form of evidence a particular audit objective calls for. CaatMaster treats sampling as a three-stage discipline: planning how large a sample needs to be, selecting the specific items that make up the sample, and, once the sample has been examined, projecting its results back onto the full population with a defined level of statistical confidence.

Planning a Sample Size

Classical variables sample size planning answers a specific question: given an estimate of how much the population's values vary, expressed as a standard deviation, how large a sample is required to estimate the population's total value within a chosen tolerable precision, at a chosen confidence level. This is a standalone tool, separate from the operation catalog described in Chapter 7, and is reached from the sidebar rather than from any menu: under Analyze, click Variables Sampling to open the Classical Variables Sampling dialog, which holds sample size planning and sample evaluation as two tabs of the same window.

CaatMaster's sample size calculation follows the standard mean-per-unit formula taught in AICPA audit sampling guidance, taking the population size, an estimated population standard deviation drawn from a pilot sample or a prior period's results, the desired confidence level, and the tolerable precision as inputs, and applying a finite population correction that reduces the required sample size once the sample would otherwise represent a substantial share of a small population. A confidence level of ninety-five percent, the most commonly used standard in audit practice, corresponds to a fixed multiplier of 1.960 in the underlying formula; CaatMaster looks this value up directly for the confidence levels most commonly used in practice and computes it from the normal distribution for any other confidence level requested.

1. From the sidebar, under Analyze, click `Variables Sampling`.
2. On the `Calculate Sample Size` tab, enter Population Size `2100`, Estimated Population Std Dev `380`, Confidence Level `95`, and Tolerable Precision `150000`.
3. Click `Calculate Sample Size`.

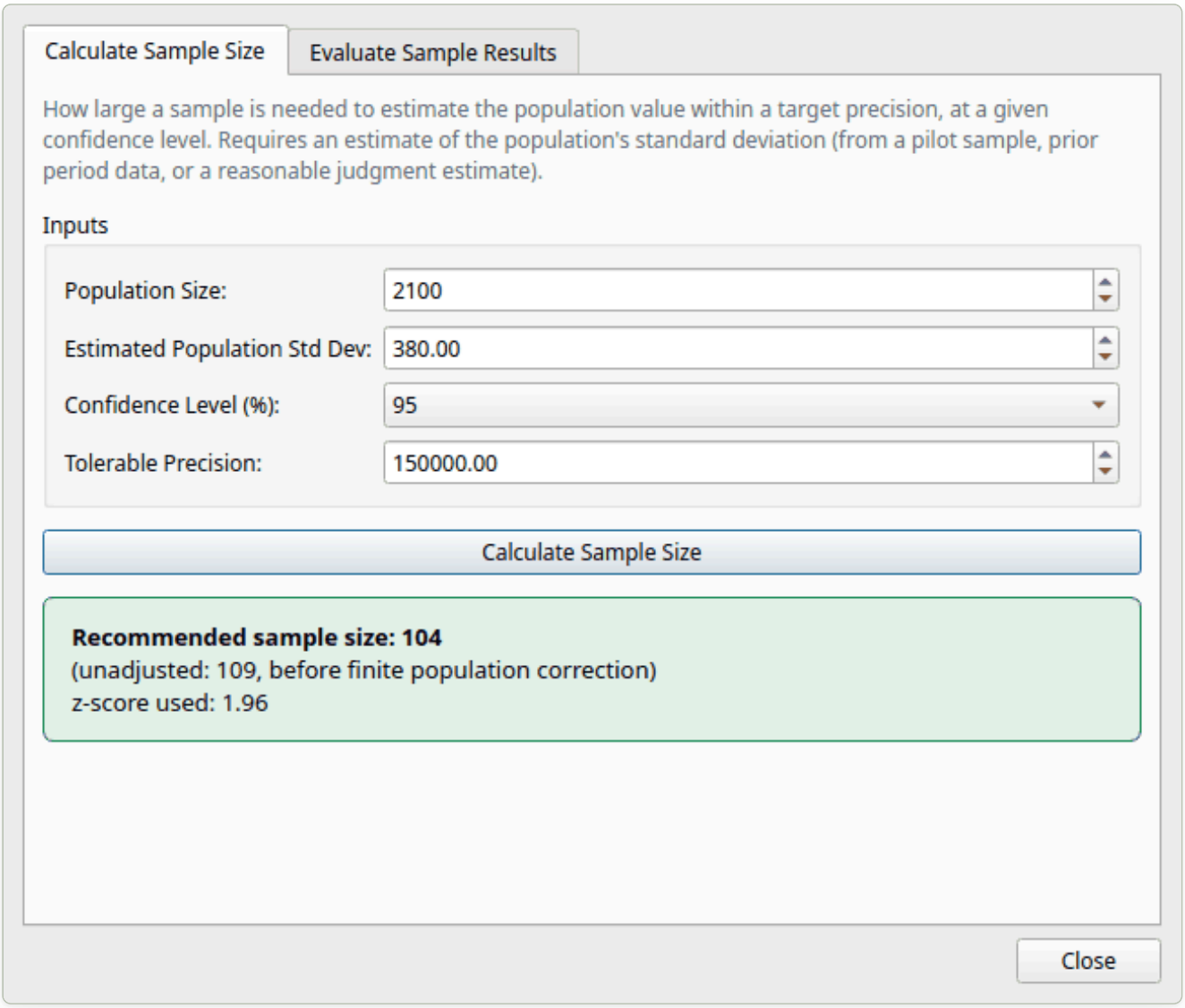


Figure 11.1 — Sample size planning for a hypothetical accounts receivable population of 2,100 invoices. A recommended sample of 104 items, out of an unadjusted 109, would satisfy a $150,000 tolerable precision at 95 percent confidence given the assumed $380 per-invoice standard deviation.

Selecting the Sample

Sample size planning determines how many items to select; a separate dialog determines which specific items those are. This dialog, labeled Sampling, has no button in the sidebar; it is reached, like Cross-Tabulation and Financial Ratios, by right-clicking a column header and choosing Statistics → Sampling…, or from the top menu bar's own Statistics menu. It offers four selection methods. Random sampling draws a specified number of rows from the population with each row equally likely to be selected, using a specified seed value where the selection needs to be reproducible for a second reviewer. Systematic sampling selects every kth row starting from a random point within the first interval, which is faster to explain and defend than pure random selection but assumes the population carries no periodic pattern that could coincide with the chosen interval. Stratified sampling divides the population into groups based on a chosen column, such as region or transaction type, and draws a number of items from each group in proportion to that group's share of the population, which ensures that smaller but potentially important segments of the population are not overlooked entirely by chance the way pure random sampling might overlook them. The fourth method, monetary unit sampling, is the technique most specific to a financial statement audit and is given a full walkthrough below.

Monetary Unit Sampling

Rather than treating each row as an equally likely sampling unit, monetary unit sampling, also known as dollar-unit or probability-proportional-to-size sampling, treats each individual dollar of value as the sampling unit, so that a row recording one hundred thousand dollars is one hundred times more likely to be selected than a row recording one thousand dollars. CaatMaster implements this by cumulating the absolute value of the chosen value column across the population, dividing the total by the desired sample size to establish a fixed sampling interval, choosing a single random starting point within that interval, and then selecting every row whose position in the cumulative total falls at the starting point or at each subsequent multiple of the interval beyond it.

Because larger items are proportionally more likely to be selected under this method, any item whose own value exceeds the sampling interval is guaranteed selection, sometimes at more than one selection point if the item is large enough to span several intervals; CaatMaster records how many selection points each selected item absorbed, so a reviewer can see directly which items were examined because of their sheer size rather than through the ordinary probability mechanism. This property, guaranteed coverage of the largest items in a population, is precisely why monetary unit sampling has been the dominant substantive sampling method in financial statement auditing for decades: an error concealed in a small transaction can only ever cause a

small misstatement, while an error concealed in a very large transaction is exactly the kind of item this method ensures gets examined.

1. Right-click any column header in the AP Transactions dataset and choose `Statistics → Sampling...`.
2. Under Sampling Method, select `Monetary Unit Sample (MUS/PPS)`.
3. Under Sample Size, select `Fixed number of rows` and enter `8`.
4. Set Value column (dollar amount) to `amount`, and Random Seed to `7` so the selection can be reproduced by a second reviewer.
5. Click `Generate Sample`.

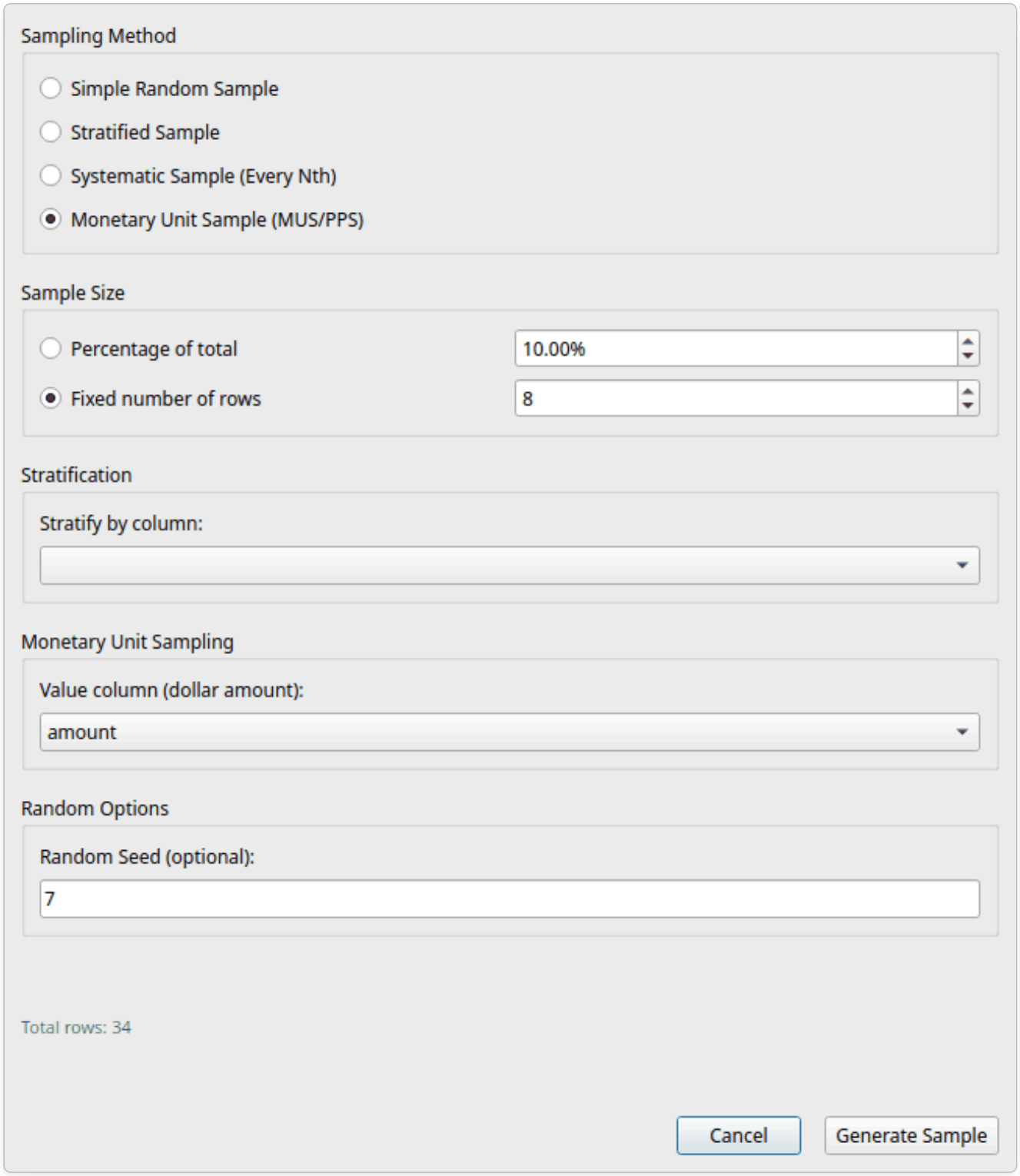


Figure 11.1 — Monetary Unit Sampling configured against ap_transactions.csv, targeting eight selection points.

Invoice	Vendor	Amount	Selection Points Absorbed
20002	Cascade IT Solutions	1,290.00	1

Invoice	Vendor	Amount	Selection Points Absorbed
20008	Anderson Paper Co	842.50	1
20014	GreenLeaf Catering	2,500.00	1
20022	Evergreen Facilities	3,210.15	1
20027	Cascade IT Solutions	48,750.00	3
20030	Fenwick Legal Group	3,320.00	1

Figure 11.2 — The result of the eight-point selection above. Six physical invoices cover the eight selection points, because the 48,750.00 server rack cooling fan invoice, the same outlier identified in Chapter 10, is large enough on its own to absorb three points. It would have been selected under this method regardless of the random seed chosen, which is exactly the guarantee monetary unit sampling is designed to provide.

Evaluating a Sample's Results

Once a sample has been drawn and each selected item has been examined against its supporting evidence, producing both the originally recorded, or book, value and the verified, or audited, value for each item, the Evaluate Sample Results tab of the same Classical Variables Sampling dialog used for size planning projects those results back onto the full population. Three estimation methods are available. Mean-per-unit estimation projects the sample's average audited value across the entire population and is the simplest to explain, though the least precise of the three where book values are already known. Difference estimation projects the sample's average difference between audited and book value across the population and adds that projected difference to the population's known book total, which produces a considerably more precise estimate whenever errors tend to be roughly similar in dollar size regardless of an item's own value. Ratio estimation instead projects the sample's ratio of audited value to book value onto the population's book total, and is the more precise choice whenever errors tend to scale with the size of the item rather than remaining constant in dollar terms.

A SCENARIO WORTH RECOGNIZING

Suppose an accounts receivable confirmation exercise draws a monetary unit sample of sixty invoices from a receivable population with a book total of eight million four hundred thousand dollars, out of two thousand one hundred outstanding invoices. Of the sixty invoices selected, fifty-six confirm exactly with the customer, and four confirm at a lower amount than recorded, reflecting short payments not yet applied in the ledger, for shortfalls of $150.00, $220.00, $95.00, and $310.00.

1. With the sixty sampled items entered as a dataset carrying a Book Value Column and an Audited Value Column, open `Variables Sampling` from the sidebar and switch to the `Evaluate Sample Results` tab.
2. Set Book Value Column and Audited Value Column to the two columns holding the recorded and confirmed amounts.
3. Set Population Size to `2100` and Population Book Total to `8400000`, and leave Confidence Level at `95`.
4. Under Estimation Method, select `Difference`, since the four shortfalls found are closer to a constant dollar amount than a constant proportion of invoice size.
5. Click `Evaluate Sample`.

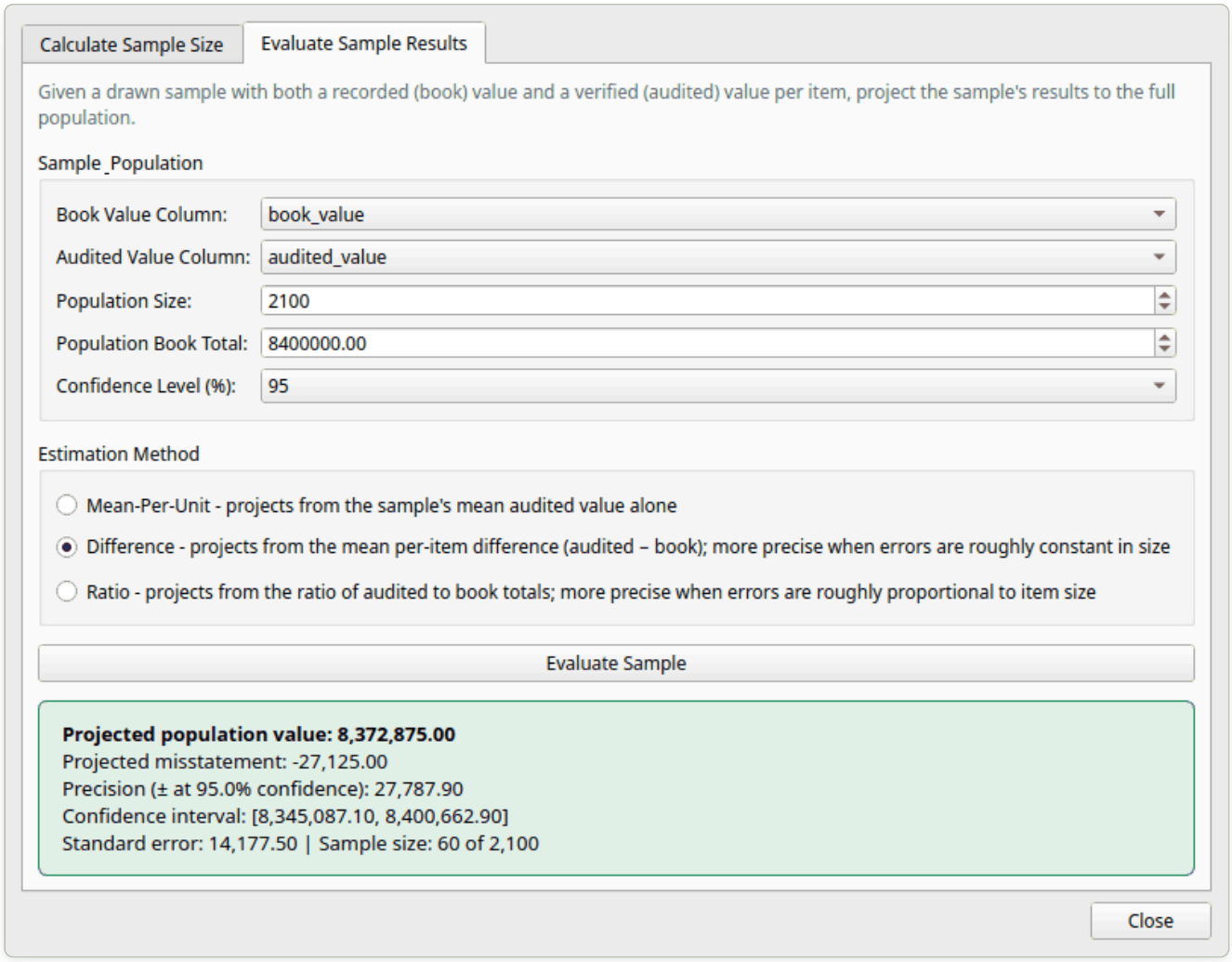


Figure 11.3 — Difference estimation applied to the accounts receivable confirmation scenario above. The projected population value of $8,372,875.00 sits close enough to the recorded book total of $8,400,000.00 that the entire 95 percent confidence interval, $8,345,087.10 to $8,400,662.90, would fall comfortably inside almost any reasonable materiality threshold for a population this size.

Whichever method is chosen, the evaluation returns a point estimate of the population’s true value, the implied projected misstatement, and a precision figure, computed at the chosen confidence level and adjusted by the same finite population correction used in sample size planning, that expresses the range within which the population’s true value is expected to fall. In the run above, that precision is $27,787.90 on a projected misstatement of negative $27,125.00, giving a standard error of $14,177.50 from a sample of sixty items drawn against a population of two thousand one hundred. If the resulting upper bound of likely misstatement remains comfortably below the engagement’s materiality threshold, as it does here, the population can be accepted as fairly stated; if it does not, the sample has done exactly what it was designed to do, which is to indicate that further work on the population is required before an opinion can be supported.

Interpreting the Result

WHEN A SAMPLE EVALUATION PROJECTS A MISSTATEMENT

1. **Benign explanation.** A small number of timing differences, such as a payment made shortly before period end that had not yet been recorded when the population was extracted, can account for a modest projected misstatement without indicating any underlying error in the recording process itself.
2. **Error or judgment lapse.** A consistent, one-directional difference across most or all of the exceptions in a sample, rather than differences scattered in both directions, suggests a systematic cause, such as a miscalculated rate applied uniformly, rather than isolated mistakes.
3. **What would indicate something more serious.** Exceptions concentrated among items associated with a specific customer, region, or approver, combined with a pattern of responses that appear designed to avoid a direct confirmation, warrant an expanded sample and direct inquiry beyond what the statistical projection alone can resolve.

PART III

Internal Controls

Chapter 12

Segregation of Duties and Access Review

A control built entirely around a single person's honesty is not a control. Segregation of duties addresses this by ensuring that no one individual holds two or more incompatible responsibilities within the same process, such as creating a vendor record and also approving payments to that vendor, so that a single person acting alone cannot both cause a transaction and approve it. Access review addresses a related but distinct question: whether the technical permissions granted to a system continue to match the person's current employment status and role, since a termination or a transfer that is not promptly reflected in system access leaves a door open that no one is actively watching.

The Working Dataset for This Chapter

This chapter introduces a third recurring file, user_activity_log.csv, an extract of twenty-one system activity records covering eleven users of Meridian Office Supply Co.'s procurement and payment system over January 2025. Each row records one user performing one action on one date, together with that user's current employment status and, where applicable, a termination date.

User	Department	Action	Date	Status
Grace Liu	Procurement	Create Vendor	2025-01-06	Active
Grace Liu	Procurement	Create PO	2025-01-09	Active
Elena Petrova	Procurement	Create Vendor	2025-01-08	Active
Elena Petrova	Procurement	Pay Vendor	2025-01-14	Active
David Osei	Procurement	Create PO	2025-01-09	Active
David Osei	Procurement	Approve PO	2025-01-11	Active
Wendy Chu	Finance	Approve Invoice	2025-01-22	Terminated

… fourteen further records …

Figure 12.1 — A sample of user_activity_log.csv. The complete file additionally includes Marcus Webb, Priya Nair, Tom Brennan, Sarah Kim, James Okoro, Alan Brooks, and Robert Lin.

Running a Segregation of Duties Scan

Segregation of Duties Analysis is reached by right-clicking any column header and choosing Audit → SOX SoD Analysis..., or from the top menu bar's Audit → Audit Checks submenu. It opens a dedicated dialog rather than the generic parameter form most audit operations use, since defining a set of toxic combinations benefits from a purpose-built table rather than a single-row parameter list.

1. Right-click any column header in user_activity_log.csv and choose `Audit → SOX SoD Analysis...`.
2. Set User ID Column to `user_id` and Action/Role Column to `action`.
3. Under Load Template, select `Procure to Pay (P2P)`. Three toxic combinations populate automatically: Create Vendor paired with Pay Vendor, Create PO paired with Approve PO, and Receive Goods paired with Approve Invoice, each at High severity.
4. Click `Scan for Conflicts`.

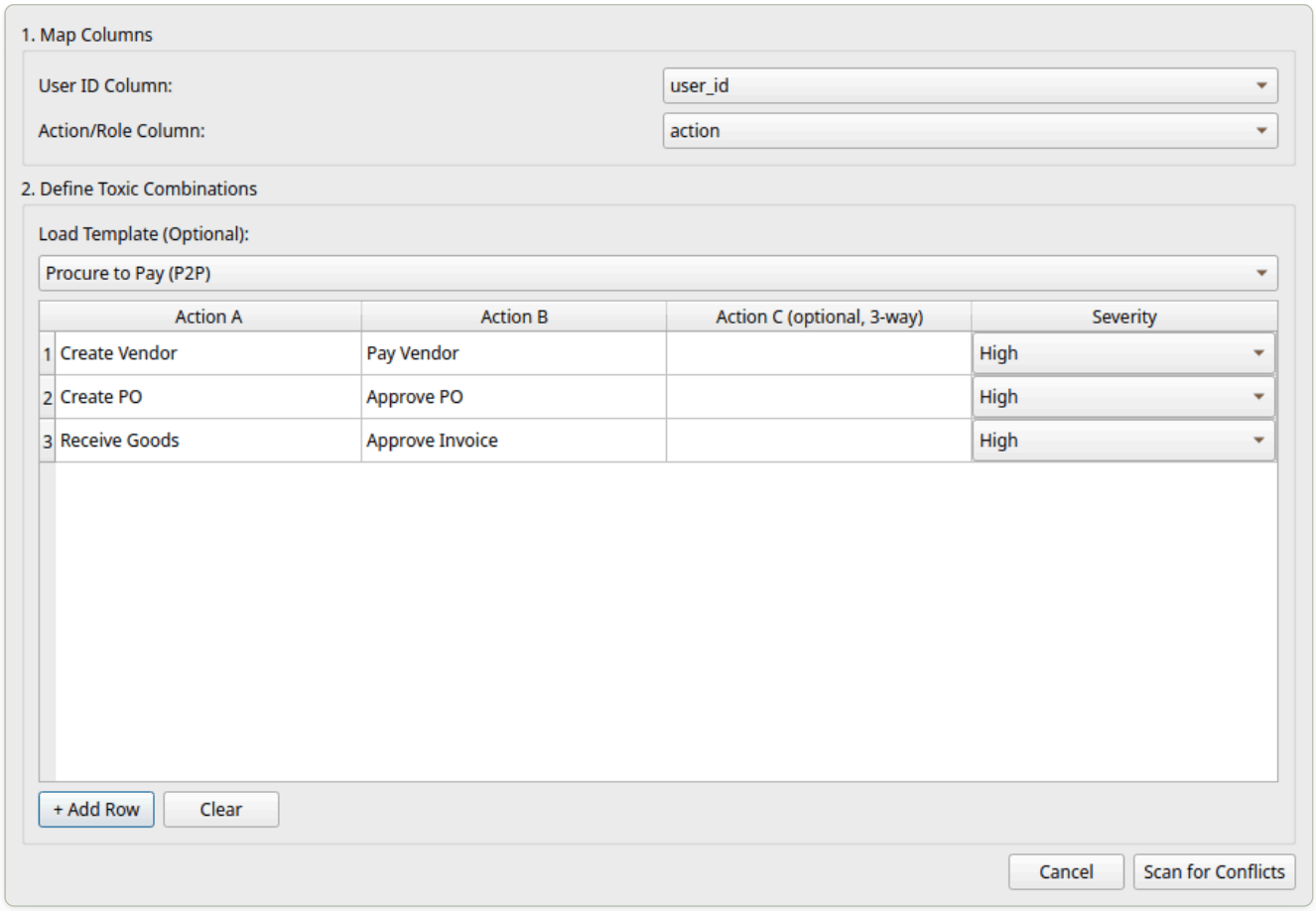


Figure 12.2 — The built-in Procure to Pay template. A third, optional column, Action C, is available for a three-way toxic combination, left blank here since none of these three rules needs it.

Two new columns, sod_conflict_found and sod_conflict_detail, are added to every row belonging to a user who holds both sides of at least one toxic combination, not merely to the specific rows where the conflicting actions themselves appear. Run against user_activity_log.csv, the scan identifies three users:

User	Conflict Detail
Elena Petrova	High: Create Vendor + Pay Vendor
David Osei	High: Create PO + Approve PO
Priya Nair	High: Receive Goods + Approve Invoice

Figure 12.3 — Every user in the working file who holds both sides of a Procure to Pay toxic combination. Grace Liu, who holds Create Vendor and Create PO, is not flagged, since that specific pairing is not one of the three rules loaded.

Toxic Combinations Beyond a Pair

The optional third column in the conflict table exists for a genuinely three-way toxic combination, where a finding depends on a user holding all three of a set of actions rather than any two. A user who can create a purchase order, approve that same purchase order, and record the goods as received controls an entire acquisition cycle alone, a materially different risk from holding any two of those three capabilities on their own. CaatMaster evaluates this the same way it evaluates a pairwise rule: a user is flagged only if every non-blank action in the row is present in that user's own action history, so a three-way rule with two of its three conditions met does not produce a partial or watered-down flag.

Running an Access Review

SOX Access Review, reached the same way as the SoD scan, by right-clicking any column header and choosing Audit → SOX Access Review..., flags every activity record belonging to a user whose status column currently reads as terminated, inactive, disabled, or suspended, a list of qualifying values that can be edited directly in the dialog.

1. Right-click any column header in user_activity_log.csv and choose `Audit → SOX Access Review...`.
2. Set User Status Column to `status`.
3. Set Activity Date Column to `action_date` and Termination Date Column to `termination_date`. Both are optional; supplying them extends the check to also flag activity dated on or after a recorded termination date, regardless of what the status column currently shows.
4. Click `Run Access Review`.

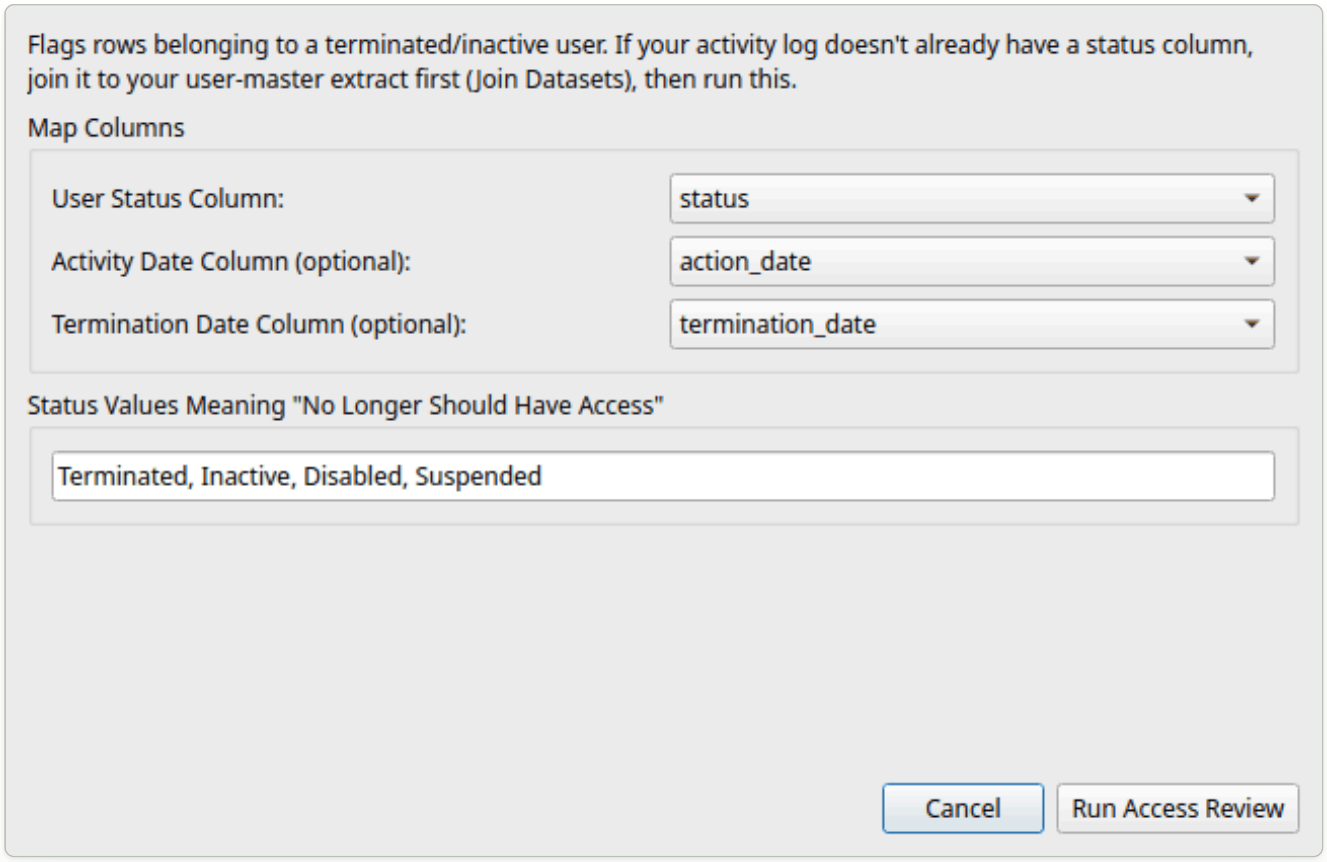


Figure 12.4 — SOX Access Review configured against user_activity_log.csv.

Run against the working file, this flags all four activity records belonging to the two users whose status column currently reads Terminated: three records for Wendy Chu and one for Robert Lin. This result rewards a closer look rather than a single glance. Every one of a currently terminated user's logged actions is flagged, not only the ones recorded after that user's termination date, because the fact that a user is now terminated is itself a reason to confirm that everything they did while employed was legitimate. The termination date supplied alongside the status becomes useful precisely for telling these flagged records apart from one another: Robert Lin's single flagged record is dated December 10, 2024, ten days before his December 20, 2024 termination date, while one of Wendy Chu's three flagged records is dated January 22, 2025, seven days after her January 15, 2025 termination date. The flag column alone cannot make this distinction; the reviewer has to look at the dates.

A SCENARIO WORTH RECOGNIZING

Suppose an internal audit team runs an access review against a full year of ERP activity for a company with four hundred active users and reviews every flagged record by status alone, without cross-referencing the termination date column also present in the extract. The review concludes that eighteen terminated employees generated activity while marked terminated in the system, and reports this as eighteen access control failures of roughly equal severity.

A closer reading of the termination dates shows that fifteen of the eighteen are timing artifacts: the status field was updated on the employee's last physical day in the office, while their final approved expense reports and timesheet submissions, entirely legitimate, posted to the system over the following one to two weeks through a normal payroll cutoff process. The remaining three, however, show system activity dated three to six weeks after both the status change and the termination date, well past any plausible administrative lag, and involve one user who continued approving purchase orders a full month after departure. Treating all eighteen as equally severe would have buried the three findings that actually matter inside fifteen that do not.

Interpreting the Result

WHEN A USER IS FLAGGED FOR A TOXIC COMBINATION OR POST-TERMINATION ACTIVITY

1. **Benign explanation.** A small company or a backup-coverage arrangement during a colleague's absence can require one person to temporarily hold two normally separated responsibilities, with a compensating control, such as a supervisor's independent review, standing in for the segregation that role design alone does not provide.
2. **Error or judgment lapse.** A termination processed in the human resources system without a corresponding, timely removal of system access is usually an administrative gap between two systems that do not automatically synchronize, rather than a deliberate decision to leave access open.
3. **What would indicate something more serious.** Continued, substantive activity, such as approving payments rather than a single delayed administrative transaction, well beyond any plausible processing lag after a termination date, combined with no compensating review of that activity by anyone else, warrants immediate escalation rather than inclusion in a routine finding.

Chapter 13

Mapping Findings to COSO

A list of findings, however well supported each one is individually, does not by itself tell an audit committee where an organization's control environment is weakest. The COSO Internal Control Framework, published by the Committee of Sponsoring Organizations of the Treadway Commission and the most widely referenced control framework in financial statement auditing, organizes internal control into five components: Control Environment, the tone and structure an organization sets for control consciousness; Risk Assessment, the process of identifying and analyzing risks to objectives; Control Activities, the specific policies and procedures that address those risks; Information and Communication, the systems that ensure relevant information reaches the people who need it; and Monitoring Activities, the ongoing and separate evaluations that confirm the other four components continue to function. Mapping a set of findings onto these five components turns a flat list into a picture of where control weakness actually concentrates.

Two Ways CaatMaster Builds a Heatmap

CaatMaster offers two paths to a COSO heatmap, suited to two different situations. Where individual findings have already been produced elsewhere in the application, each carrying its own COSO component and severity, as every rule in the deterministic rule libraries covered in the next chapter already does, the heatmap can be built directly from that list of findings, with no further mapping step required. Where a reviewer instead starts from a general risk register or a list of process areas that have not yet been tagged with a COSO component, the COSO Risk Heatmap operation described below provides the mapping step itself, as part of running the operation.

Running a COSO Risk Heatmap

COSO Risk Heatmap, reached by right-clicking any column header and choosing Audit → COSO Risk Heatmap..., takes a category column, such as a process area or business unit, and a numeric risk score column, and produces a summary table with one row per COSO component, aggregating the average and maximum risk score and the number of underlying issues mapped to each. This chapter's working example uses a fourth file, process_risk_register.csv, an eight-

row risk register listing distinct process areas across Meridian Office Supply Co. together with a risk score from zero to ten previously assigned during a risk assessment workshop.

1. With process_risk_register.csv open, right-click any column header and choose `Audit → COSO Risk Heatmap...`.
2. Set Category/Process Column to `process_area` and Risk Score Column to `risk_score`.
3. Click `Load Unique Categories`. One row appears per distinct process area, each with its own COSO Component selector, defaulted to Control Environment.
4. Set each row's COSO Component: IT User Access Provisioning and Accounts Payable Authorization to Control Activities; Vendor Master Maintenance to Control Environment; Sales Order Credit Checks and Board Risk Committee Reporting to Risk Assessment; Journal Entry Review and Physical Inventory Counts to Monitoring; Whistleblower Hotline Monitoring to Info & Comm.
5. Click `Generate Heatmap Data`.

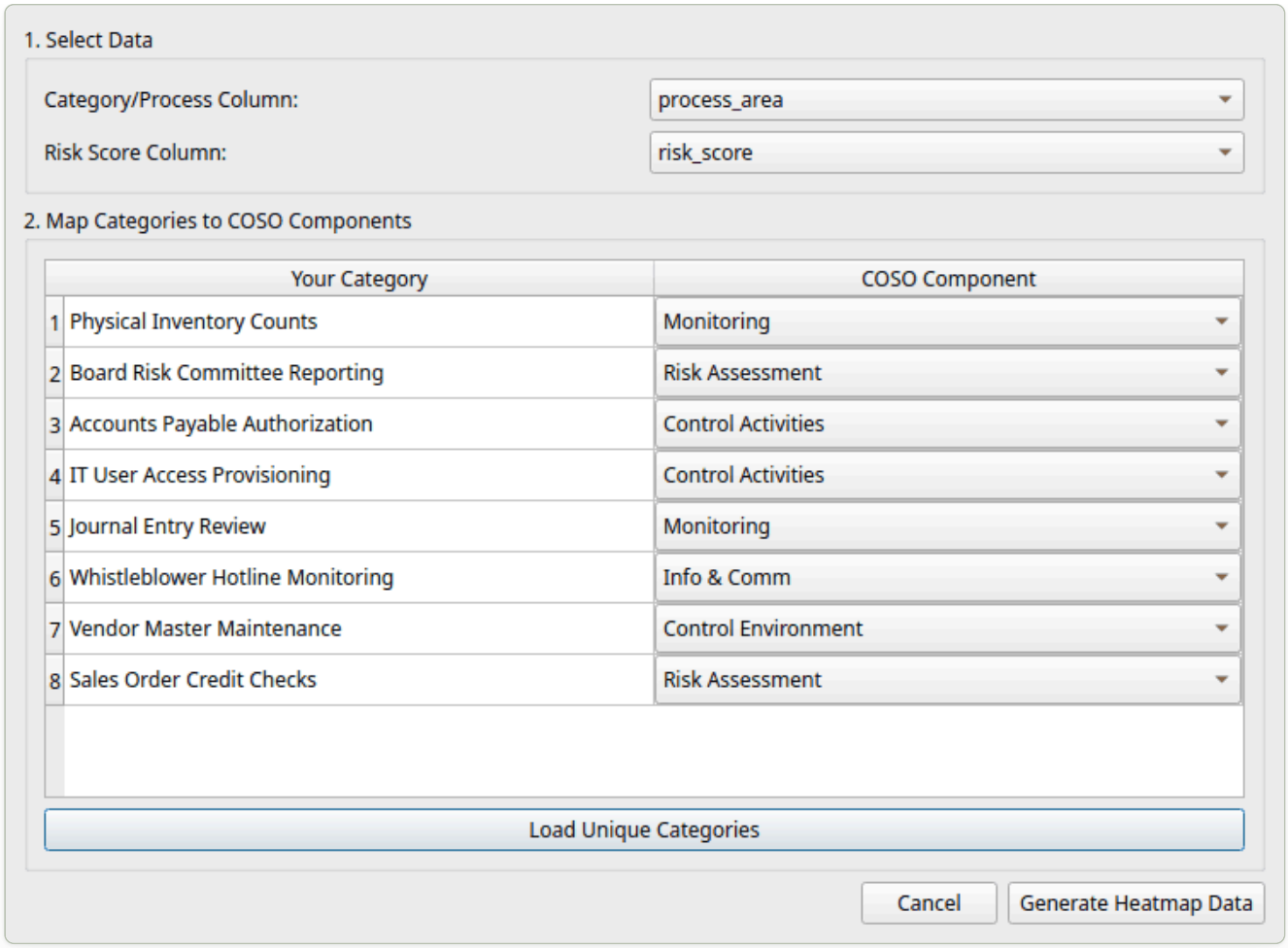


Figure 13.1 — Mapping eight process areas from process_risk_register.csv to their COSO components before generating the heatmap.

COSO Component	Avg Risk Score	Max Risk Score	Issue Count
Control Activities	7.75	8.0	2
Control Environment	7.0	7.0	1
Monitoring Activities	5.75	6.0	2
Risk Assessment	4.0	4.5	2
Information & Communication	3.0	3.0	1

Figure 13.2 — The resulting heatmap, sorted by average risk score. Control Activities carries both the highest average and the highest single risk score in this register, driven by the Accounts Payable Authorization and IT User Access Provisioning process areas.

Every figure in this table traces back to a risk score a reviewer assigned directly to a named process area during the underlying risk assessment; CaatMaster contributes the aggregation and

the mapping, not the underlying judgment about how risky any one process area actually is. A component score is only as reliable as the risk scores and the category-to-component mapping supplied to produce it, and both belong in the workpaper alongside the resulting heatmap.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a controls self-assessment across a larger organization's forty process areas produces a heatmap showing Control Activities and Monitoring Activities both scoring above seven on a ten-point scale, while Control Environment scores below three. Read in isolation, the low Control Environment score might appear to be the strongest result on the page. Read against COSO's own framework, in which Control Environment is described as the foundation the other four components depend on, a low score there deserves more attention, not less, since a weak tone at the top can eventually undermine control activities and monitoring that currently test as strong, even though today's heatmap does not yet show any resulting deterioration in those components.

Interpreting the Result

WHEN ONE COSO COMPONENT SCORES MARKEDLY HIGHER THAN THE OTHERS

1. **Benign explanation.** A single component can score high simply because more findings have been actively looked for and documented there, such as a recently completed IT general controls review, rather than because that component is genuinely weaker than areas that have not yet been reviewed as closely.
2. **Error or judgment lapse.** A category-to-component mapping applied inconsistently, such as one reviewer routing access-related findings to Control Activities and another routing similar findings to Information and Communication, can distort a heatmap in a way that has nothing to do with the underlying risk.
3. **What would indicate something more serious.** A Control Environment score that is low not because it has been under-reviewed but because specific, documented findings, such as an unresolved whistleblower complaint or a compromised reporting line, support it, warrants escalation to the audit committee regardless of how the other four components currently score.

Chapter 14

Deterministic Rule Libraries

Chapter 4 described a governing rule for CaatMaster's AI-assisted features: a language model may be asked to translate a policy written in prose into a structured, checkable condition, but the condition itself is always evaluated against the complete dataset using ordinary, deterministic code. The deterministic rule libraries covered in this chapter apply the second half of that pattern without needing the first half at all. Each rule ships with its checkable condition already written by hand, against logical field names such as amount or approver, so that running it requires only mapping those logical names to the actual column names in a specific dataset, with no AI provider, no API cost, and no translation step of any kind.

The Libraries CaatMaster Ships With

Three libraries are included as of this writing, spanning eight individual rules in total. SOX — Accounts Payable & Procurement Controls contains three rules: Payment Without Approval on File, which flags any payment at or above a configurable threshold with no approver recorded; Duplicate Vendor Payment, which flags any repeated combination of vendor, amount, and reference number; and Payment Just Below Approval Threshold, which flags payments falling in the ten percent band immediately under the approval threshold, a deterministic counterpart to the threshold-avoidance pattern introduced through Benford's Law in Chapter 8. COSO — Expense & Travel Control Activities contains three parallel rules aimed at expense reimbursement: Expense Without Receipt Reference, Expense Over Policy Limit, and Duplicate Expense Claim. Period-End Journal Entry Risk contains two rules aimed at the manual journal entries examined in a typical ISA 240 fraud-risk procedure: Large Manual/Topside Journal Entry and Journal Entry Missing Description.

Every rule in every library carries its own citation, its COSO component, a default severity, and a plain-language description and recommendation, so that a finding produced by a rule library arrives already written in the vocabulary of a workpaper rather than as a bare true-or-false flag.

Running a Rule Library Check

Rule libraries are reached from the top menu bar through Reports → LLM Analysis..., despite that menu label, which names the dialog's original purpose rather than everything it now does; the Rule Detection tab of the dialog this opens runs entirely without AI. This chapter revisits the missing-approver finding from Chapter 6 and the near-threshold outlier from Chapter 8 and Chapter 10, and shows both are also reachable as a single, named, citable rule.

1. From the top menu bar, choose `Reports → LLM Analysis...` against the AP Transactions dataset, and open the `Rule Detection` tab.
2. Select the `SOX — Accounts Payable & Procurement Controls` library, then choose the `Payment Without Approval on File` rule. CaatMaster proposes a mapping from the rule's logical fields to matching columns in the open dataset.
3. Confirm amount maps to `amount` and approver maps to `approver`, and set reference to `invoice_id`.
4. Lower the Approval threshold ($) parameter from its default of 10000 to `300`, since this working file's amounts are far smaller than a typical company-wide approval limit.
5. Click `Run Rules`.

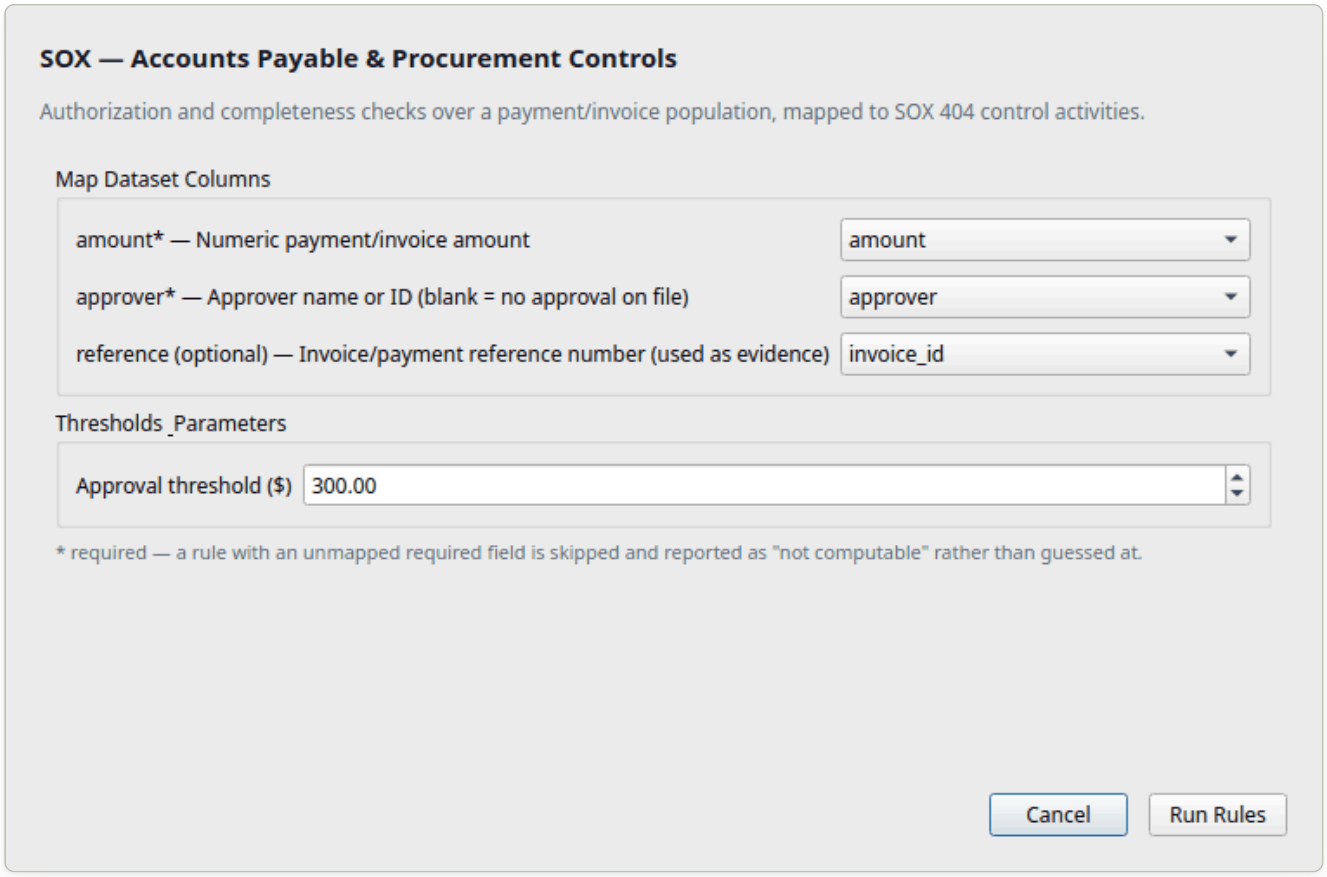


Figure 14.1 — Mapping the Payment Without Approval on File rule's two required fields and one optional field to ap_transactions.csv, with the threshold lowered to suit this working file's scale. The note below the mapping states the rule's own discipline plainly: a required field left unmapped causes the rule to report itself as not computable, rather than a guess standing in for a real answer.

The rule flags the same five records Chapter 6's no-nulls check on the approver column found, for a combined exposed amount of $4,248.40, because both tests are, underneath their different names, asking the same question of the same data. Where the rule library adds real value over the completeness check alone is in what it hands back: not just a fail count, but a named finding, SOX 404 — Control Activities (Authorization), a COSO component, Control Activities, a severity, high, and a ready-made recommendation to obtain evidence of approval or correct the workflow, none of which a generic completeness check produces on its own.

Running the same library's Payment Just Below Approval Threshold rule at its default $10,000 threshold against the same working file flags exactly one record, invoice 20010, Cascade IT Solutions, $9,900.00, the same invoice already identified as a statistical outlier in Chapter 10. A single population examined through three different techniques in this book, an outlier test, a Benford digit test, and now a named compliance rule, converges on the same transaction, which is a considerably stronger basis for inquiry than any one of the three findings would be alone.

A SCENARIO WORTH RECOGNIZING

Suppose a compliance team maintains its authorization policy as a two-page written document, reviewed and re-approved annually by internal audit, stating that payments of ten thousand dollars or more require a named approver on file. Rather than re-reading that document each quarter and manually querying the payment ledger, the team maps the Payment Without Approval on File rule once, at the start of the year, to the general ledger extract's actual column names, and re-runs it against each new quarterly extract without any further translation step. When the policy is revised mid-year to lower the threshold to seven thousand five hundred dollars, updating the check requires changing one number in the rule's threshold parameter, not rewriting a query or re-explaining the policy to anyone.

Interpreting the Result

WHEN A RULE LIBRARY FLAGS A POPULATION OF TRANSACTIONS

1. **Benign explanation.** A missing approver field can reflect a control that is genuinely performed outside the system under test, such as a paper sign-off retained in a physical file, rather than a control that was skipped entirely.
2. **Error or judgment lapse.** A threshold rule calibrated to a company-wide policy, applied without adjustment to a single small department or a small working file, as this chapter's own example demonstrates, can flag a rate of exceptions that looks alarming purely because the threshold does not match the scale of the population being tested.
3. **What would indicate something more serious.** The same transaction surfacing under more than one independent rule or technique, as invoice 20010 does across this book's outlier, Benford, and rule-library chapters, is a materially stronger signal than any single flag on its own and warrants priority follow-up.

PART IV

IFRS Assertions

Chapter 15

Revenue Recognition (IFRS 15)

Revenue recognition is judgment-heavy in a way that few other line items are. IFRS 15 requires revenue to be recognized as an entity satisfies a performance obligation, not on any single, universally observable event, which leaves genuine room for a contract's margin or its recognition timing to be managed toward a target rather than derived purely from the underlying economics. This chapter covers two deterministic techniques CaatMaster applies to a population of contracts: a control-chart test for a sustained shift in a contract's margin over time, and a straightforward comparison between the date revenue was recognized and the date the underlying performance obligation was actually satisfied.

The working file for this chapter, contract_revenue_schedule.csv, holds fourteen rows: three contracts, each reporting revenue and cost across four to six monthly periods, together with a recognition date and a delivery date for every period.

Every IFRS and IAS operation in this part of the book, including the two covered in this chapter, is reachable two ways: by right-clicking any column header and choosing Financial, the path every walkthrough below uses, or from the top menu bar's Financial menu, added once early beta feedback flagged that a reviewer had no way to discover these operations without first knowing to right-click a column header. Both paths open the identical dialog and compute identically; neither is more complete than the other.

Detecting a Margin Shift with CUSUM

A contract whose margin holds steady for several periods and then moves sharply is a pattern worth understanding, since a sudden shift is more consistent with a change in how costs are being allocated between periods than with a genuine change in the underlying economics of the work. CaatMaster tests for this using a CUSUM, or cumulative sum, control chart, a technique long used in manufacturing quality control to detect a change in a process level. The test accumulates each period's deviation from the contract's own baseline margin, allowing a small amount of normal noise, the slack parameter k, before that deviation starts counting, and signals a shift once the accumulated deviation crosses a threshold, h, expressed as a multiple of the margin's own scale. Both k and h scale automatically with the contract's own margin

volatility, so the same default multipliers apply sensibly whether a contract's margin normally moves by half a percentage point or by ten.

The baseline itself is worth a direct word, because an earlier version of this check used the plain mean and standard deviation of the whole margin history to set that baseline, and that choice had a real, verified weakness: this is a retrospective scan of a complete history, not a monitoring chart with a separately established, known-good reference period, so the shift being searched for is necessarily included in the very data used to decide what normal looks like. A single sharp collapse can pull the mean toward it and inflate the standard deviation enough that the collapse no longer clears a multiple of its own inflated denominator, so a genuine shift could go undetected precisely because it was large enough to distort the baseline meant to catch it. CaatMaster now estimates that baseline using the median and the interquartile range instead, both of which stay put unless a shift affects a genuinely large share of the population, which resolved the failure directly: a real six-period contract history, five months of stable margin followed by one sharp collapse, went from undetected at the default threshold to correctly flagged, with no false positive introduced against a longer, evenly-split synthetic history used elsewhere to test this same function.

1. With contract_revenue_schedule.csv open, right-click any column header and choose `Financial → IFRS 15 Margin Shift Detection (CUSUM)...`.
2. Set Revenue Column to `revenue`, Cost Column to `cost`, Period Date Column to `recognition_date`, and Contract/Group Column to `contract_id`, so the CUSUM baseline is computed independently for each contract rather than across the whole population.
3. Leave Slack (k) and Threshold (h) at their defaults of 0.50 and 4.00.
4. Click `Detect Margin Shifts`.

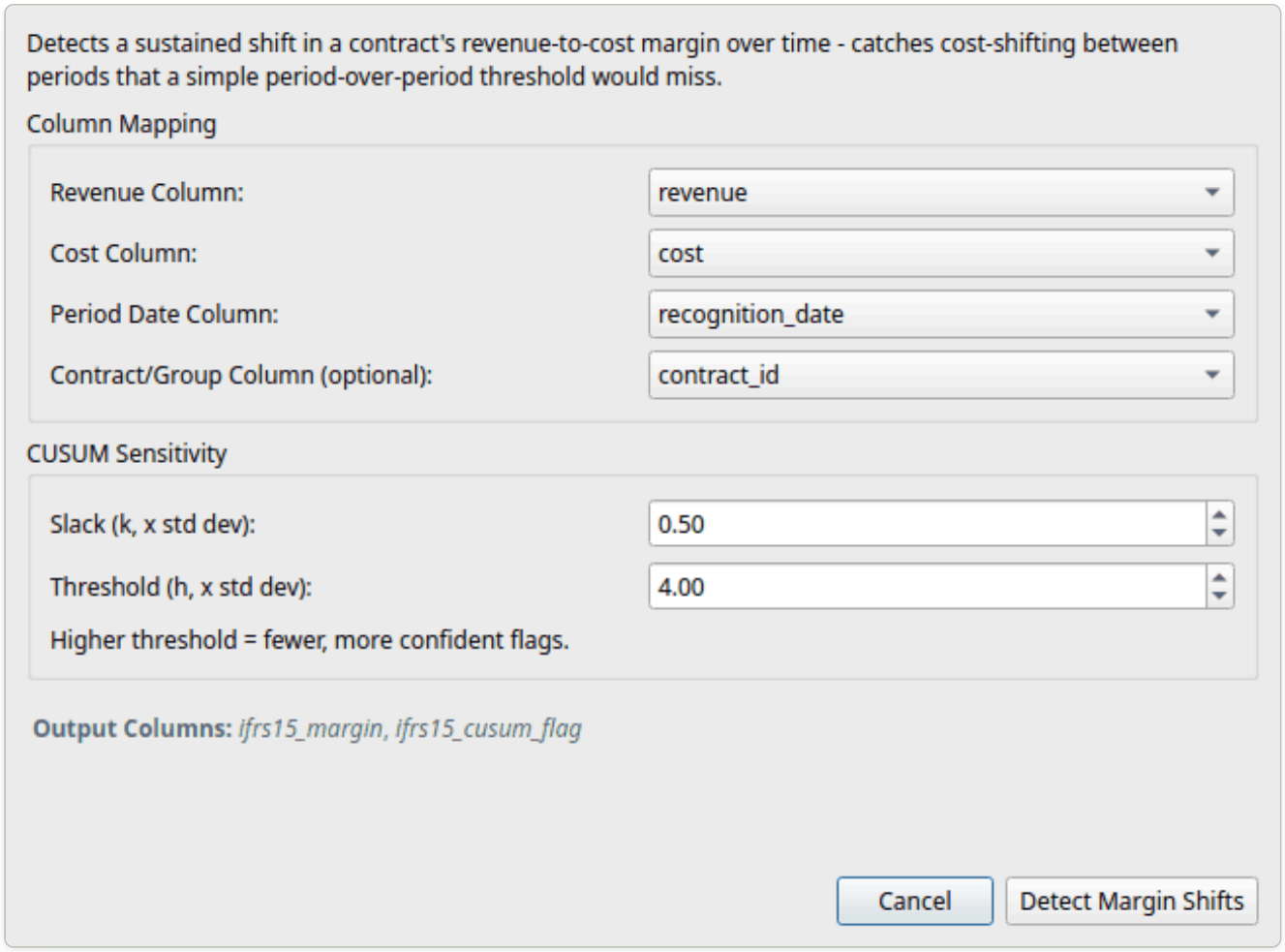


Figure 15.1 — CUSUM margin-shift detection configured against contract_revenue_schedule.csv, left at its default sensitivity.

Contract CT-1001 runs at a stable margin near forty percent for five months, then drops to fifteen percent in the sixth. At the default threshold, this is now flagged correctly, and neither of the other two contracts, whose margins hold genuinely steady throughout, produces a false positive.

Checking Revenue Timing Against Delivery

Revenue Timing Gap is reached the same way, through Financial → IFRS 15 Revenue Timing Gap…, and answers a narrower, more mechanical question: for each row, how many days separate the date revenue was recognized from the date the performance obligation was satisfied. A negative gap means revenue was booked before delivery, a bill-and-hold-style pattern that IFRS 15's transfer-of-control principle treats as premature recognition unless the specific conditions for recognizing revenue ahead of physical delivery are separately met and

documented. Run against this chapter's working file with Revenue Recognition Date Column set to recognition_date, Delivery/Shipment Date Column set to delivery_date, and the allowed buffer left at its default of zero days, the check flags exactly one row: CT-1003's March period, recognized twelve days before the goods were actually delivered.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a technology reseller runs both checks together across several hundred active contracts at quarter end. The CUSUM test flags four contracts with a sustained margin shift; the timing-gap test flags eleven contracts with revenue recognized ahead of delivery. Two contracts appear on both lists. For those two, the margin shift traces directly to revenue being pulled forward into the current quarter from a shipment that had not yet gone out, inflating the current period's recognized margin relative to a true, delivery-matched calculation. The other contracts on each list turn out to have independent, unrelated explanations, an unusually large parts cost on one CUSUM-flagged contract and a routine same-week shipping delay on several timing-gap-flagged ones, but the two contracts flagged by both tests together receive priority follow-up precisely because two independent techniques converged on the same population.

Interpreting the Result

WHEN A CONTRACT IS FLAGGED BY EITHER TEST

1. **Benign explanation.** A genuine change in a contract's scope, such as a renegotiated rate for a new phase of work, can produce a real, legitimate step change in margin that a control chart, which has no knowledge of contract terms, cannot distinguish from cost-shifting on its own.

2. **Error or judgment lapse.** A shipping or paperwork delay that causes revenue to be booked slightly ahead of a delivery confirmation being logged in the system is a timing and process issue, not typically an attempt to misstate results, particularly where the gap is small and consistent across many unrelated contracts.

3. **What would indicate something more serious.** A margin shift concentrated in the final period of a reporting quarter, affecting several contracts under the same account manager, combined with a timing gap that grows specifically at quarter end rather than remaining constant throughout the year, is a pattern consistent with deliberate period-end management rather than routine operational noise.

Chapter 16

Expected Credit Losses (IFRS 9)

IFRS 9 replaced the older incurred-loss model for financial asset impairment with an expected credit loss model, requiring an entity to recognize a loss allowance based on forward-looking expectations rather than waiting for objective evidence that a loss has already happened. This forward-looking requirement is implemented through a three-stage classification, Stage 1 for performing assets, Stage 2 for assets that have experienced a significant increase in credit risk since origination, and Stage 3 for assets that are credit-impaired, with the expected credit loss calculation growing more conservative at each stage. This chapter covers CaatMaster's two IFRS 9 techniques: staging a loan population and calculating its expected credit loss, and recalculating that loss as a probability-weighted average across macroeconomic scenarios rather than a single point estimate.

The working file, loan_portfolio.csv, holds ten loans, each carrying an exposure balance, a days-past-due figure, a current and an at-origination probability of default, a loss given default assumption, probability of default estimates under base, upside, and downside macroeconomic scenarios, and management's own booked expected credit loss for comparison.

Staging a Portfolio and Calculating ECL

IFRS 9 Staging, reached by right-clicking any column header and choosing Financial → IFRS 9 Staging…, classifies each loan by days past due, using the standard convention of thirty days as the boundary into Stage 2 and ninety days as the boundary into Stage 3, both configurable on the dialog's second tab. Where exposure, probability of default, and loss given default are all mapped, it additionally calculates an expected credit loss for every row as the direct product of the three, the standard EAD times PD times LGD formula.

1. Right-click any column header in loan_portfolio.csv and choose `Financial → IFRS 9 Staging...`.

2. Confirm Days Past Due (DPD) maps to `dpd`, Exposure (EAD) to `exposure`, and Loss Given Default (LGD) to `lgd`, all detected automatically from the column names.

3. Set Prob. Default (PD) to `pd_current`.

4. Under SICR Trigger by PD Deterioration, set Current PD to `pd_current` and PD at Origination to `pd_origination`. On the Thresholds tab, set the SICR PD Multiplier to `3.00x`: a loan whose current probability of default has risen to three times its probability at origination is escalated to Stage 2 even if it remains fully current on its payments.

5. Click `Run Staging Analysis`.

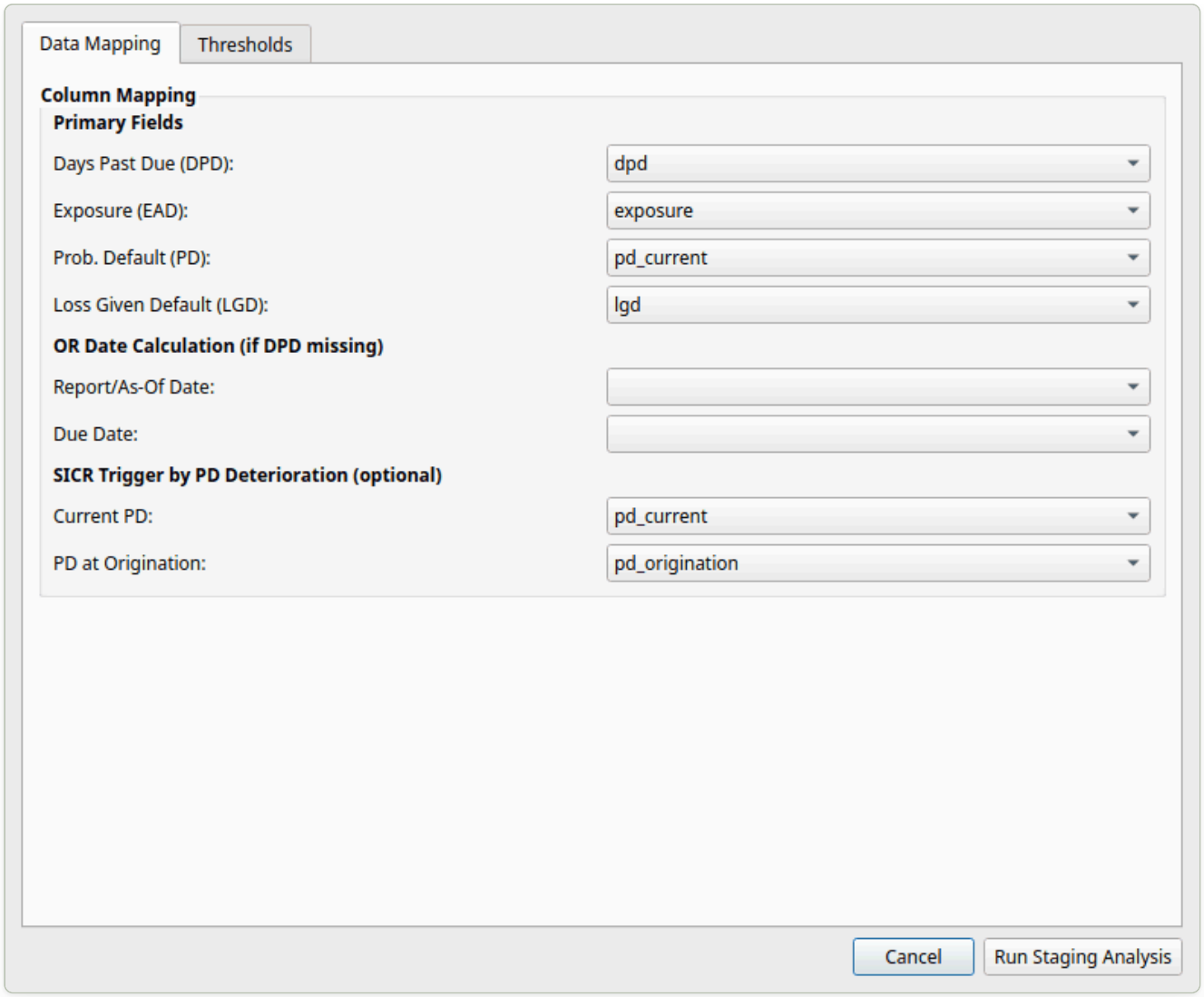


Figure 16.1 — IFRS 9 Staging configured against loan_portfolio.csv, including the optional SICR trigger by PD deterioration.

Loan	DPD	Stage	Status	ECL
L006	0	2	Underperforming	2,025.00
L002	45	2	Underperforming	8,000.00
L010	35	2	Underperforming	14,630.00
L004	100	3	Non-Performing	23,100.00
L008	95	3	Non-Performing	12,000.00

Figure 16.2 — Every loan staged above Stage 1. L006 is the case the SICR trigger exists for: zero days past due, current on every payment, but escalated to Stage 2 because its probability of default has risen to five times what it was at origination.

Recalculating ECL Across Scenarios

IFRS 9 Scenario-Weighted ECL, reached through Financial → IFRS 9 Scenario-Weighted ECL..., recalculates expected credit loss as a probability-weighted average across base, upside, and downside macroeconomic scenarios, rather than management's single point-estimate probability of default, and, where management's own booked figure is mapped, flags any loan where the two differ by more than a chosen threshold. Run against loan_portfolio.csv with the default scenario weights of fifty percent base, twenty-five percent upside, and twenty-five percent downside, and a twenty percent variance threshold, one loan is flagged: L008, whose recalculated scenario-weighted expected credit loss of $12,000.00 exceeds management's booked $7,000.00 by 41.67 percent, a full-population, deterministic recalculation, not a simulation with a different random result on every run.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a bank's allowance committee sets scenario weights of forty percent base, twenty percent upside, and forty percent downside ahead of an anticipated economic slowdown, a heavier downside weighting than the fifty-twenty-five-twenty-five split used in the walkthrough above. Recalculating the same loan population under the heavier downside weighting raises the scenario-weighted expected credit loss on every loan with a meaningfully worse downside probability of default than its base case, and a loan that passed the twenty percent variance check under the original weights can fail it under the revised ones. Neither weighting is more correct in the abstract; the choice belongs to the allowance committee's own judgment about the economic outlook, and CaatMaster's role is to make that judgment's consequences visible across the whole portfolio at once, rather than to make the judgment itself.

Interpreting the Result

WHEN A LOAN IS ESCALATED TO STAGE 2 OR 3, OR ITS ECL VARIES MATERIALLY FROM MANAGEMENT'S FIGURE

1. **Benign explanation.** A loan escalated purely by the SICR trigger, while remaining current on every payment, may reflect a genuine, temporary deterioration in a single credit factor, such as a covenant metric, that the borrower has already begun correcting, rather than a change that ultimately proves permanent.
2. **Error or judgment lapse.** A material ECL variance can trace to a stale probability of default that was not refreshed at the reporting date, an input error rather than a deliberate understatement.
3. **What would indicate something more serious.** A pattern of management's booked ECL consistently landing below the recalculated figure across many loans, rather than scattered in both directions, together with no documented, loan-specific reason for each variance, warrants inquiry into whether the allowance process itself is biased toward understatement.

Chapter 17

Leases (IFRS 16)

IFRS 16 requires a lessee to recognize almost every lease on the balance sheet, as a lease liability equal to the present value of remaining lease payments and a corresponding right-of-use asset. The single input with the most leverage over the size of that recorded liability, and the one most exposed to management judgment, is the discount rate applied to those future payments, the lessee's incremental borrowing rate. A higher discount rate produces a lower present value and therefore a smaller recorded liability from the same stream of payments, which means an aggressively high incremental borrowing rate is a direct, quantifiable way a lease obligation can be understated without changing a single contractual payment term.

The working file, lease_schedule.csv, holds six leases, each with a periodic payment amount, the number of remaining payment periods, management's selected incremental borrowing rate, a benchmark rate for comparison, and, for two of the leases, initial direct costs and lease incentives that adjust the right-of-use asset without affecting the liability itself.

Running the Lease Liability and IBR Check

IFRS 16 Lease Liability & IBR Check, reached by right-clicking any column header and choosing Financial → IFRS 16 Lease Liability & IBR Check..., calculates each lease's liability as the present value of an ordinary annuity of its remaining payments, discounted at management's incremental borrowing rate. Where a benchmark rate is also supplied, typically a risk-free rate plus an appropriate credit spread that the reviewer determines independently, CaatMaster does not build or assume any credit-spread model itself, it recalculates the same liability using that benchmark rate and flags any lease whose management-selected rate deviates from the benchmark by more than a chosen number of basis points.

1. Right-click any column header in lease_schedule.csv and choose `Financial → IFRS 16 Lease Liability & IBR Check...`.
2. Set Periodic Payment Amount to `payment`, Remaining Number of Periods to `periods`, and Management's IBR to `management_ibr`.
3. Set Benchmark IBR to `benchmark_ibr`, Initial Direct Costs to `initial_direct_costs`, and Lease Incentives to `lease_incentives`.
4. Leave the flag threshold at its default of 50.00 bps.
5. Click `Calculate Lease Liability`.

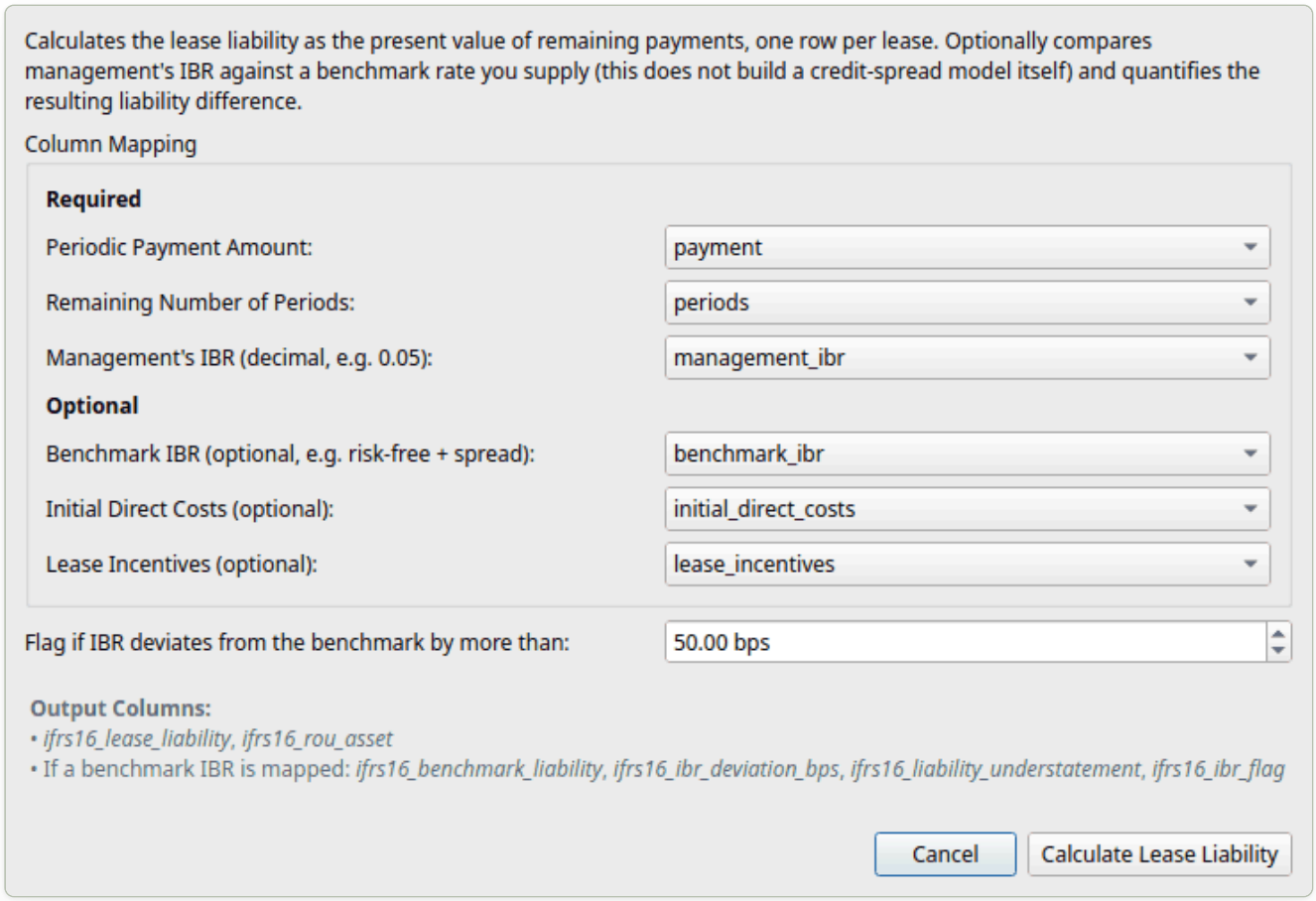


Figure 17.1 — IFRS 16 Lease Liability and IBR Check configured against lease_schedule.csv.

Lease	Mgmt IBR	Benchmark IBR	Deviation (bps)	Liability	Understatement
LS-03 Delivery Trucks	0.90%	0.35%	+55	98,026.95	10,033.57

Figure 17.2 — The single lease flagged. Management's 0.90 percent monthly rate, against a 0.35 percent benchmark, understates this lease's recorded liability by $10,033.57 relative to what the benchmark rate would have produced. The other five leases each deviate by two basis points or less and are not flagged.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a logistics company leases its delivery fleet under a series of short-term vehicle leases and applies a single, company-wide incremental borrowing rate to every lease on its books, regardless of the underlying asset's type or the lease term's length. A benchmark rate built specifically for vehicle-class assets, reflecting their shorter useful life and the fact that a vehicle lease is typically well secured by the vehicle itself, comes in meaningfully lower than the company-wide rate applied to real estate and equipment leases alike. Every vehicle lease in the portfolio is flagged, not because any single lease was deliberately mismeasured, but because a rate genuinely appropriate for a ten-year office lease was applied uniformly to three-year vehicle leases with a very different risk profile.

Interpreting the Result

WHEN A LEASE'S INCREMENTAL BORROWING RATE DEVIATES FROM THE BENCHMARK

1. **Benign explanation.** A genuinely higher incremental borrowing rate can be appropriate for an asset class or a counterparty with real, documented credit risk that the general benchmark rate does not reflect.

2. **Error or judgment lapse.** A single company-wide rate applied without adjustment for asset class or lease term, as in the scenario above, produces a mechanical rather than deliberate deviation, correctable by refining how the rate is set rather than by any individual lease.

3. **What would indicate something more serious.** A pattern of high IBRs concentrated specifically among the largest leases, or those closest to a debt covenant threshold that a smaller recorded lease liability would help preserve, warrants a closer look at how each rate was actually determined.

Chapter 18

Impairment (IAS 36)

IAS 36 requires an asset, or a cash-generating unit, the smallest identifiable group of assets that generates independent cash inflows, to be tested for impairment whenever there is an indication its carrying value may not be recoverable, and impaired down to its recoverable amount when it is not. Where recoverable amount is estimated as a value in use, a discounted cash flow model, the forecast itself is built from management's own assumptions, including a terminal growth rate applied to cash flows beyond the explicit forecast period, an assumption with no external market price to check it against directly. CaatMaster addresses this by running the discounted cash flow model in reverse: rather than taking a terminal growth assumption as an input and calculating whether the resulting valuation supports the carrying value, it takes the carrying value as fixed and solves for the exact terminal growth rate that value implies.

The working file, cgu_impairment_test.csv, holds four cash-generating units, each with a carrying value, a discount rate, and five years of explicit-period cash flow forecasts.

Running the Reverse-DCF Test

IAS 36 Implied Growth Rate (Reverse DCF), reached by right-clicking any column header and choosing Financial → IAS 36 Implied Growth Rate (Reverse DCF)…, solves for the terminal growth rate g that makes the present value of the explicit-period cash flows plus a Gordon-growth terminal value exactly equal the carrying value, using numerical root-finding rather than a closed-form formula, since the equation is not linear in g once more than one explicit cash flow period is involved. Where no valid growth rate exists in a plausible range, typically because the explicit cash flows alone already exceed the carrying value even without any terminal growth at all, CaatMaster reports a missing value rather than forcing a number, consistent with the standing rule described in Chapter 4 against ever fabricating a figure that cannot actually be computed.

1. Right-click any column header in cgu_impairment_test.csv and choose `Financial → IAS 36 Implied Growth Rate (Reverse DCF)...`.

2. Set Carrying Value Column to `carrying_value` and Discount Rate Column to `discount_rate`.

3. Under Explicit-Period Cash Flow Columns, click `+ Add Year` four times to reach five rows, and set them in order to `cf_y1` through `cf_y5`.

4. Under Growth Benchmark, select `Use a single benchmark value` and enter `0.0250`, a long-run GDP-growth-style benchmark applied to every row. Leave the flag threshold at its default of 2.00 pp.

5. Click `Solve Implied Growth Rate`.

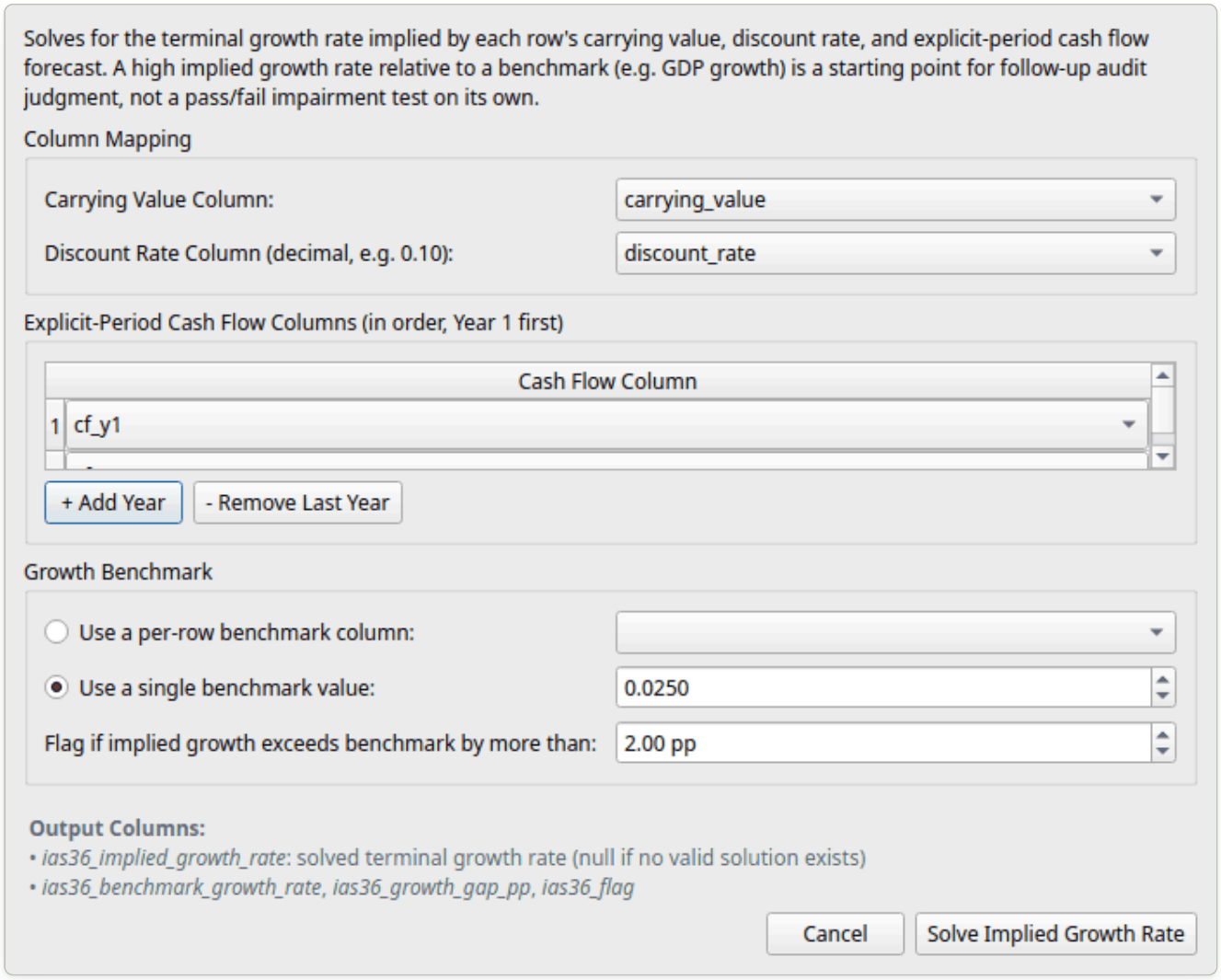


Figure 18.1 — The reverse-DCF dialog configured against cgu_impairment_test.csv. The cash flow year list is a scrollable table; five rows have been added and mapped in order even though only the first is visible at once.

CGU	Carrying Value	Implied Growth	Gap vs. 2.5% Benchmark	Flagged
CGU-Retail-North	7,800,000	4.71%	+2.21 pp	Yes
CGU-Retail-South	3,200,000	-4.70%	-7.20 pp	No
CGU-Logistics	2,000,000	-7.04%	-9.54 pp	No
CGU-Wholesale	1,500,000	-14.29%	-16.79 pp	No

Figure 18.2 — CGU-Retail-North requires a 4.71 percent perpetual growth rate to support its carrying value, 2.21 percentage points above the 2.5 percent benchmark, and is flagged. The other three CGUs imply negative growth requirements, meaning their explicit forecast cash flows alone

comfortably support the carrying value without needing any terminal growth assumption at all, let alone an aggressive one.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a retail chain's largest cash-generating unit implies a required terminal growth rate of six percent against a benchmark of two and a half percent, a three-and-a-half-point gap comfortably past the flag threshold. Management's support for the underlying carrying value cites a store-remodeling program expected to lift same-store sales meaningfully above the broader market for several years. A six percent perpetual growth rate, sustained forever rather than for the two or three years a remodel's effect might reasonably be expected to last, is a materially different claim than a temporary lift, and the reviewer's question becomes specifically whether management's five-year explicit forecast already captures the remodel's expected benefit, in which case a lower, more defensible terminal growth rate should be sufficient, or whether the remodel's benefit has been pushed into the unobservable terminal period precisely because it is easier to support a growth assumption there than a specific, checkable forecast figure.

Interpreting the Result

WHEN A CGU'S IMPLIED GROWTH RATE EXCEEDS THE BENCHMARK

1. **Benign explanation.** A CGU operating in a genuinely higher-growth market or geography than the general benchmark reflects can legitimately support an above-benchmark terminal growth rate, provided that market's own growth characteristics are documented.
2. **Error or judgment lapse.** An overly optimistic explicit-period forecast, carried through mechanically into the terminal growth solve, can produce a high implied rate that traces back to the forecast itself rather than to any terminal-period assumption being separately aggressive.
3. **What would indicate something more serious.** An implied growth rate that has increased year over year specifically to avoid recognizing an impairment that a more conservative, consistent growth assumption would trigger, without any corresponding change in the CGU's actual business prospects, warrants direct challenge of the underlying forecast.

Chapter 19

Provisions and Inventory (IAS 37, IAS 2)

This chapter covers two shorter, more mechanical tests, paired together because each addresses a balance that depends on management's own estimate of a future outcome, a provision for a probable future obligation under IAS 37 and the net realizable value of inventory under IAS 2, and each is more useful as a triage tool that narrows a large population down to a short list worth a closer look than as a standalone conclusion.

Provision Trend Breaks

A provision balance that moves sharply from one year to the next deserves a closer look regardless of the direction of the move, though a large decrease is generally the higher-risk case, since it can reflect an obligation being released without a corresponding event that actually justifies releasing it, sometimes called income smoothing through the provisions balance. IAS 37 Provision Trend Break, reached by right-clicking any column header and choosing Financial → IAS 37 Provision Trend Break..., flags any provision category whose year-over-year balance changed by more than a chosen percentage, and, where a settlement or legal-claim indicator has already been joined onto the same population, distinguishes a flagged decrease that lines up with a recorded settlement from one that does not.

1. Right-click any column header in provisions_register.csv and choose `Financial → IAS 37 Provision Trend Break...`.
2. Set Prior Year Balance Column to `prior_year`, Current Year Balance Column to `current_year`, and Settlement/Claim Flag Column to `settlement_flag`.
3. Leave the flag threshold at its default of 30.00%.
4. Click `Check Provision Trend`.

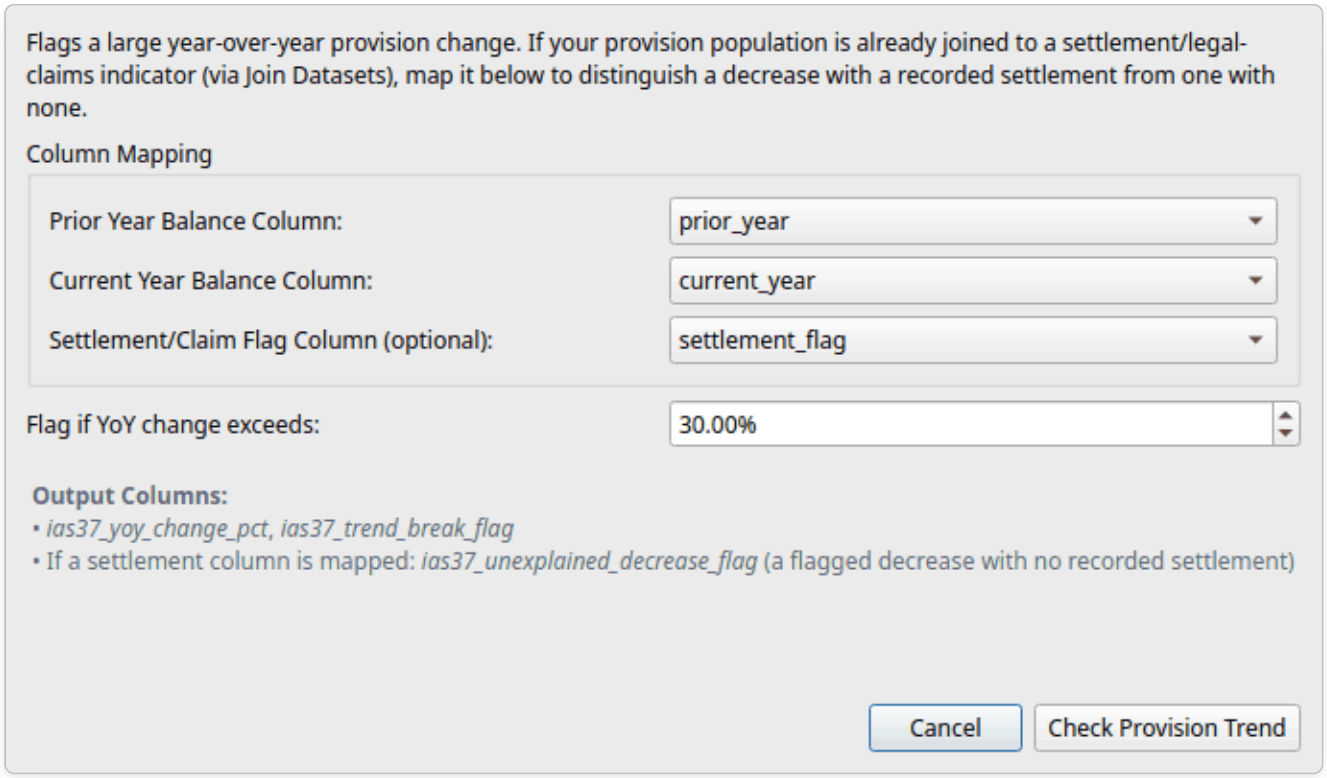


Figure 19.1 — IAS 37 Provision Trend Break configured against provisions_register.csv.

Two of the five provision categories in the working file move by more than thirty percent. Legal Claims Provision drops sixty-seven percent year over year, from $1,200,000 to $400,000, but the settlement flag on that row is set, so the decrease, while still flagged as a trend break, is not additionally flagged as unexplained. Environmental Remediation drops sixty-four percent, from $500,000 to $180,000, with no recorded settlement, and is flagged as an unexplained decrease. The distinction the settlement flag draws is exactly the distinction a reviewer would otherwise have to make by hand, row by row, against a separate legal-claims schedule.

Inventory Net Realizable Value

IAS 2 requires inventory to be carried at the lower of cost and net realizable value, its estimated selling price less the estimated costs still required to complete and sell it. IAS 2 Inventory NRV Write-Down, reached through Financial → IAS 2 Inventory NRV Write-Down…, calculates net realizable value for every SKU and flags, with a computed dollar write-down amount, any line where cost exceeds it.

Run against inventory_valuation.csv with cost mapped to cost, selling price to selling_price, costs to sell to costs_to_sell, and quantity to quantity, two of the five SKUs require a write-down:

Legacy Printer Cartridges, carried at $18.50 against a net realizable value of $13.00, a $5.50 per-unit excess across five hundred units for a $2,750.00 total write-down, and Discontinued Monitor Stands, carried at $32.00 against a net realizable value of $17.00, a $15.00 per-unit excess across one hundred fifty units for a $2,250.00 write-down. The combined write-down across the population is $5,000.00.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a distributor runs both checks together ahead of year end and finds that its largest flagged NRV write-down, on a discontinued product line, coincides with a provision category for inventory obsolescence that dropped sharply in the same period with no recorded settlement. The two findings, examined separately, each look like an isolated estimate that moved. Examined together, they suggest the same underlying fact, that a discontinued line has lost value, is being recognized inconsistently: the specific inventory items are being written down toward realistic values, while the general obsolescence provision meant to anticipate exactly this kind of loss is simultaneously being released, which has the net effect of understating the period's combined impact from the same underlying cause.

Interpreting the Result

WHEN A PROVISION BREAKS TREND OR AN INVENTORY LINE REQUIRES A WRITE-DOWN

1. **Benign explanation.** A provision decrease can reflect a genuine, favorable resolution, such as a claim withdrawn or a warranty period expiring without incident, and an inventory write-down can reflect an honestly recognized, ordinary consequence of a product reaching the end of its life cycle.
2. **Error or judgment lapse.** A provision release recorded before the underlying uncertainty was actually resolved, based on an optimistic assessment rather than a confirmed outcome, is a premature judgment call rather than a deliberate misstatement.
3. **What would indicate something more serious.** A provision release and an inventory write-down affecting the same underlying cause, recognized in a way that understates their combined effect on the period, as in the scenario above, or a pattern of unexplained provision decreases concentrated in periods immediately before a debt covenant test, warrants a coordinated review of both balances together rather than each in isolation.

Chapter 20

Tax and Currency (IAS 12, IAS 21)

This final chapter of Part IV covers two tests unrelated in subject matter but similar in shape: each takes a balance built from an estimate or a rate a preparer selected, and checks it against an independently determinable benchmark, a recoverability test for a deferred tax asset under IAS 12, and a rate-consistency check for foreign-currency transactions under IAS 21.

Deferred Tax Asset Recoverability

A deferred tax asset, commonly arising from a tax loss carried forward, is only recognized to the extent future taxable profit is probable within the window available to use it. IAS 12 DTA Recoverability Check, reached by right-clicking any column header and choosing Financial → IAS 12 DTA Recoverability Check..., inverts this test: rather than asking whether a forecast supports the recognized balance, it solves the plain algebra for the average annual taxable profit the balance itself requires, dta_balance divided by the product of the tax rate and the number of years available to use it, since a deferred tax asset is not discounted under IAS 12, unlike a lease liability or an impairment cash flow model, so no root-finding is needed here. It then compares that required figure to a benchmark profit, typically the entity's own historical average.

1. Right-click any column header in deferred_tax_register.csv and choose `Financial → IAS 12 DTA Recoverability Check...`.
2. Confirm DTA Balance Column, Tax Rate Column, and Utilization Years Remaining Column map to `dta_balance`, `tax_rate`, and `utilization_years`, all detected automatically.
3. Under Benchmark Taxable Profit, select `Use a per-row benchmark column` and set it to `benchmark_profit`.
4. Leave the flag threshold at its default of 20.00%.
5. Click `Check DTA Recoverability`.

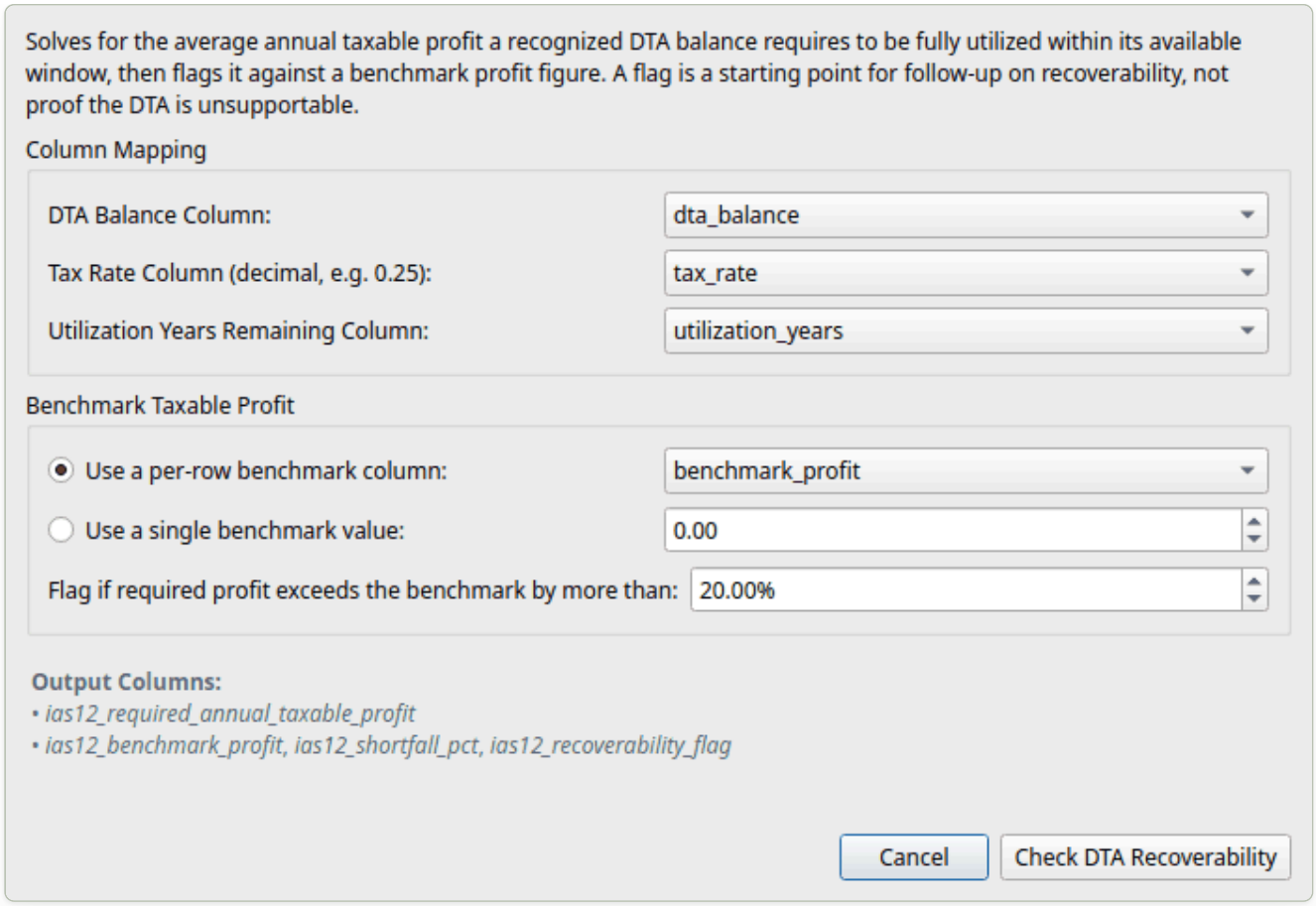


Figure 20.1 — IAS 12 DTA Recoverability Check configured against deferred_tax_register.csv, using each jurisdiction's own historical benchmark profit rather than a single figure applied to all four.

Jurisdiction	DTA Balance	Required Annual Profit	Benchmark Profit	Shortfall
Home Country HQ	4,000,000	3,200,000	3,500,000	−8.6%
Subsidiary B	800,000	601,504	900,000	−33.2%
Subsidiary A (loss carryforward)	**2,500,000**	**3,968,254**	**400,000**	**+892.1%**
Subsidiary C (new market entry)	**1,200,000**	**1,363,636**	**300,000**	**+354.5%**

Figure 20.2 — Subsidiary A must generate nearly ten times its historical average annual profit, every year, for three years straight, to fully utilize its recognized deferred tax asset. Subsidiary C, a newer operation with only one year of thin historical profit to benchmark against, requires more than four and a half times that benchmark.

Foreign Currency Rate Consistency

IAS 21 FX Rate Consistency Check, reached through Financial → IAS 21 FX Rate Consistency Check..., compares the exchange rate actually used to record each foreign-currency transaction against a benchmark rate, typically the published spot or closing rate for that date and currency pair, already joined onto the population beforehand. Run against fx_transactions.csv with booked_rate, benchmark_rate, and local_amount mapped and the tolerance left at its default of 1.0%, one transaction is flagged: FX-2002, a GBP/USD payment booked at 1.2650 against a benchmark of 1.2400, a 2.02 percent deviation, understating the transaction's recorded USD value by $2,125.00 relative to what the benchmark rate would have produced.

> A SCENARIO WORTH RECOGNIZING
>
> Suppose a multinational's treasury function books most foreign-currency receipts using a rate pulled automatically from its banking platform at the moment of settlement, but one regional office continues to key in rates manually from a printed daily rate sheet distributed each morning. Every transaction from that regional office shows a small, consistent deviation from the benchmark rate, not because any individual transaction was mishandled, but because a rate fixed at the start of the day drifts slightly from the actual settlement-time rate by the time a transaction clears in the afternoon. The pattern across many small, similarly sized deviations from one specific office is a process finding about that office's rate source, not a series of unrelated individual errors.

Interpreting the Result

WHEN A DTA'S REQUIRED PROFIT EXCEEDS ITS BENCHMARK, OR AN FX RATE DEVIATES FROM THE BENCHMARK

1. **Benign explanation.** A newly established subsidiary, like Subsidiary C above, may genuinely lack a meaningful historical profit benchmark, and a forward-looking business plan, rather than thin historical results, may be the more appropriate support for its recognized deferred tax asset.
2. **Error or judgment lapse.** An FX rate deviation traced to a single office's outdated rate source, as in the scenario above, is a process gap correctable by standardizing the rate source, not a series of individually mishandled transactions.
3. **What would indicate something more serious.** A deferred tax asset whose required profit has grown for several consecutive periods without a corresponding, documented improvement in the underlying business's actual prospects, or FX rate deviations that consistently favor a more advantageous reported result rather than scattering randomly around the benchmark, both warrant escalation beyond a routine finding.

PART V

Source Data

Chapter 21

Ratio Analysis Done Right

Chapter 3 established how a single messy number gets parsed correctly, and Chapter 9 walked through calculating ratios once every figure already sits in a clean, one-row-per-period file with the right column names. Between those two chapters lies the part of the journey most exposed to error in practice: a real financial statement rarely arrives as a clean wide table. It arrives as a trial balance with one row per account, spread across a filing that separates the income statement, the balance sheet, and the cash flow statement into three different exports entirely. This chapter closes that gap, covering how CaatMaster maps an arbitrary account label to the right concept without guessing, and how it recombines statements that were never in the same file to begin with.

Mapping a Line Item to a Concept, Without Guessing Wrong

Every automatic concept suggestion in CaatMaster, whether it appears in the Financial Import Wizard from Chapter 3, the Auto-Detect buttons in the Financial Ratios dialog from Chapter 9, or the reshape operation covered below, runs through the same function: a label such as an account name or a column header is checked against a library of patterns, one set per financial concept, and the pattern that matches most specifically wins. Specificity is measured by the length of the matching pattern itself, which is what correctly resolves a label like “Total Current Assets” to current assets rather than the shorter, more generic pattern for total assets, without either pattern needing to be manually ranked against the other. Where two different concepts match a label equally specifically, the function returns nothing rather than picking one arbitrarily, on the reasoning that a silently wrong guess here would feed directly into every ratio calculated from it, and an admitted ambiguity is safer than a confident mistake.

Label	Suggested Concept
Total Current Assets	current_assets
Total Assets	total_assets
Retained Earnings	retained_earnings
Total Shareholders Equity	total_equity
Operating Income	ebit
Total Revenue	revenue

Label	Suggested Concept
Accrued Liabilities	*(none — left for manual review)*

Figure 21.1 — Real output of CaatMaster's concept-suggestion function against a set of representative statement labels. Retained Earnings resolves correctly to its own concept rather than being folded into total equity, and Accrued Liabilities, a real account with no corresponding ratio concept, is correctly left unresolved rather than forced into the nearest-sounding option.

Reshaping a Row-Based Statement

Reshape Statement to Ratio Inputs, reached by right-clicking any column header and choosing Financial → Reshape Statement to Ratio Inputs…, bridges a statement with accounts as rows into the wide, concept-as-column shape Ratio Analysis needs. Loading the unique account values from a chosen column, it proposes a concept for each one automatically, but only for a row that looks like a subtotal or aggregate line in the first place, such as one containing the word "Total." A detail-level row, one without that kind of language, defaults to Ignore even where the concept-suggestion function above could resolve it confidently on its own. This is a deliberate, second layer of caution beyond the specificity ranking already described: mapping both an explicit subtotal, such as Total Current Assets, and one of the detail accounts that roll up into it, such as Cash, to concepts at the same time would double-count the same dollars once the two are separately summed into a ratio input, and the dialog would rather leave a real, resolvable row unmapped than risk that silently.

1. Right-click any column header in meridian_trial_balance_q1.csv and choose `Financial → Reshape Statement to Ratio Inputs...`.

2. Set Account/Line Item Column to `account`, check `balance` under Value Column(s), and set Period Column to `period`.

3. Click `Load Unique Accounts`. Six rows whose label contains “Total” are mapped automatically; six are not.

4. Review the six unmapped rows individually. Set Cash and Cash Equivalents to `cash`, Interest Expense, Net to `interest_expense`, Cost of Goods Sold to `cogs`, Retained Earnings to `retained_earnings`, and Operating Income to `ebit`. Leave Accrued Liabilities at `Ignore`, since no ratio concept corresponds to it.

5. Click `Reshape Dataset`.

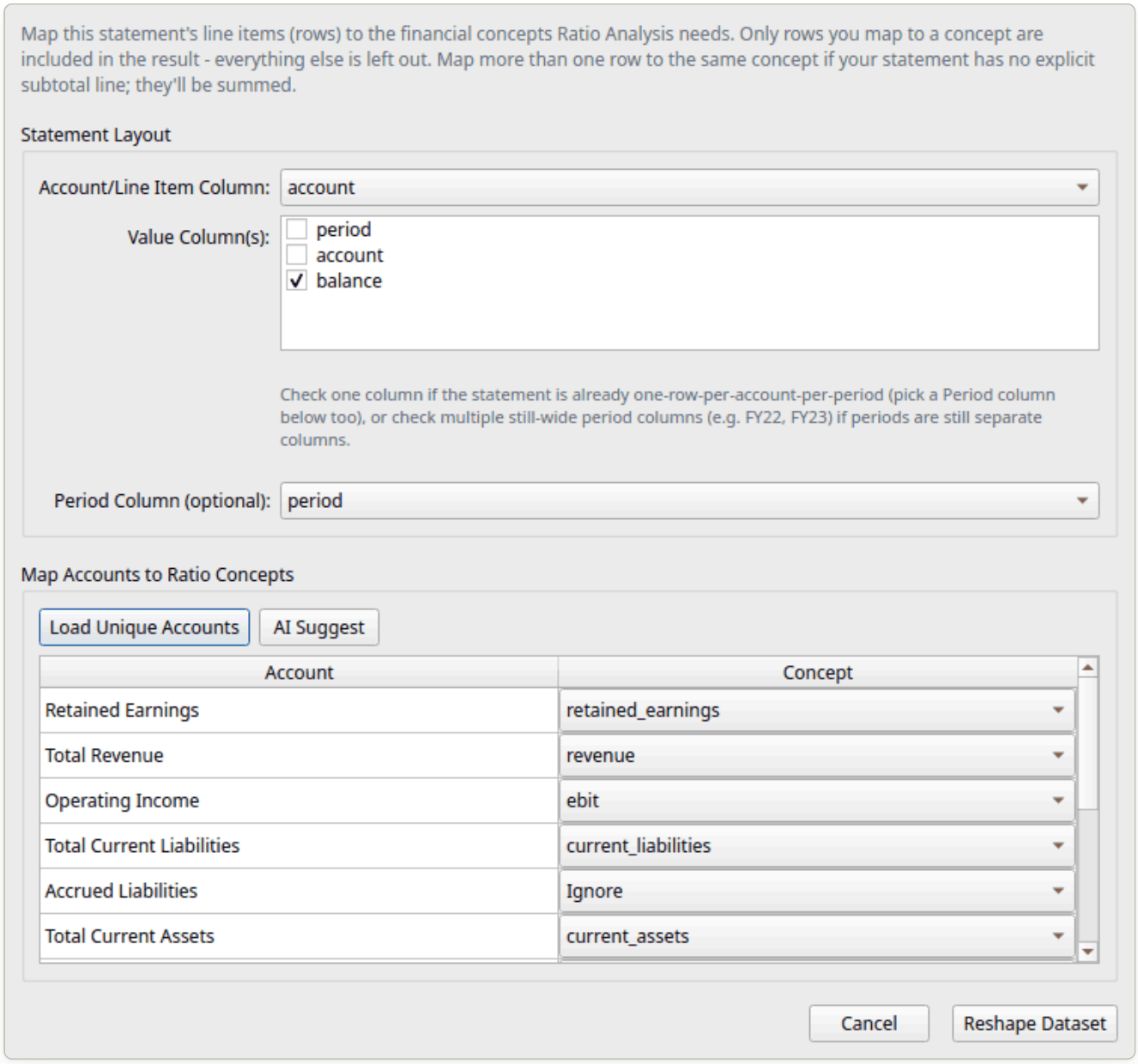


Figure 21.2 — Reshaping meridian_trial_balance_q1.csv. Retained Earnings, Total Revenue, Operating Income, and Total Current Liabilities are visible here, three auto-suggested from their subtotal-style wording and one, Operating Income, corrected by hand.

The result is a single row for the 2025-Q1 period, carrying eleven mapped concept columns, with Accrued Liabilities correctly absent from the output rather than silently zeroed or guessed into an unrelated concept. That row is now the same shape Chapter 9's ratio calculation expects, regardless of the fact that it began life as twelve separate account rows rather than a single wide line.

Combining Statements That Arrive as Separate Files

The Financial Ratios dialog from Chapter 9 accepts more than one dataset at once specifically for this reason. Its Select Dataset for Each Statement row lets the income statement, balance sheet, and cash flow statement each be pointed at a different open dataset, rather than assuming all three already live in one file; a statement left at Current Dataset simply uses whichever dataset the dialog was opened against.

1. Import income_statement.csv and balance_sheet.csv as two separate datasets, each holding one quarter's figures for Meridian Office Supply Co.
2. Open `Financial Ratios` against either dataset.
3. Under Select Dataset for Each Statement, set Income Statement (P&L) to `Income Statement` and Balance Sheet to `Balance Sheet`. Leave Cash Flow Statement at `Current Dataset`, since this example has no separate cash flow file.
4. Map each statement's columns to concepts as in Chapter 9, then select the ratios to calculate.

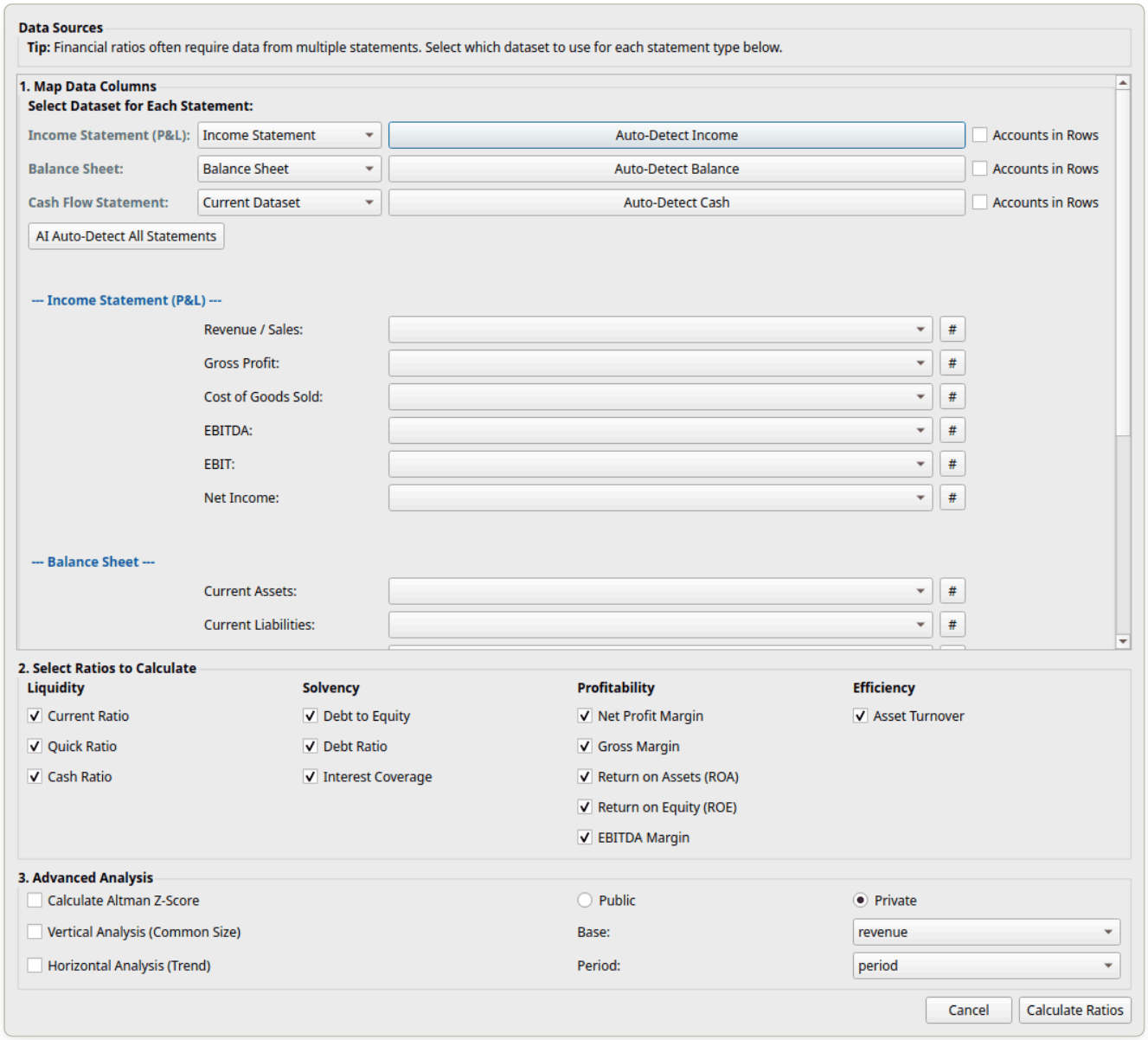


Figure 21.3 — Two separate datasets, an income statement and a balance sheet, pointed at from the same Financial Ratios run.

CaatMaster aligns the two statements by period label before combining them into a single base table, rather than simply pairing up whichever rows happen to occupy the same position in each file, which matters as soon as either statement covers more than one period. Calculated from these two separate files, the resulting ratios, a current ratio of 1.420, a quick ratio of 0.727, a net profit margin of 5.9 percent, and a gross margin of 37.4 percent, match Chapter 9's fourth-quarter figures exactly, because the two chapters describe the same underlying numbers arriving by two different routes.

Interpreting the Result

WHEN A CONCEPT IS LEFT UNMAPPED, OR TWO STATEMENTS WILL NOT COMBINE

1. **Benign explanation.** A genuinely unusual account, or a chart of accounts with its own internal terminology, may simply not correspond to any of CaatMaster's standard ratio concepts, and Ignore is the correct, final answer for that row rather than a gap to force closed.
2. **Error or judgment lapse.** Two statements that fail to combine because their period labels do not match, such as one file using calendar quarters and the other using a fiscal year ending mid-year, points to a labeling inconsistency between two source systems rather than any error in the underlying figures themselves.
3. **What would indicate something more serious.** A material concept, such as revenue or total assets, that repeatedly fails to auto-map across many consecutive periods from the same source, forcing a manual override every time, suggests the source system's own export or chart of accounts has changed in a way that deserves its own investigation, separate from whatever ratio work prompted the reviewer to notice it.

Chapter 22

Reading XBRL Directly

Most regulators now require financial statements to be filed in XBRL, eXtensible Business Reporting Language, or its human-readable variant, inline XBRL, in which the same tags are embedded directly inside an ordinarily formatted HTML filing rather than a separate machine-only document. Every numeric fact in an XBRL filing is tagged against a published taxonomy, US-GAAP or IFRS being the two most common, which means the figures a company reports are, in principle, already machine-readable without anyone needing to copy a number out of a PDF or a spreadsheet export at all. CaatMaster reads both formats directly, using the Arelle library, an open-source XBRL processor with a multi-decade history in regulatory filing software, and maps the tags it finds onto the same standard concepts every other chapter in this part of the book has used.

Running an Import

Import XBRL / iXBRL is reached from the top menu bar through File → Import XBRL / iXBRL..., a genuine top-level command rather than something buried in a right-click menu, reflecting how central a filing import is to this workflow. The working file for this chapter, test-multi-period.xml, is a small instance document carrying three Revenue facts: the current year, the prior year, and a current-year figure reported against one specific business segment.

1. From the top menu bar, choose `File → Import XBRL / iXBRL...`.
2. Click `Browse...` and select the instance document.
3. Click `Parse File`.

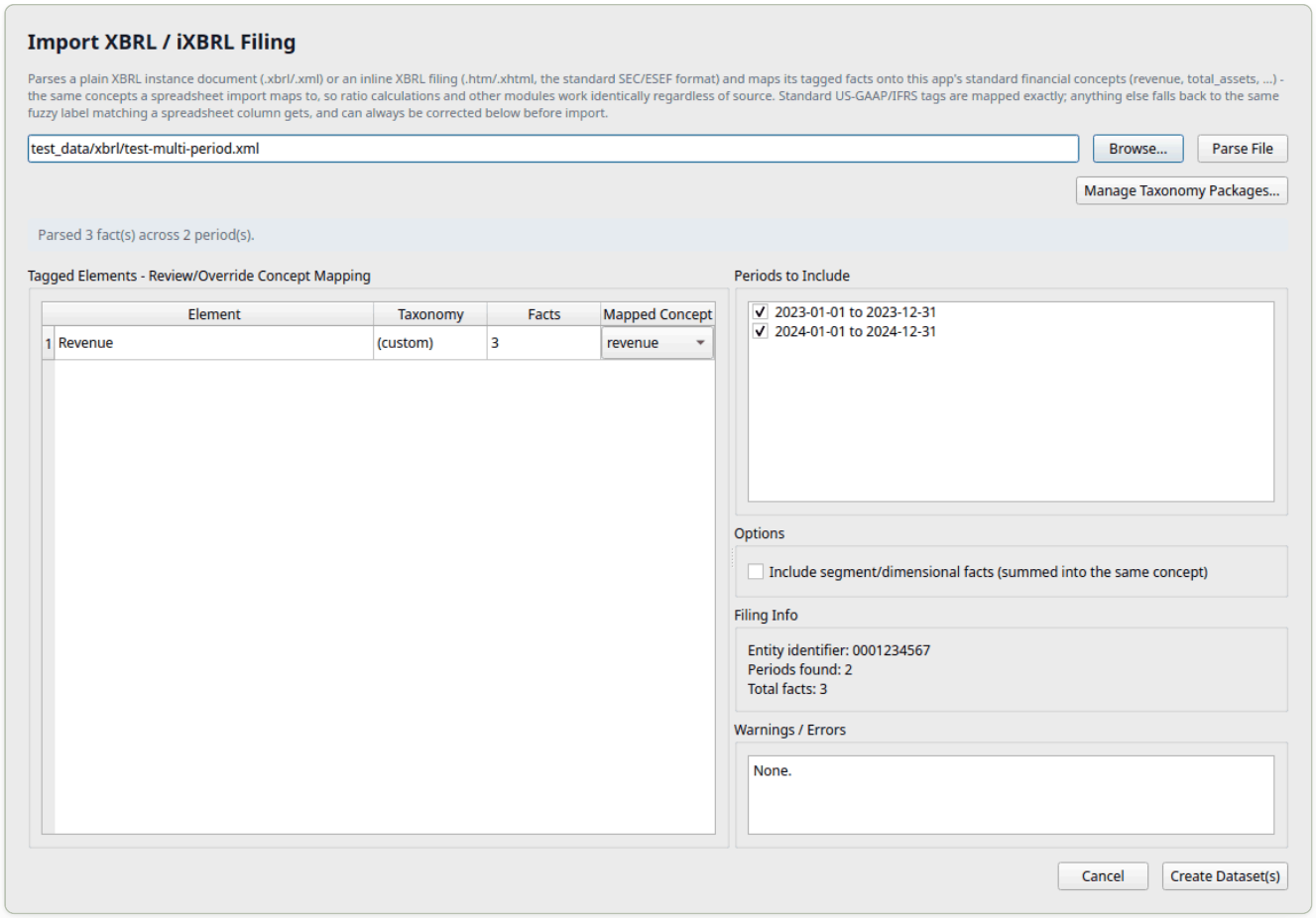


Figure 22.1 — A completed parse of test-multi-period.xml. The filing tags its Revenue element against a private, company-specific taxonomy rather than US-GAAP or IFRS, shown as (custom) in the Taxonomy column; CaatMaster still maps it correctly, through the same fuzzy label matching a spreadsheet column name would get, rather than only recognizing the two standard taxonomies it has an exact table for.

The Tagged Elements table lists every distinct element the filing uses, how many individual facts were tagged against it, and CaatMaster's proposed concept mapping, editable in place exactly like every other concept mapping in this part of the book. The Periods to Include list shows both the current and prior year, each independently selectable for the eventual import. The three facts parsed here include one tagged against a segment dimension rather than the whole entity; left at its default, the Include segment/dimensional facts option keeps that fact out of the resulting dataset, since a segment-level Revenue figure and the consolidated Revenue figure for the same period are not meant to be summed together, and checking that option here would silently produce a period total nearly fifty percent too high, the segment figure counted once as part of the consolidated total and then a second time on its own.

When a Filing Needs a Taxonomy This Machine Cannot Reach

An XBRL filing's own instance document does not contain its taxonomy; it only references one, by a URL that Arelle must resolve, ordinarily by fetching it over the network. On a machine with no internet access, or simply a taxonomy publisher's server that is temporarily unreachable, this reference cannot resolve, and the import fails outright rather than falling back to a partial or approximate result.

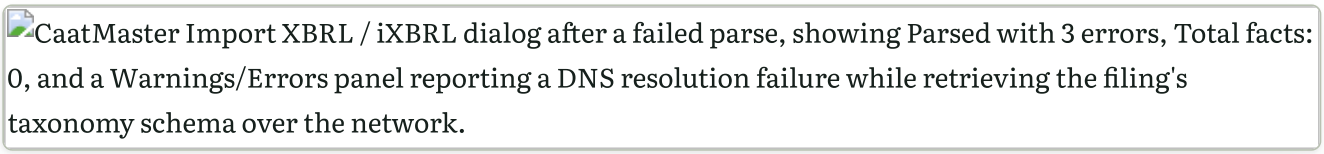

CaatMaster Import XBRL / iXBRL dialog after a failed parse, showing Parsed with 3 errors, Total facts: 0, and a Warnings/Errors panel reporting a DNS resolution failure while retrieving the filing's taxonomy schema over the network.

Figure 22.2 — The same dialog, given a filing whose schema reference points to a URL this machine cannot resolve. Total facts drops to zero and Create Dataset(s) is disabled; the underlying error, a real DNS lookup failure, is preserved and shown directly rather than replaced with a generic message.

Manage Taxonomy Packages, reached from the same File menu entry that opens the import dialog, or from the Manage Taxonomy Packages… button inside it, registers a local copy of a taxonomy so every future parse resolves it from disk first, before any network attempt is made at all. A Taxonomy Package is a standard, spec-defined zip file, the same format the SEC and the IFRS Foundation both publish their own official taxonomies in, containing the schema and linkbase files together with a catalog that maps each remote URL a filing might reference back to a local file inside the package.

1. From the Import XBRL / iXBRL dialog, click `Manage Taxonomy Packages...`.
2. Click `Add Package...` and select the taxonomy package's zip file.
3. Close the dialog and parse the filing again.

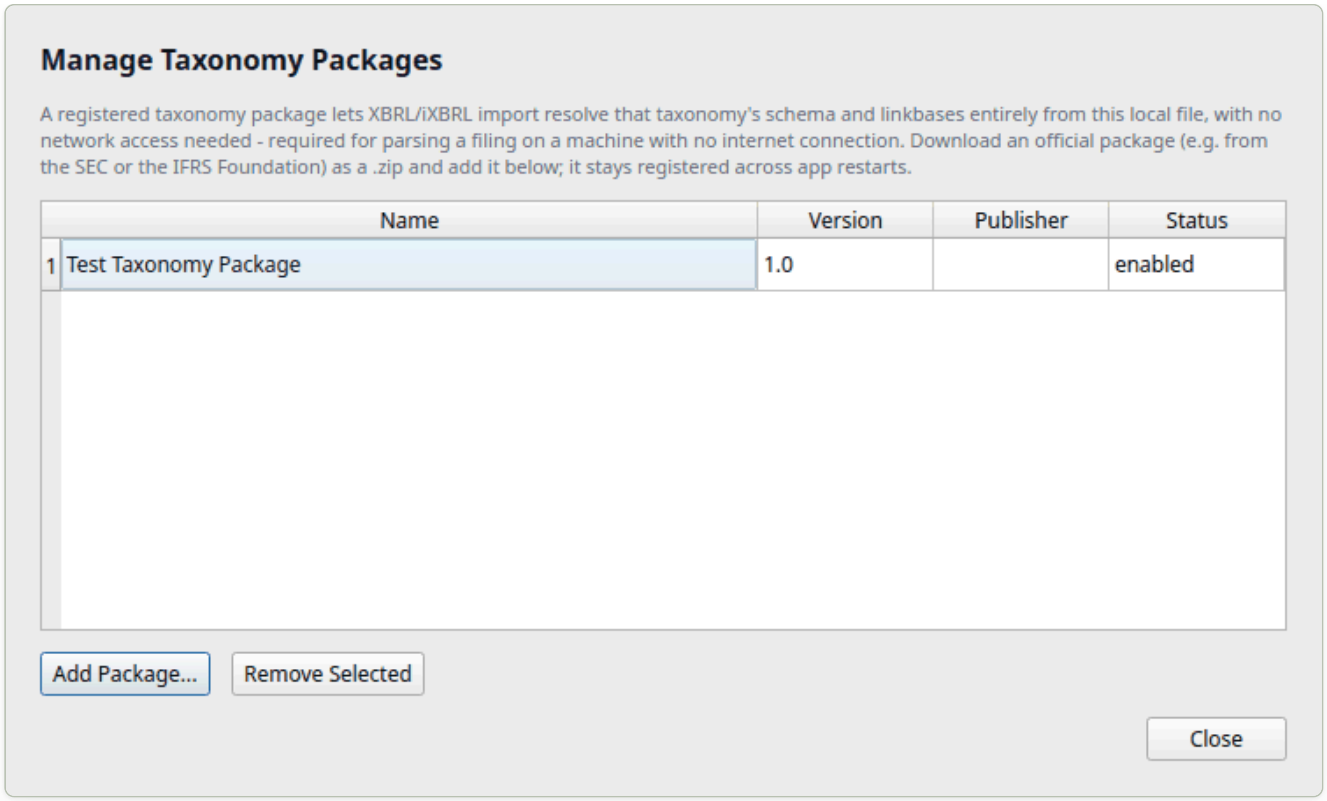


Figure 22.3 — A taxonomy package registered for local-first resolution. This registration is stored in Arelle's own configuration file and persists across CaatMaster restarts.

With the package registered, reparsing the identical filing that failed above now succeeds, with zero errors and one fact recovered, entirely without a network request ever being attempted, since the catalog inside the package resolves the filing's schema reference to a local file before Arelle ever considers reaching out over the network. This is not a fallback that activates only after a network attempt times out; the local file is checked first, which is what makes the technique usable on a machine with no internet access at all, not merely one with a slow connection.

Interpreting the Result

WHEN A TAGGED FACT WILL NOT MAP TO A CONCEPT, OR A FILING WILL NOT RESOLVE

1. **Benign explanation.** A filer's own custom extension taxonomy, used for a line item specific to that company's business, has no standard concept to map to at all, and Ignore, or an unmapped element left out of the final import, is the correct outcome rather than a gap to close.
2. **Error or judgment lapse.** A resolution failure on a machine that does have internet access more often traces to a taxonomy publisher's server being temporarily unreachable than to anything wrong with the filing itself; retrying later, or registering a local package proactively, resolves it without treating the filing as defective.
3. **What would indicate something more serious.** A standard concept, such as revenue or total assets, that a filing tags using a heavily customized extension element rather than the standard US-GAAP or IFRS tag every comparable filer uses for the same line item, warrants a direct look at why that specific figure was pulled out of the standard taxonomy.

PART VI

AI, Used Responsibly

Chapter 23

The Grounded AI Pattern

Chapter 4 stated the governing rule behind every AI-assisted feature in CaatMaster: a number is never asked of a language model when Polars can compute it exactly. This chapter and the three that follow return to that rule with the detail Chapter 4 deliberately left out, covering how a provider is configured, what CaatMaster actually sends and receives on every call, where that exchange is recorded, and the specific, tested failure modes the translate-then-compute pattern exists to close off.

Nine Providers, One Interface

CaatMaster does not build its own language model or commit to a single vendor. It supports nine provider integrations, OpenAI, Anthropic, Google Gemini, DeepSeek, Hugging Face, Ollama, LlamaCPP, OpenRouter, and a generic custom endpoint for anything compatible with the OpenAI API shape, behind one shared interface. Two of these, Ollama and LlamaCPP, are local providers: the model runs on the reviewer's own machine rather than a remote service, which means no API key is required and no data leaves the machine at all, at some cost in the scale of model available compared to a hosted frontier model. Configuration for any provider is reached from the top menu bar through AI → AI Settings, later renamed in some builds simply Settings, one tab per provider, each holding an API key, an optional custom endpoint URL, a model identifier, and a Test Connection button that makes one real, minimal call before anything else is allowed to depend on that provider.

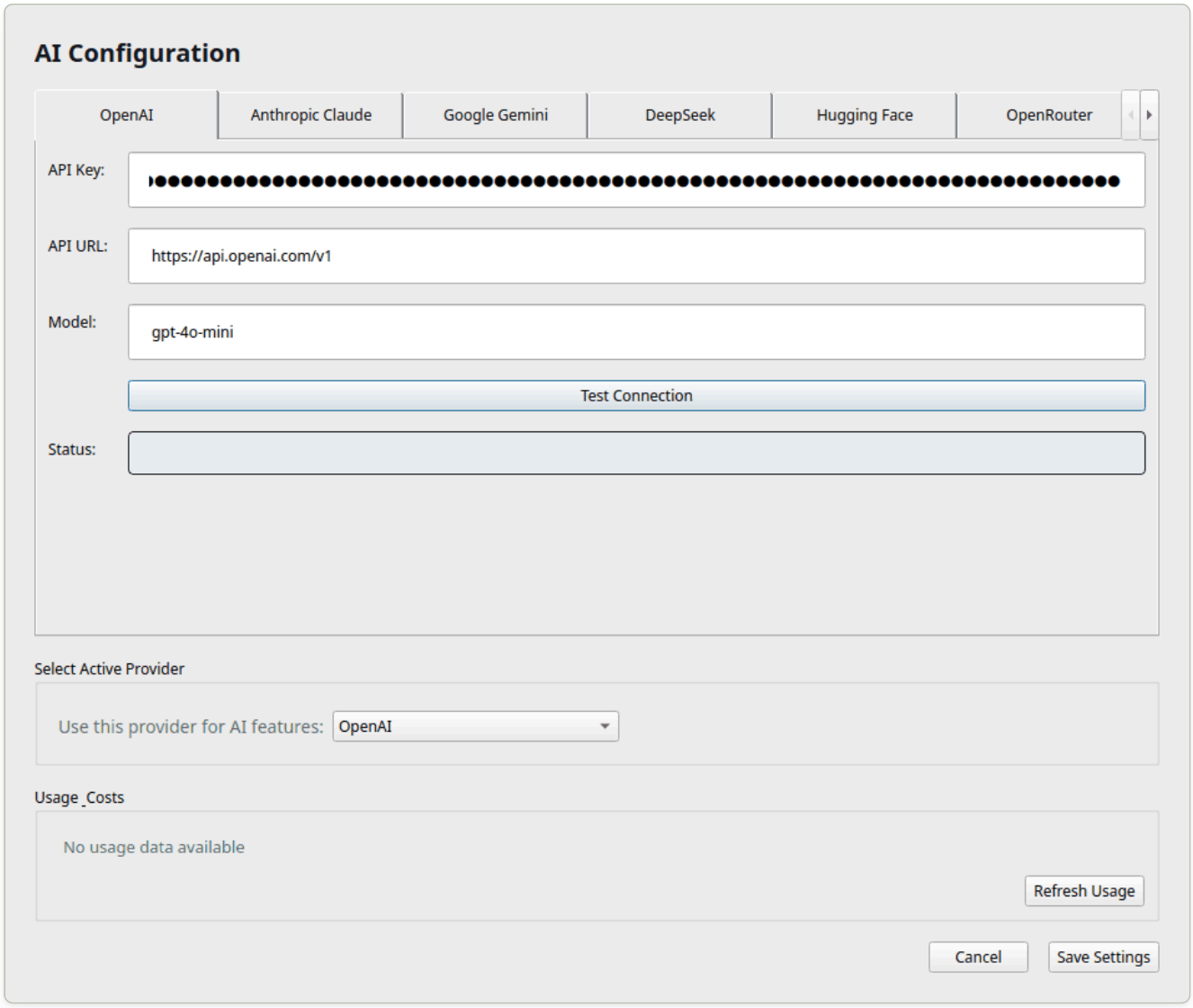


Figure 23.1 — The AI Configuration dialog. Every provider tab follows the same shape: a credential, an endpoint, a model, and a way to verify the connection before relying on it.

Whichever provider is marked active under Select Active Provider is the one every AI-assisted operation in the application uses, until changed again. Nothing about which provider is active changes what any operation is permitted to do; every constraint described in the rest of this chapter applies identically regardless of which of the nine sits behind the interface.

Where a Credential Actually Lives

An API key typed into the AI Configuration dialog is never written to disk in plain text. It is encrypted at rest using Fernet, a symmetric encryption scheme built on AES, with the encryption key itself derived from a per-installation secret generated the first time CaatMaster runs, the same secret-key mechanism described in Chapter 3's discussion of how the

application's own credential database is protected. This is not a separate, newly added safeguard; it is the same discipline applied consistently to every category of secret the application holds, whether that secret authenticates a user or authorizes a call to a third-party AI provider.

What Actually Gets Logged

Every real call to a language model, regardless of which of the nine providers handles it or which feature triggered it, is appended to an audit log on disk, one JSON record per call: the full prompt sent, the provider and model that answered, the complete response text, token usage, and a timestamp, whether the call succeeded or failed. This log exists because a tool whose entire premise is auditability cannot itself be a black box; what an AI feature actually saw and actually said back has to be reviewable after the fact by someone other than the person who ran it, not merely trusted. AI → AI Activity Log… opens this log directly inside the application, and AI Usage Statistics, reached from the same menu, summarizes token consumption and estimated cost across providers over a chosen period.

The Translate-Then-Compute Pattern, Tested

Chapter 4 described the general shape of this pattern: a language model translates unstructured input into a structured, checkable statement, and that statement is then evaluated against the complete dataset in ordinary code. What Chapter 4 did not cover is that this pattern, in CaatMaster, was refined against the failure modes of a real model rather than designed once from first principles and left alone. The rule-translation prompt behind Chapter 14's deterministic libraries and the next chapter's Policy Check tab carries two specific corrections, each added after a real large language model produced a specific, wrong answer during testing.

The first failure was truncation: a compound rule, such as "payments over one thousand dollars must have an approver on file," genuinely contains two separate testable criteria, an amount threshold and a missing-approver check, but an early version of the translation prompt reliably captured only the first and dropped the second, since nothing in the prompt specifically instructed the model to look for more than one criterion per rule. The second failure was fabrication: a rule that is fundamentally about comparing one row to every other row, such as "no transaction should be recorded twice," cannot be expressed as a single row's own filter condition at all, and rather than recognizing this, the model invented an arbitrary, specific-looking condition instead of reporting the rule as not computable. Both failures are now named directly in the prompt itself, with the exact reasoning a reviewer would use to catch them, and

duplicate-style rules are routed to a dedicated, genuinely row-comparing check, computed through Polars' own duplicate-detection logic rather than left for the model to approximate.

> A STANDING INSTRUCTION, NOT A SUGGESTION
>
> The rule-translation prompt states plainly: an honest "not computable" is always better than a fabricated condition. This is not a stylistic preference. A wrong but plausible-looking filter condition produces a wrong count that looks exactly like a right one, while a rule correctly marked not computable is visibly incomplete and prompts a human to look at it, which is precisely the difference this book has treated as central since Chapter 1.

Batching, and What Happens With No Provider at All

A column-level AI operation, such as generating a plain-language explanation for every value in a column, does not make one language model call per row. Rows are grouped into batches, twenty-five at a time by default, and each batch is sent as a single call asking for one structured result per row in the batch, which keeps a thousand-row column to roughly forty calls rather than a thousand. Where a specific batch fails, whether from a malformed response or a provider error, that batch's rows fall back to a placeholder value rather than aborting the entire column; every other batch that succeeded keeps its real result. Where no AI provider is configured at all, CaatMaster does not attempt a call and silently return an empty-looking result: every batching operation checks for a configured provider first and returns the same fallback value for every row immediately, so a reviewer who has not set up a provider sees an unmistakably incomplete result rather than something that could pass for real output.

Interpreting the Result

This chapter has described the machinery every AI-assisted feature in the rest of this book depends on, rather than produced a finding of its own. The discipline worth carrying forward is narrower and more durable than any one feature: before treating a number that came out of an AI-assisted step as evidence, confirm it followed the path this chapter describes, translated into a structured statement, computed in Python against the complete population, and logged. A number that took a shortcut around that path deserves the scrutiny a rough estimate deserves, not the confidence a computed figure earns.

Chapter 24

AI-Assisted Text and Document Analysis

Some of the most useful applications of a language model inside an audit tool have nothing to do with arithmetic at all: reading a free-text description and classifying it, drafting a plain-language explanation of what a column of codes actually represents, or reading a written policy and turning it into something a computer can test. This chapter covers the AI-assisted operations built for exactly this kind of task, every one of them reached from the top menu bar's AI menu as well as by right-clicking any column header and choosing AI, and the Policy Check tab that extends the same translate-then-compute discipline from Chapter 23 to a policy written in the reviewer's own words rather than pulled from a pre-built library.

Column-Level AI Operations

Six operations share the AI category: AI Explain drafts a plain-language interpretation of each value in a column, one or two sentences depending on the style chosen. Sentiment Analysis scores free text on a positive-to-negative scale. Extract Entities pulls a specific kind of value, an email address, a phone number, a date, or a monetary amount, out of unstructured text. AI Text Classification sorts free text into a fixed list of categories the reviewer supplies. AI Summarize condenses a longer piece of text to a target length. AI Enrich adds a specific kind of derived information, an inferred industry, sentiment, category, or entity, depending on which enrichment type is selected. Every one of these produces a new column, never modifies the source data, and, per Chapter 23, runs in batches of twenty-five rows per call rather than one call per row.

1. Right-click any column header in the AP Transactions dataset and choose `AI → Text Classify`.
2. Set Text Column to `description`.
3. Set Categories to a comma-separated list: `Office Supplies, Professional Services, Facilities, Technology, Travel`.
4. Leave Category Column and Confidence Column at their defaults.
5. Click `OK`.

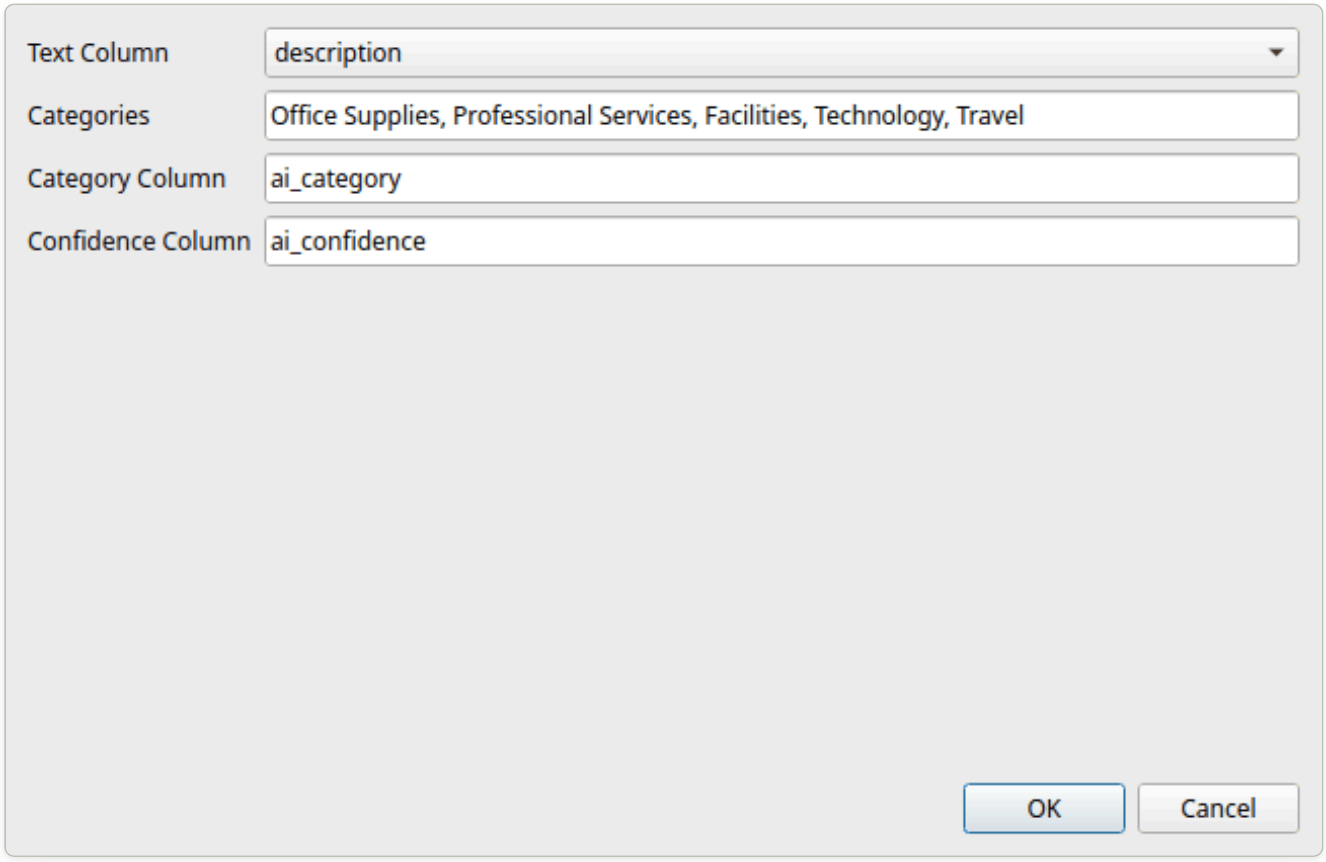


Figure 24.1 — AI Text Classification configured against the AP Transactions dataset's free-text description column.

Run against a real provider, this produces one predicted category and one confidence score per transaction description, batched twenty-five rows to a call as described in Chapter 23. A category

assignment is a classification, not a computation the way a ratio or an outlier flag is; it belongs in an audit workpaper as a starting point for grouping and sampling, not as a substantive finding on its own, and a low-confidence assignment is worth a manual glance before being relied on for anything.

Policy Check: Translating a Written Policy on the Spot

Chapter 14 covered three pre-built rule libraries, each shipped with its checkable condition already written by hand against logical field names. Policy Check, the first tab of the Compliance Report Generator, reached from the top menu bar through Reports → LLM Analysis..., exists for the much more common case: a policy that has not been pre-encoded into a library at all, typed or pasted directly from the reviewer's own compliance documentation.

1. From the top menu bar, choose `Reports → LLM Analysis...` against the AP Transactions dataset, and open the `Policy Check` tab.
2. Type or paste the policy: `All payments of $300 or more must have an approver recorded before disbursement.`
3. Click `Run Compliance Check`.

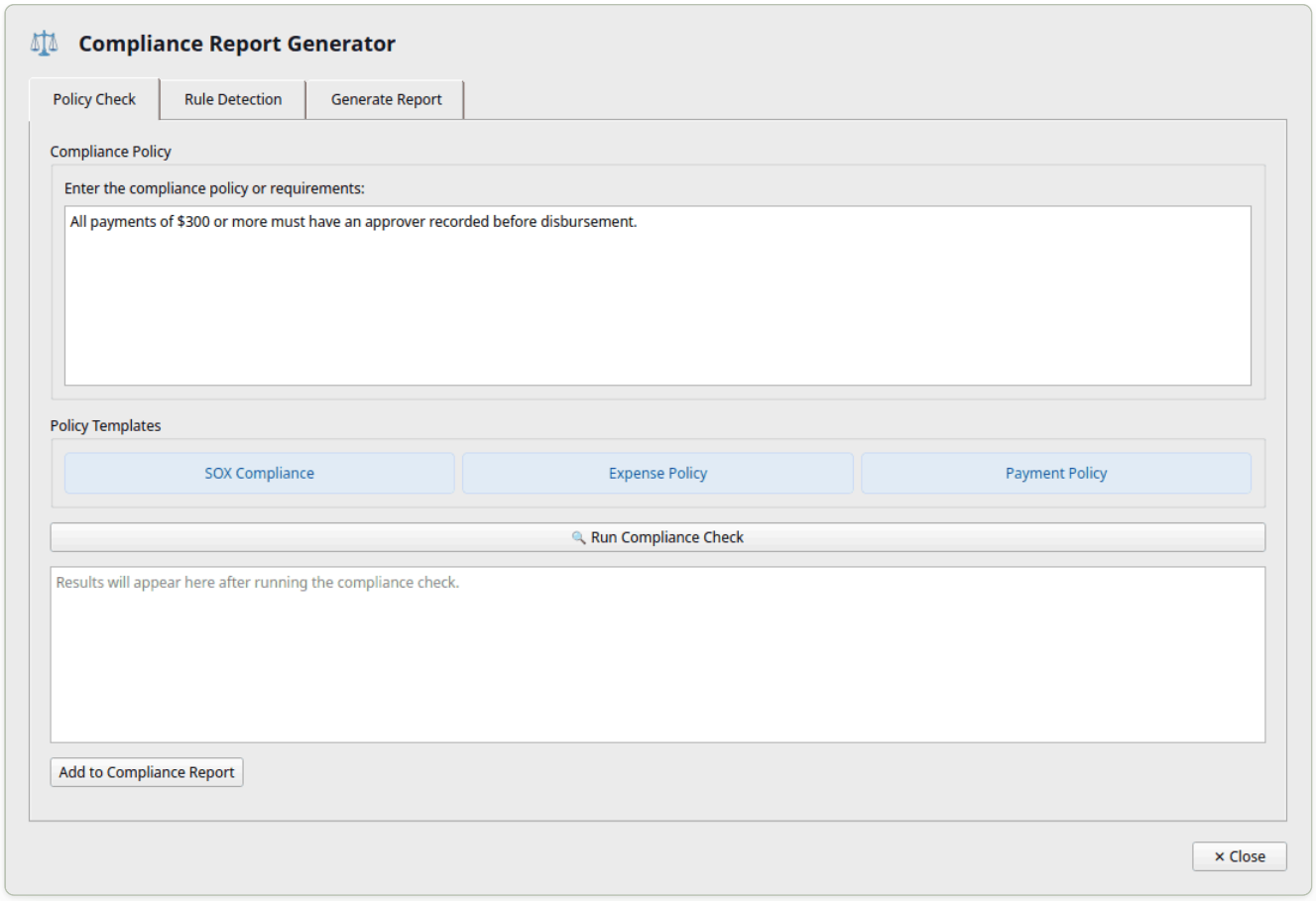


Figure 24.2 — A policy typed directly into Policy Check, phrased almost identically to Chapter 14's Payment Without Approval on File rule.

Behind this button sits exactly the translate-then-compute pattern described in Chapter 23: the language model reads the policy text and returns a structured condition, amount greater than or equal to three hundred, approver is null, combined with AND, and that condition is then evaluated against the complete dataset in Python, the same way Chapter 14's pre-built rule evaluates it, and the same way it would flag the same five records this book has now traced through three separate chapters. The difference between this tab and Chapter 14's rule libraries is not the computation, which is identical; it is that a rule library's condition was written once by a person and reused indefinitely at zero AI cost, while Policy Check translates a policy fresh every time it runs, which is the right tradeoff for a policy that genuinely is being seen for the first time, and the wrong one for a policy tested every reporting period, where encoding it once as a reusable library rule, as Chapter 14 describes, is both cheaper and faster.

Interpreting the Result

WHEN AN AI-DRAFTED CLASSIFICATION OR TRANSLATED CONDITION LOOKS WRONG

1. **Benign explanation.** A text classification genuinely near a category boundary, such as a purchase that is plausibly either Office Supplies or Facilities, can receive a low-confidence or borderline label without any defect in the classifier; free text is often genuinely ambiguous.

2. **Error or judgment lapse.** A translated policy condition that omits part of a compound requirement, the exact truncation failure described in Chapter 23, is a translation error the reviewer can catch immediately by reading the structured condition the tool shows before it runs, which is exactly why that condition is always shown rather than hidden behind the final count.

3. **What would indicate something more serious.** A classification or translated condition that is wrong in the same direction repeatedly, always under-flagging a specific vendor or category rather than erring randomly, warrants checking whether the input text itself, not the AI step, is the actual source of the pattern.

Chapter 25

Numeric Disclosure Reconciliation

A note to the financial statements routinely states a policy figure in prose: a depreciation rate, a useful life, a loss given default assumption. Confirming that figure actually matches what the underlying records show has traditionally meant a reviewer reading the note by eye and separately querying the data by hand. Numeric Disclosure Reconciliation automates the reading, using AI strictly to find and transcribe each figure exactly as written, and leaves the comparison itself, along with the verdict on whether it matches, to ordinary Python arithmetic against a real dataset column the reviewer maps in.

Extraction Finds a Figure; It Never Judges One

The extraction step, reached from the top menu bar through Reports → Numeric Disclosure Reconciliation…, is deliberately narrow. Given a block of narrative text, pasted directly or loaded from a PDF, it returns a list of figures, each carrying a short label, the value exactly as written, a unit, and the verbatim sentence the figure came from. It is not asked whether a figure is correct, is not asked to resolve a stated range such as "three to five years" to a single number, and is not asked to convert units; a range is extracted as a range and left for the reviewer to judge, since resolving it to one number would be the model quietly putting words in the disclosure's mouth.

Suppose a note to Meridian Office Supply Co.'s financial statements states: *"The Group estimates expected credit losses using a loss given default assumption of 45% across the retail loan portfolio."* Extraction returns a single figure: label "Loss Given Default assumption," value text "45%," unit percent, with that exact sentence preserved as the context snippet. Nothing about this step touches loan_portfolio.csv, the working file from Chapter 16, at all; it only reads the sentence.

Reconciliation Is Arithmetic, Not Another AI Call

Once a figure has been extracted, the reviewer maps it to a real dataset and column, and the comparison that follows never involves a language model again. CaatMaster computes the real mean, minimum, maximum, and distinct-value count of the mapped column, converts the

disclosed figure to the same decimal scale where the unit is a percentage, and flags a mismatch only where the deviation clears two thresholds together: an absolute difference, 0.005 by default, half a percentage point for a rate, and a relative difference, ten percent by default, of the actual value. Both thresholds have to be cleared before a flag is raised, which keeps an immaterial rounding difference on a small-magnitude figure from being flagged purely because the relative percentage looks large.

1. Paste the disclosure text, or click `Extract Figures` after loading a PDF.
2. For each extracted figure's row, set Dataset to `loan_portfolio` and Column to `lgd`.
3. Leave the tolerances at their defaults, half a percentage point absolute and ten percent relative.
4. Click `Reconcile Against Real Data`.

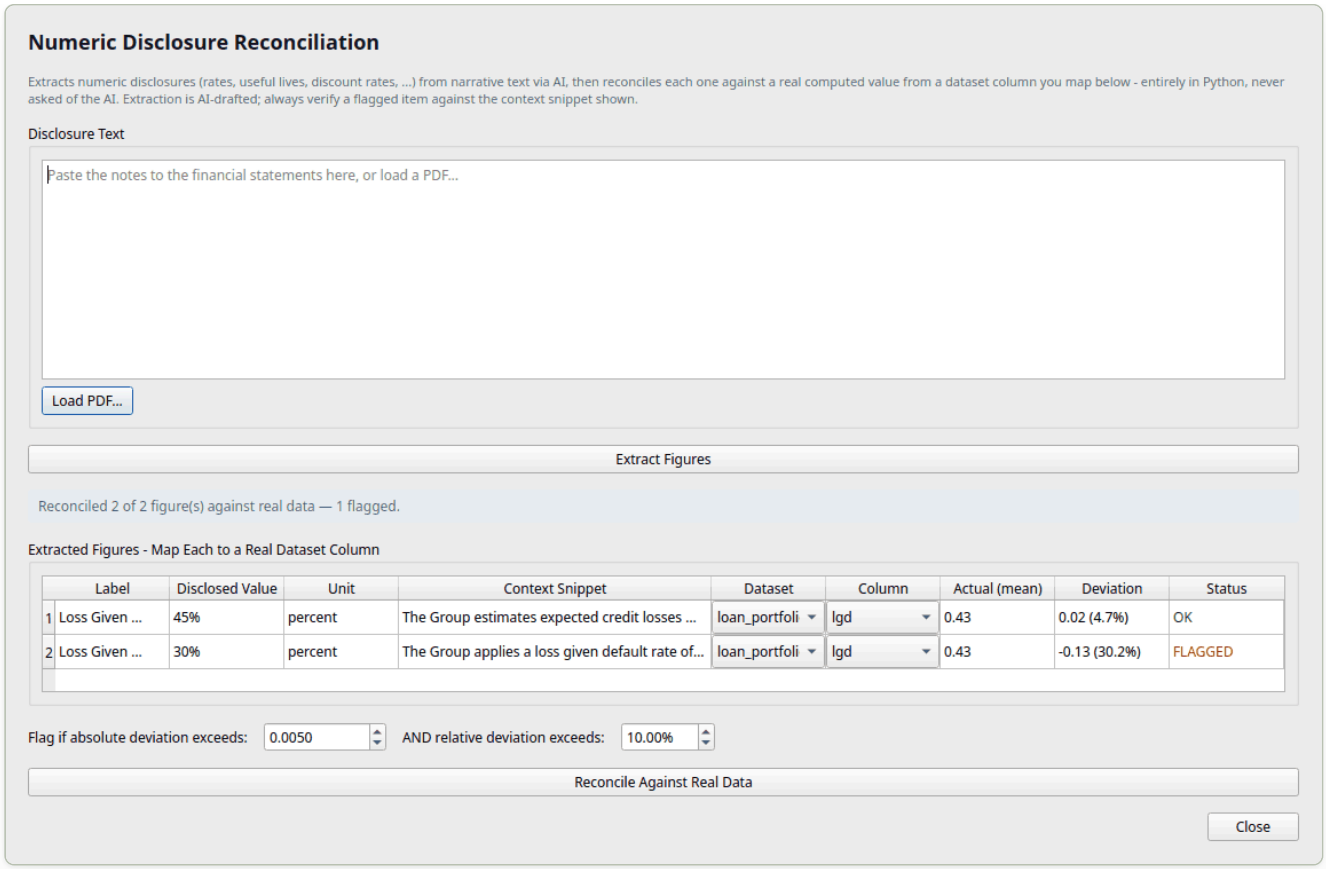


Figure 25.1 — Two disclosed figures reconciled against the real mean of loan_portfolio.csv's lgd column, 0.43. The 45 percent disclosure deviates by 4.7 percent relative and is not flagged; a hypothetical 30 percent disclosure of the same policy deviates by 30.2 percent relative and is flagged.

The first figure, forty-five percent against a real portfolio mean of forty-three percent, deviates by two-tenths of a percentage point in absolute terms and 4.7 percent in relative terms, clearing neither threshold, and reconciles cleanly. A second, hypothetical version of the same disclosure, thirty percent against the same forty-three percent actual mean, deviates by thirteen full percentage points, 30.2 percent in relative terms, and is flagged. Both results are exact, reproducible arithmetic against loan_portfolio.csv's real values; the only thing that changed between them is which sentence a person, not the model, decided to disclose.

A SCENARIO WORTH RECOGNIZING

Suppose a reviewer reconciles a full set of notes to the financial statements ahead of year end and finds a single flagged item: a stated depreciation rate of ten percent for office equipment, against a real portfolio mean, computed from the fixed asset register, of six and a half percent. Rather than treating this as a single wrong sentence, the reviewer maps the same disclosed rate against the register filtered to equipment purchased in the current year alone, and finds the current year's own mean rate sits at 9.8 percent, close to the disclosed figure. The note's stated rate turns out to describe the rate applied to new purchases going forward, not a blended rate across the entire existing asset base, which the original, unfiltered mapping could not distinguish. The flag was correct exactly as computed; what it needed was a second, more specific mapping to reveal that the disclosure and the data were both right, describing two different populations.

Interpreting the Result

WHEN A DISCLOSED FIGURE DOES NOT RECONCILE

1. **Benign explanation.** A disclosed rate can genuinely describe a subset of the population, such as new additions rather than the full asset base, that a first, unfiltered mapping does not isolate, as in the scenario above.
2. **Error or judgment lapse.** A note carried forward unchanged from a prior period after the underlying policy or portfolio composition changed is a drafting oversight, not a deliberate misstatement, and is corrected by updating the note to match the current, real figure.
3. **What would indicate something more serious.** A disclosed figure that consistently understates a risk-relevant rate, such as a loss given default or an impairment rate, relative to what every reasonable mapping of the underlying data shows, across more than one reporting period, warrants direct inquiry into how that specific note is drafted and reviewed.

Chapter 26

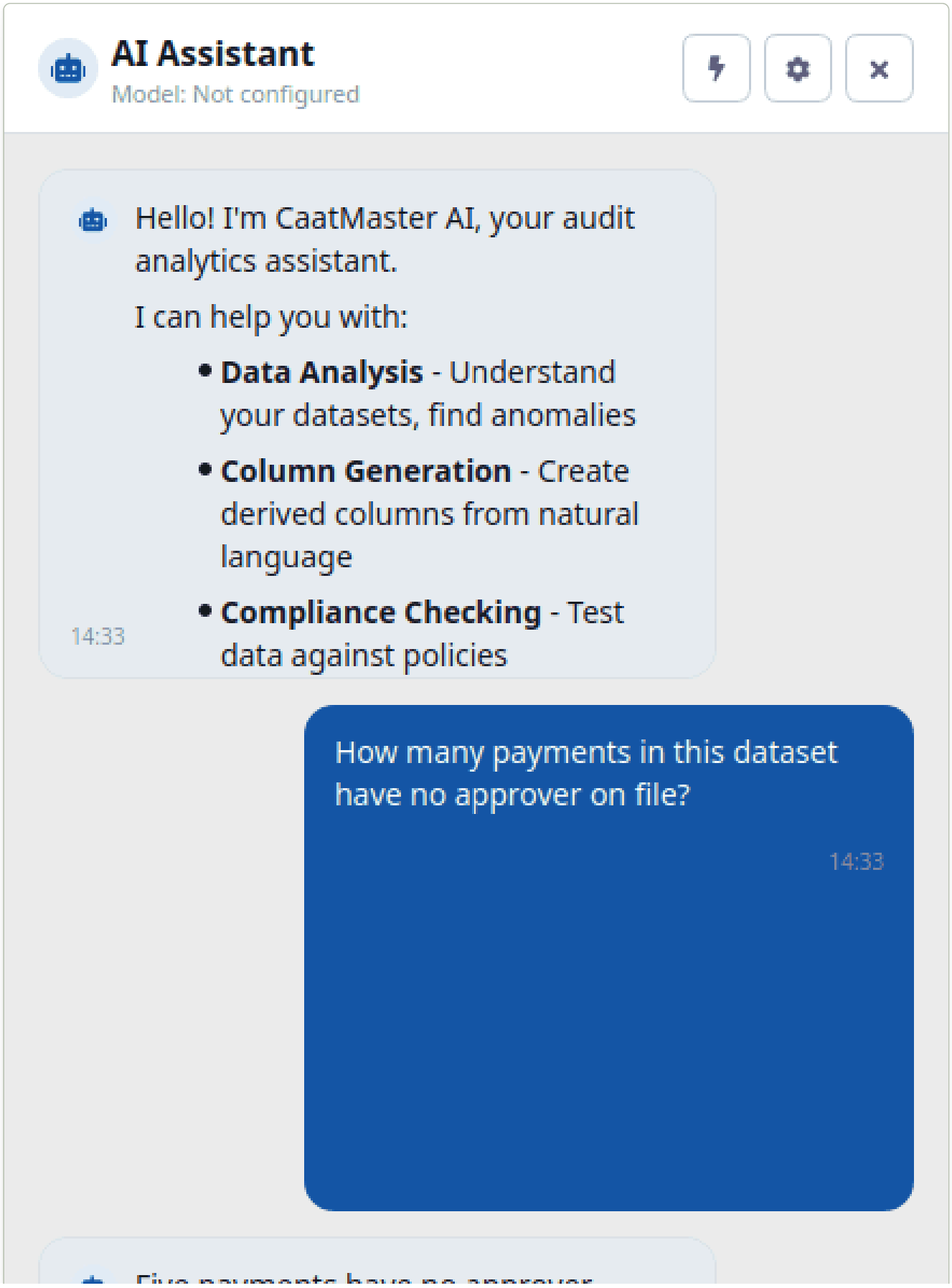
Conversational Analysis

The AI Assistant panel, docked to the right side of the main window and reached from the sidebar or the lightning-bolt icon in its own header, is the most open-ended AI-assisted feature in CaatMaster: a running conversation about whichever dataset is currently open, capable of answering a question directly, running a real query to answer one it cannot answer from memory, proposing a chart, or proposing a transformation step. What keeps this openness from becoming a liability is a narrow, strictly enforced boundary described in this chapter's second half: the panel can decide what ought to happen, but it never makes anything happen by itself.

A Conversation Grounded in Real Summary Statistics

Every question sent to the panel carries context ahead of it: a schema summary of the active dataset and a set of real, precomputed data-quality statistics, row and column counts, null percentages, distinct-value counts, generated the same way Chapter 5's Profile Dataset command generates its own summary, not asked of the model. A question the model can answer directly from that context, such as how many columns a dataset has or which column carries the most missing values, is answered immediately, from real numbers already in hand.

1. With the AP Transactions dataset open, click the AI Assistant icon in the sidebar.
2. Type a question, for example: `How many payments in this dataset have no approver on file?`
3. Press Enter.


AI Assistant
Model: Not configured
Hello! I'm CaatMaster AI, your audit analytics assistant.
I can help you with:
• Data Analysis - Understand your datasets, find anomalies
• Column Generation - Create derived columns from natural language
• Compliance Checking - Test data against policies
14:33
How many payments in this dataset have no approver on file?
14:33

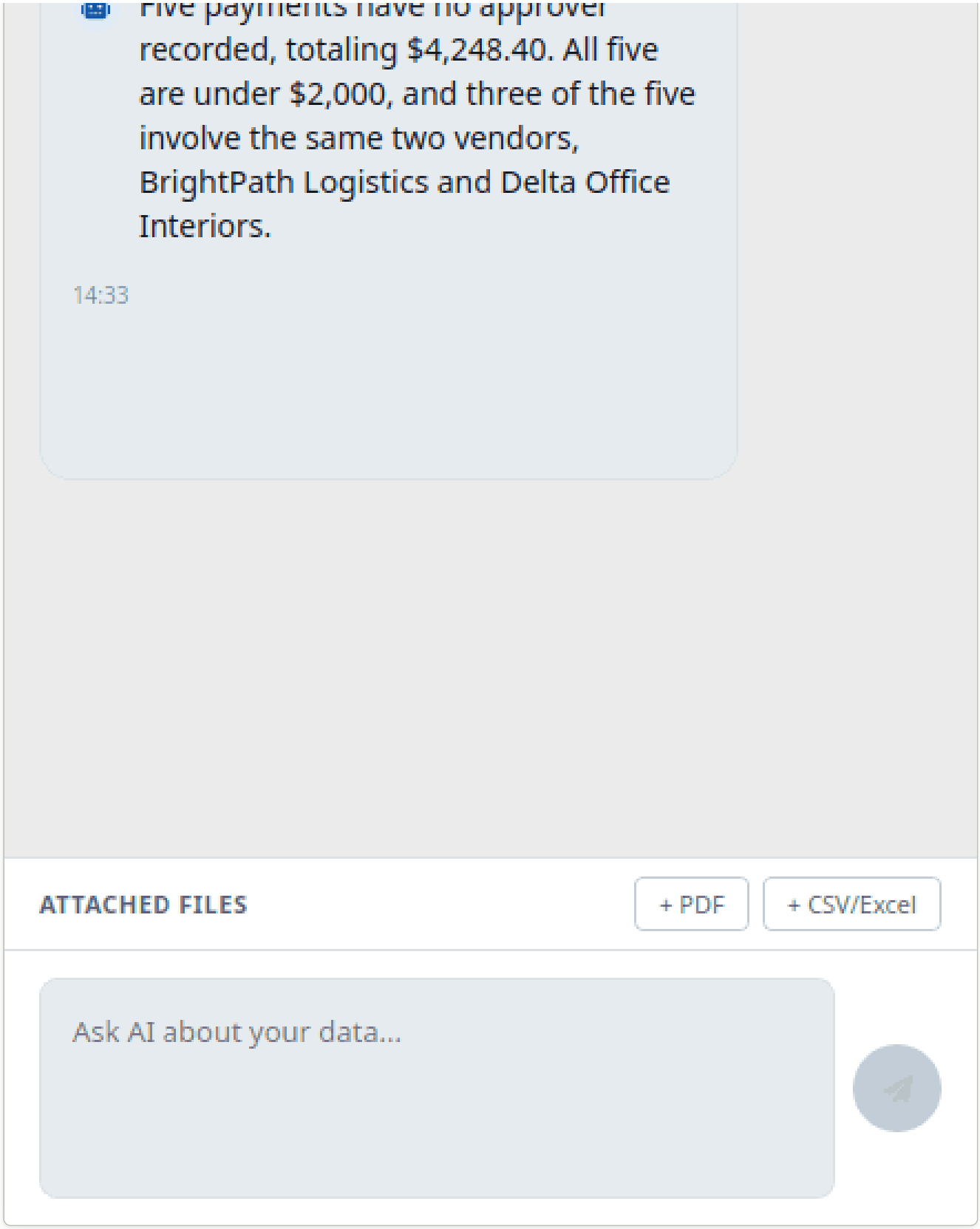


Figure 26.1 — A conversation in the AI Assistant panel. The answer shown here is illustrative of the kind of grounded response the panel is designed to give, phrased to match the real completeness finding from Chapter 6, rather than a transcript of a live model call.

When the Context Alone Is Not Enough

A question the precomputed context cannot answer, one that requires filtering, grouping, or joining the real data in a way no summary statistic captures, triggers a second step. The model does not guess; it returns a decision, structured as one of six types: answer directly, run a specific SQL query, apply a specific transformation operation, build a specific chart, create a new dataset from a query, or ask for help on a topic. Where the decision is a query or an operation, CaatMaster validates it for safety before anything runs, the same validation any other query or operation in the application passes through, and only a query or operation that passes is actually executed, against the real, complete data, with the real result then handed back to the model for a second, grounded response. A query or operation that fails validation is never silently retried or forced through; the model is told plainly that its request could not be run and asked to answer from the context already available instead, stating openly that a precise answer is not available if the context does not cover the question.

This two-step shape, decide first, then execute deterministically, then answer from the real result, is the same translate-then-compute discipline from Chapter 23 applied to a conversational interface rather than a single-purpose dialog. The panel is never the thing that runs a query, applies a step, or draws a chart; main_window.py is, and only after the panel's proposal has passed the same checks every other action in the application passes through.

Interpreting the Result

A response from the AI Assistant panel deserves the same question this entire part of the book has returned to repeatedly: did this number take the grounded path, structured, computed, and logged, or did it arrive as a plausible-sounding guess. A response that cites a specific count or amount, especially one attributable to a visible query the panel ran, carries the same weight as any other computed figure in this book. A response phrased in general terms, without a specific number behind it, should be read as exactly that, a starting point for the reviewer's own next question, not a finding to cite on its own.

PART VII

Results

Chapter 27

Visualizing Findings

A table of flagged rows answers a precise question well and a general one poorly. Whether a population's exceptions concentrate in a handful of vendors, or a metric has drifted outside its normal range over time, is usually easier to see in a chart than to infer from a column of numbers, and CaatMaster's Visualizations view offers fourteen chart types for exactly this purpose, reached from the top menu bar through Statistics → Visualize Dataset…, or by right-clicking any column header and choosing Visualize.

Most of the fourteen, column and bar charts, line charts, scatter plots, histograms, pie charts, box plots, heatmaps, area charts, violin plots, stacked column charts, and Q-Q plots, are general-purpose statistical graphics common to any data analysis tool. Two are worth a closer look because they are built specifically for audit work: the Pareto chart and the control chart.

The Pareto Chart: Finding the Vital Few

A Pareto chart sorts categories in descending order by total value and overlays a cumulative-percentage line, with a reference line at eighty percent, making it the standard tool for the vital-few observation that a small number of categories, vendors, error types, or accounts, typically accounts for the great majority of total value or exceptions in a population.

1. From the top menu bar, choose `Statistics → Visualize Dataset...` against the AP Transactions dataset.
2. Set Chart Type to `Pareto Chart`.
3. Set X Axis to `vendor` and Y Axis to `amount`.
4. Click `Create Chart`.

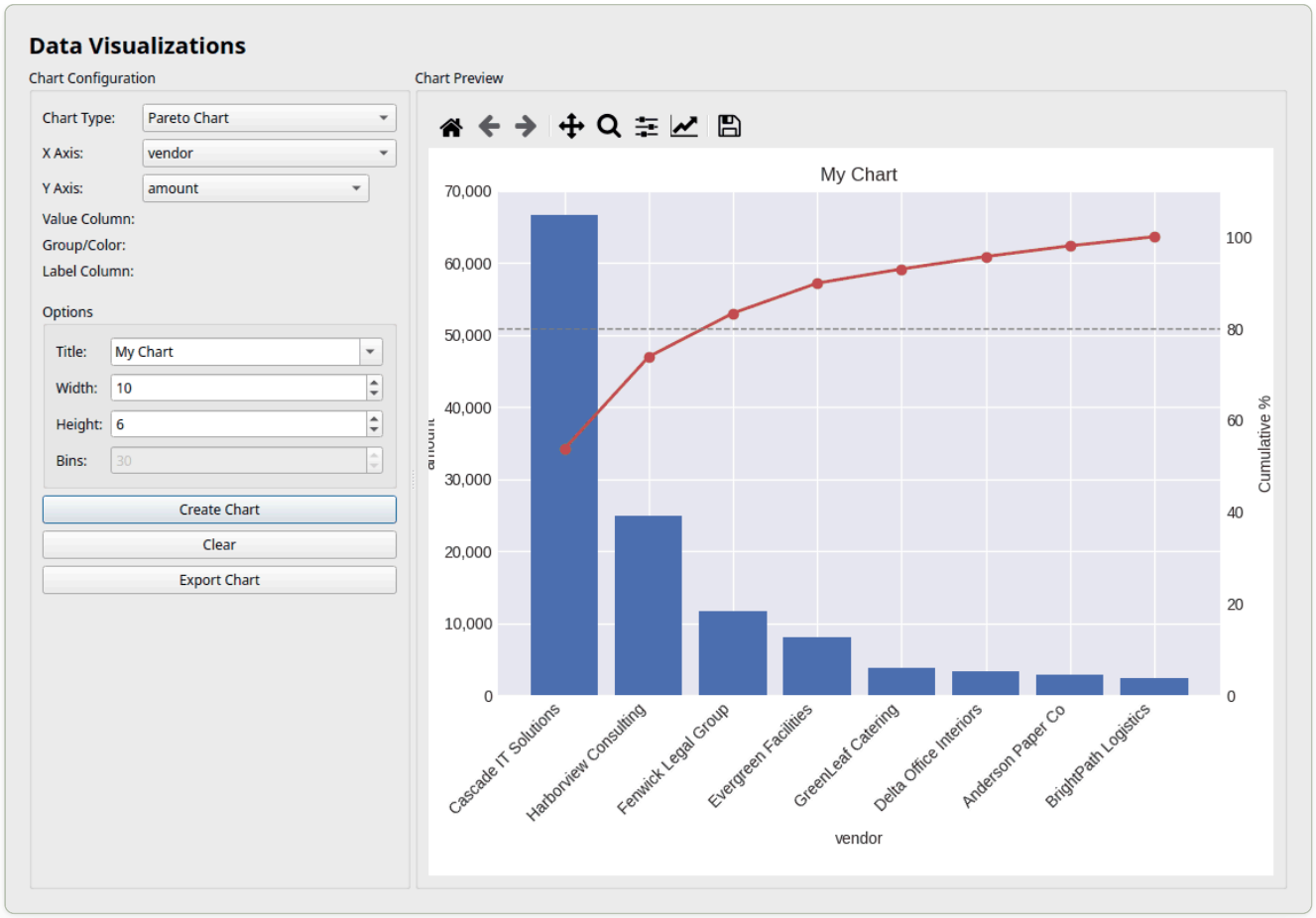


Figure 27.1 — A Pareto chart of ap_transactions.csv by vendor. Cascade IT Solutions, at $66,675.00 across five invoices, is the single largest vendor by a wide margin, driven chiefly by the $48,750.00 server rack invoice already identified as an outlier in Chapter 10 and selected with certainty by Chapter 11's monetary unit sample. A second Cascade IT Solutions invoice, $9,900.00, is the separate transaction flagged as near-threshold in Chapter 14 and as a milder outlier in Chapter 10; the two are different invoices from the same vendor, not the same invoice found twice.

The Control Chart: Watching a Metric Over Its Own History

A control chart, adapted from statistical process control, plots a metric in sequence against its own mean and a pair of three-standard-deviation control limits, flagging any point that falls outside them. This is a different lens from Chapter 15's CUSUM technique: a control chart flags an individual point that is extreme relative to the whole population's spread, evaluated independently at each point, while CUSUM accumulates smaller, sustained deviations over several consecutive points and can catch a shift too gradual for any single point to cross a three-sigma line on its own. The two techniques answer different questions and are not substitutes for one another.

1. Set Chart Type to `Control Chart`.
2. Set Value Column to `amount`, leaving X Axis blank so points plot in row sequence.
3. Click `Create Chart`.

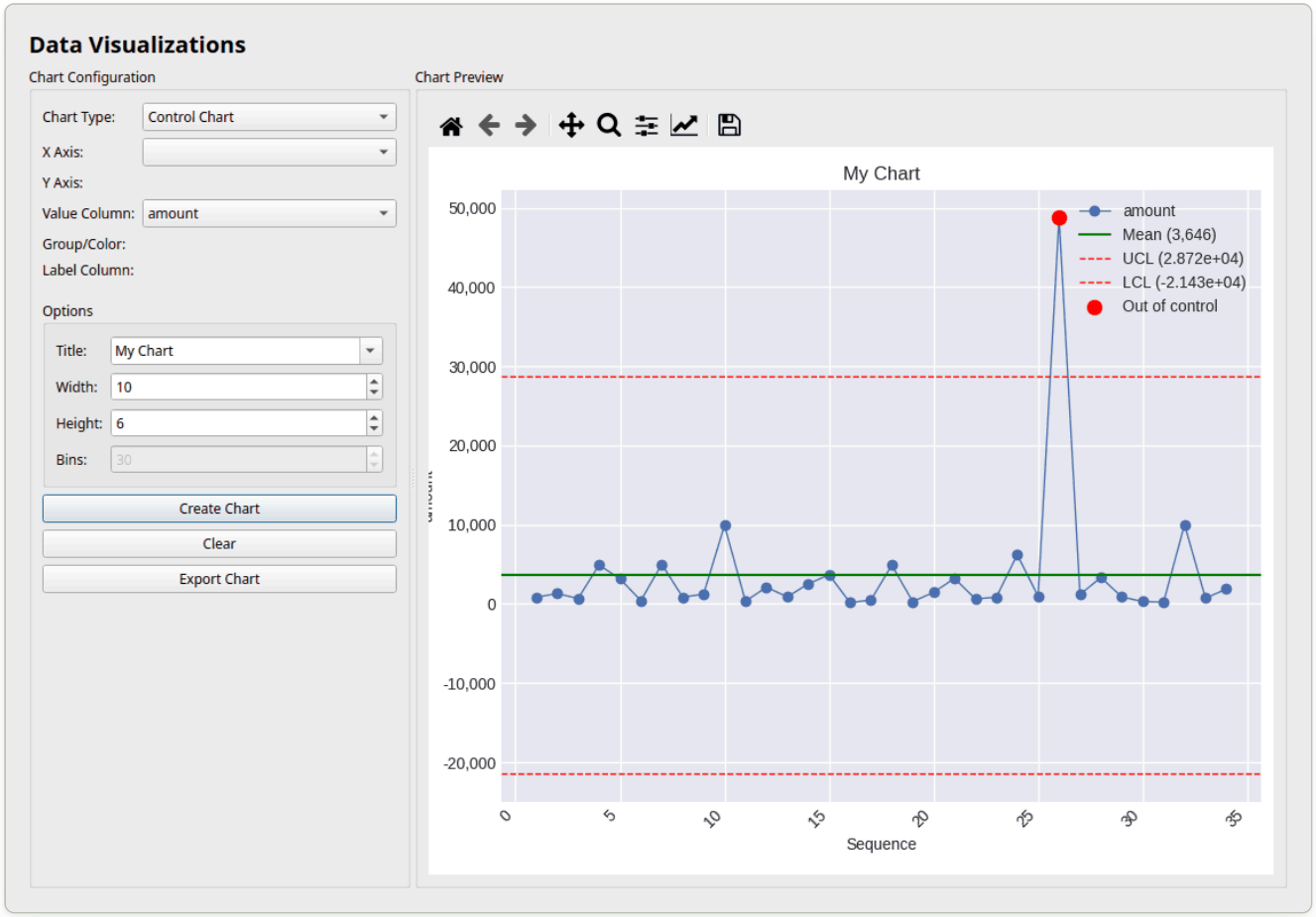


Figure 27.2 — A control chart of ap_transactions.csv's amount column. The mean, $3,646, matches Chapter 5's Field Statistics exactly. One point, the same $48,750.00 invoice, sits far above the upper control limit of $28,720.00 and is flagged. The lower control limit, a mathematically symmetric $−21,430.00, is a reminder that a plain, symmetric three-sigma limit assumes a roughly normal distribution; a population of transaction amounts, which can never go negative, is rarely well described by one, and the lower limit here is not a meaningful boundary in practice.

Charts Stay Connected to the Pipeline

A chart built from a dataset is not a static snapshot. Because every view in CaatMaster reads from the same Applied Steps pipeline described in Chapter 2, a chart built against a dataset updates automatically if an upstream step in that dataset's pipeline is later modified, added, or

removed. A Pareto chart built before an outlier flag is added, then revisited afterward, reflects the current state of the data, not the state at the moment the chart was first drawn.

Interpreting the Result

A chart is a way of seeing a population, not a finding in its own right. The same three-question discipline from Chapter 1 applies to whatever a chart makes visible: a Pareto chart's dominant vendor deserves the same benign-explanation-first reading as any other concentration finding, covered at length in Chapter 7, and a control chart's out-of-control point deserves the same reading as the outlier techniques covered in Chapter 10. What a chart adds is not a new category of evidence, but a faster route to noticing where the existing evidence concentrates.

Chapter 28

Building the Audit Report

Every finding in this book, up to this point, has lived inside CaatMaster's own views. A workpaper eventually has to leave the application as a document someone else can read without it open in front of them. The Reports view, reached from the top menu bar through Reports → Generate Audit Report…, assembles a self-contained HTML report from named sections, each pulling its content directly from the real, current state of a dataset, never from a value typed in by hand.

Five Kinds of Section

A report is a title, an author, and an ordered list of sections, each one of five types. A text section holds prose the reviewer writes directly, an executive summary or a narrative introduction. A summary section is generated automatically: row and column counts, memory size, total missing values, and a per-column breakdown of data type, null count, and distinct-value count, computed fresh from the dataset at the moment the report is built, the same figures Chapter 5's Profile Dataset command reports. A chart section embeds a real chart, rendered from eight of the fourteen types covered in Chapter 27, bar, line, scatter, histogram, pie, box, heatmap, and area, as a genuine image, encoded directly into the report's HTML rather than linked externally, so the report remains a single, self-contained file. A table section embeds a slice of the dataset itself. A transformation steps section lists the Applied Steps pipeline that produced the dataset, numbered in order, the same record described in Chapter 2, giving a reader of the report a visible trail back to how every figure in it was actually produced.

Metric	Value
Total Rows	34
Total Columns	10
Missing Values	5

Figure 28.1 — The real Population Overview section generated for ap_transactions.csv, matching Chapters 5 and 6 exactly.

Exporting the Result

Once assembled, a report exports through Reports → Export as a Word document, an Excel workbook, or a PDF, each conversion preserving the section structure built in the Reports view. A report built this way can be regenerated at the start of the next reporting period against a refreshed extract, exactly as Chapter 2 described for the Applied Steps pipeline underlying it, since the report itself contains no figures that were not pulled fresh from the dataset at build time.

Interpreting the Result

A report assembled this way carries the same guarantee every other output in this book carries: every number in it traces back to the Applied Steps pipeline the transformation steps section makes visible. A reviewer reading someone else's report and wanting to confirm a figure does not have to take the author's word for it; the same dataset, replayed through the same listed steps, reproduces the same report.

Chapter 29

Reusable Audit Programs

An engagement team rarely runs a procedure exactly once. The same segregation of duties scan, the same three-way match, the same Benford's Law test against a general ledger extract, recurs every reporting period against a freshly extracted population. An Audit Program captures a dataset's current Applied Steps pipeline, described throughout this book since Chapter 2, as a named, reusable template that can be applied to any other dataset later, reached from the sidebar under Analyze as Audit Programs.

Saving and Applying a Program

A program is saved directly from a dataset already carrying the steps worth reusing: build the pipeline once, by hand, exactly as every technique chapter in this book has shown, then save it as a program rather than discarding it once the immediate review is done.

1. With a dataset carrying the Applied Steps to reuse, open `Audit Programs` from the sidebar.
2. Click `Save Current Steps As New Program...`, and give it a name and a description.
3. To reuse it later, open Audit Programs against a new dataset, select the saved program from the list, and click `Apply to Current Dataset`.

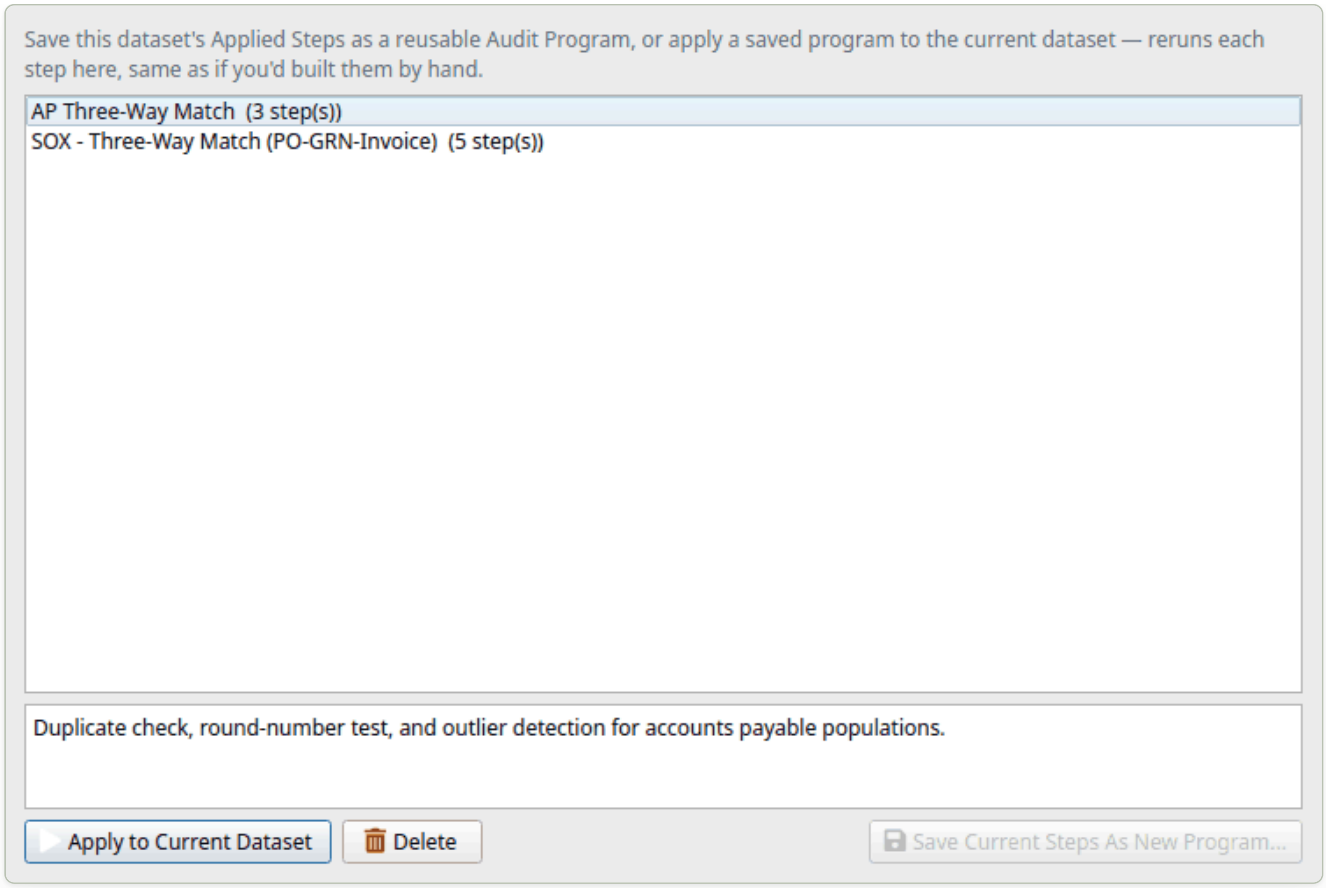


Figure 29.1 — The Audit Programs dialog. SOX — Three-Way Match (PO-GRN-Invoice) ships with CaatMaster itself: a five-step program joining a Purchase Order, Goods Received Note, and Invoice dataset and flagging any invoice whose quantity or price exceeds what was ordered or received, built entirely from the ordinary Join Datasets and Custom Column operations covered in Chapter 2, with no operation specific to this one program.

Applying a program reruns each of its steps against the new dataset in order, exactly as if they had been built by hand again, which means a program built against one dataset's column names generally needs those steps reviewed, and any column reference corrected, before it produces a meaningful result against a dataset whose columns are named differently. The shipped three-way match program is explicit about this in its own description, instructing the reviewer to repoint each step's dataset and column references to the actual PO, GRN, and invoice datasets at hand before running it, rather than assuming any two companies' procurement extracts share identical column names.

Interpreting the Result

An Audit Program is a labor-saving device, not a substitute for understanding what its steps do. A finding produced by a saved program deserves exactly the same interpretation discipline as the same finding built by hand, benign explanation first, then error, then the pattern that would

indicate something more serious, covered at length in each technique's own chapter. What a program changes is how quickly the steps that produce the finding can be reassembled, not what the finding itself means once produced.

Chapter 30

Managing Projects and Sessions

Every technique in this book has operated on one dataset at a time, opened inside a project, the container introduced in Chapter 2 that holds every dataset, transformation, and saved analysis belonging to one engagement. This closing chapter of Part VII covers the project itself: how it is saved, exported for sharing outside CaatMaster, and brought back in again with nothing lost.

What a Project Actually Holds

A project carries a name, an optional client name and engagement type, a set of named parameters, covered briefly in Chapter 2, that a step can reference instead of a hardcoded value, and a dictionary of datasets, each one an original dataframe together with its Applied Steps pipeline. Saving a project through File → Save Project persists all of this in one file; nothing about a dataset's transformation history is lost between sessions, since the pipeline itself, not just its current output, is what gets saved.

Exporting a Project Outside CaatMaster

Export for Review, covered briefly in Chapter 2 and reached from the sidebar under Lineage & Export, packages a project for a second reviewer who may not have CaatMaster open at all. Three formats are available: Excel workbooks with embedded metadata, one file per dataset; Parquet, a compact columnar format that preserves every data type exactly; or JSON. Exported this way, a two-dataset project built from ap_transactions.csv and quarterly_financials.csv, thirty-eight rows combined, produces a single zip archive under seven kilobytes, holding one Parquet file and one metadata JSON file per dataset alongside a project-level manifest.

Importing that same archive back into CaatMaster reconstructs the project exactly: both datasets return with their original row counts, thirty-four and four, and their names intact, a real round trip rather than a one-way export. This is not a coincidental convenience; a project export that could not be reliably reimported would defeat the purpose of using it to hand work to a second reviewer, who needs to open the same project CaatMaster itself would recognize, not merely inspect a static copy of the numbers.

Where a Session Picks Back Up

The Dashboard view, the first thing a reopened project shows, summarizes recent activity across every dataset in the project: how many datasets are open, their combined row counts, and a running feed of recent actions. It is a starting point for orienting a reviewer returning to a project after time away, not an analytical view in its own right; every technique described earlier in this book is reached from the Transform view or one of the dialogs this book has walked through, not from the Dashboard itself.

Interpreting the Result

This chapter, like Chapter 2 before it, describes structure rather than produces a finding. What is worth carrying forward from it is the same question that opened this book: can the number in front of the reviewer be traced back to the file it came from. A project saved, exported, and reimported the way this chapter describes answers that question the same way a single dataset's Applied Steps pipeline does, at the scale of an entire engagement rather than one table.

PART VIII

Judgment

Chapter 31

Every Flag Is a Question

Thirty chapters of technique have each closed with the same three questions: what benign explanation would produce this pattern, what error or lapse in judgment would produce it, and what would have to be true for it to reflect something deliberate. That repetition was not an editorial habit. It was the whole point of this book, stated once in Chapter 1 and then rehearsed, deliberately, against every kind of finding a modern audit encounters, a statistical outlier, a control weakness, an accounting estimate, a language model's own output, so that the discipline survives past the chapter that first taught it. This closing chapter of technique does not introduce a new test. It asks what the thirty before it add up to.

A Flag Is Evidence of a Pattern, Not of a Cause

Every technique in this book, from Benford's Law in Chapter 8 to the reverse-DCF impairment solve in Chapter 18, computes something true about a population and stops there. A deviation, a variance, an implied growth rate, an out-of-control point, is a fact about the data, arrived at deterministically and reproducibly, exactly as Chapter 4 promised. None of these facts carries its own explanation. The explanation is supplied afterward, by a person who knows things the data alone cannot state: what this vendor relationship actually involves, why this estimate changed, whether this employee's access really was reviewed before the system says it was removed. A tool that computed causes as confidently as it computes deviations would be claiming knowledge it does not have, which is precisely the discipline Chapter 4 asked of CaatMaster's own AI-assisted features and which this chapter now asks of the reviewer using any of its deterministic ones.

What Convergence Actually Looks Like

This book has followed one small working dataset, ap_transactions.csv, through eleven different chapters, and two of its thirty-four rows surfaced repeatedly, for two genuinely different reasons that are worth telling apart rather than blurring together.

The first is a matter of simple completeness. Five invoices carry no approver on file. Chapter 6's no-nulls check found them by testing one column for missing values. Chapter 14's Payment

Without Approval on File rule found the identical five records, by name, with a citation and a severity, translating the same underlying fact into compliance language. Two techniques, built for different purposes and reached from different menus, landed on exactly the same five rows because both were, underneath their different vocabulary, asking the same question of the same data.

The second is a matter of magnitude, and it involves two different invoices from the same vendor, not one invoice found repeatedly. Cascade IT Solutions' $48,750.00 server rack invoice was flagged as a statistical outlier in Chapter 10, selected with certainty under monetary unit sampling in Chapter 11 for the same reason, sheer size, that made it an outlier, and stood out as the single out-of-control point on a control chart in Chapter 27, three independent statistical mechanisms converging on one transaction because each one, in its own way, measures distance from the ordinary. A separate invoice from the same vendor, $9,900.00, drew a milder outlier flag in Chapter 10 and, independently, a named rule in Chapter 14 for sitting just under a common ten-thousand-dollar approval threshold, a different kind of signal, positional rather than purely statistical, that happened to involve the same vendor without being the same finding.

Genuine convergence, several independent techniques landing on the same fact through different mechanisms, is a stronger basis for inquiry than any one technique alone, and is worth recognizing precisely because it is different from noticing the same vendor's name twice. A reviewer who treated every appearance of Cascade IT Solutions across this book as one escalating case, rather than two separate, specific findings, would be manufacturing a pattern the data does not actually support.

The Discipline Does Not Change for an AI-Assisted Finding

Part VI drew a hard line between a number that came from Polars and a number that came from a language model, and the whole architecture of CaatMaster's AI-assisted features, translate then compute, log every call, batch with an honest fallback, exists to keep that line real rather than nominal. That architecture answers whether a number can be trusted the way any other computed figure can be trusted. It does not answer what the number means, which is the same three-question discipline again, applied without exception to a classification from Chapter 24, a reconciled disclosure from Chapter 25, or an answer from the conversational panel in Chapter 26, exactly as it applies to a Benford deviation or a control chart point. An AI-assisted finding that skipped this discipline because it felt more like a direct answer than a statistical artifact

would be the one place in this book where the habit most needs to hold, precisely because it is the easiest place to let it slip.

The Auditor's Skepticism, Not the Software's

Professional skepticism is sometimes described as an attitude, a general wariness applied at the start of an engagement and then set aside once the testing begins. This book has tried to show something narrower and more durable: skepticism as a specific, repeatable operation, applied individually to each finding at the moment it appears, in the same three-step order every time. A tool can supply the finding. It cannot supply the skepticism. CaatMaster surfaces a pattern precisely, reproducibly, and without fabricating a number it does not have; everything this book has demonstrated about the software depends on that precision. What the software cannot do, and was never built to do, is decide what a precisely computed pattern means for the specific people and the specific transactions behind it. That decision belongs to the reviewer, every time, and the three questions this book has repeated in chapter after chapter are the discipline for making it well.

Chapter 32

Governance, Security, and Limits

A tool that holds client financial data, authentication credentials, and third-party AI provider keys on the same machine an auditor carries between engagements creates obligations that have nothing to do with which statistical test runs correctly. This closing chapter gathers the security architecture described piecemeal across earlier chapters into one place, for whoever at a firm is responsible for approving CaatMaster's use, and closes with a direct, unhedged statement of what this software does not do.

What Is Protected, and How

Three categories of secret live on a machine running CaatMaster, and all three follow the same pattern: nothing sensitive is ever committed to source control, and nothing sensitive is ever stored in plain text on disk. The application's own authentication database is created fresh on first run in a per-user data directory, never inside the project's own folder, specifically so it cannot be accidentally added to a version-controlled repository. The signing secret that database depends on is generated once per installation and stored separately, at file permissions that restrict it to the owning user. Every AI provider API key entered through the AI Configuration dialog covered in Chapter 23 is encrypted at rest with Fernet, an authenticated symmetric scheme, keyed off that same per-installation secret rather than a fixed, shared value baked into the application itself. A firm evaluating CaatMaster for a shared or managed machine should confirm this secret is generated per installation, not copied between machines as part of a deployment image, since copying it would let one machine decrypt another's stored credentials.

What Leaves the Machine, and What Does Not

Two features in this book make an outbound network connection under specific, documented conditions, and neither does so silently. An AI-assisted feature, covered throughout Part VI, sends a request to whichever provider is configured only when that specific feature is invoked, never in the background, and every such call, prompt, response, provider, and timestamp, is written to the audit log described in Chapter 23, reviewable afterward by someone other than whoever ran it. An XBRL or inline XBRL import, covered in Chapter 22, attempts a network fetch only when a filing's taxonomy cannot be resolved from a locally registered taxonomy

package first, and a firm operating on an air-gapped or restricted network can eliminate that outbound attempt entirely by registering the taxonomy packages it needs in advance, exactly as Chapter 22 demonstrated. Every other technique in this book, every IFRS and IAS calculation in Part IV, every classic CAAT in Part II, operates entirely on data already loaded into the application, with no network dependency of any kind.

What a Firm's Own Policy Still Has to Cover

CaatMaster protects its own credentials and gives a firm the tools to control its own network exposure, but it does not implement several controls a firm's broader policy still needs to supply. It has no built-in, per-user permission model within a single installation; anyone with access to the machine and its project files has access to whatever those files contain, which makes disk-level encryption and physical device control a firm's responsibility, not the application's. Project files, exported audit programs, and the AI audit log are all held as plain files on the local disk, covered in Chapter 30 and Chapter 29, with no built-in synchronization to a firm's own document management or backup systems; a firm intending to rely on CaatMaster output as part of its permanent audit file needs its own process for getting those files into that system. Nothing in the application logs which specific person, as distinct from which machine, ran a given analysis; a firm operating in a regulatory environment that requires per-user attribution of a working paper needs to supply that attribution through its own file-naming or check-in discipline.

What This Book Has Not Claimed

Several limits have been stated plainly at the point they mattered, rather than saved for this chapter, and are worth restating together rather than left scattered. CaatMaster has no PDF financial-statement import; the pypdf library it uses, noted in Chapter 25, reads narrative text for AI-assisted analysis, not structured figures for ratio calculation. Every threshold cited in Part IV, IFRS 16's fifty basis points, IAS 36's two percentage points, IAS 37's thirty percent, IAS 12's twenty percent, IAS 21's one percent, is a documented starting point for follow-up audit judgment, not a standard-defined bright line, and several of these chapters said so explicitly rather than let a configurable default be mistaken for a professional requirement. Two split variants, catalogued in Appendix A, exist in the underlying engine with no path to reach them from the interface at all; a set of six trigonometric operations that shared that same fate was removed from the application entirely once no legitimate audit use for them turned up, rather than left in place unreachable. Every operation in the top menu bar's ML menu Anomaly

Detection submenu, covered in Chapter 10, now runs the genuine algorithm its name references; what still distinguishes those quick, single-column entries from the trainable models covered in that same chapter is scope, one column at a time versus several at once, not authenticity.

None of these is a defect uncovered late. Each was found, verified, and written down in the chapter where it belonged, in keeping with the discipline this book asked of every finding CaatMaster itself produces: state what is actually true, not what would be more convenient to claim.

PART IX

Direct Data Access

Chapter 33

The SQL Query Workflow

Every technique covered so far in this book is reached as a discrete operation: a dialog collects a handful of parameters, and a single, named transformation is appended to the Applied Steps pipeline. That model works well for a test that can be stated as a filter or a per-row calculation, but some audit questions are naturally relational instead — comparing a table against itself, joining two datasets on a shared key, or ranking each row against others in its own group. CaatMaster's SQL Query workspace exists for exactly that class of question: a full SQL editor, backed by DuckDB, that queries one dataset or several at once without leaving the application or exporting anything to an external tool.

Reaching the SQL Workspace

The sidebar carries a segmented Data / SQL Query switch near its top, above the page navigation list. Clicking SQL Query replaces the Data workspace with the SQL editor; clicking Data switches back. This is a workspace mode, not a page in the usual sidebar list — it sits alongside Data rather than being reached from the same navigation list as Transform, Visualizations, or Reports.

How It Works Internally

The SQL workspace keeps its own in-memory DuckDB connection, entirely separate from the SQLite database that stores user accounts. Every dataset in the current project that has been checked in the Model Query list, described below, is registered as a DuckDB table under a sanitized version of its display name: spaces and punctuation become underscores, and a name that doesn't already start with a letter or underscore gets one prefixed, so a dataset named AP Transactions becomes queryable as `AP_Transactions`. Registration happens against the dataset's *current* data — whatever the Applied Steps pipeline currently produces, refreshed automatically whenever that pipeline changes, a dataset is renamed, or a new dataset is added, so a query never runs against data that CaatMaster has already moved past showing you.

Model Query: Joining Across Datasets

Above the query editor, a checklist labeled Model Query lists every dataset in the project; every checked one becomes a table you can reference by name in the query below. This is what makes a cross-dataset join possible without any separate Table → Join operation: check two datasets, and a query like `SELECT g.*, s.vendor FROM GL_Entries g JOIN Subledger s ON g.gl_id = s.gl_id` runs directly against both of them, exactly as it would against two tables in any relational database.

The Toolbar

Six controls sit above the query editor. Execute runs the current text against the registered tables and displays the result below, up to the first thousand rows. Format SQL uppercases keywords and puts one major clause per line — useful on a query pasted from elsewhere or drafted by AI, described further below. Clear empties the editor. A Template dropdown loads one of seven skeleton queries (a plain `SELECT`, a filtered `SELECT`, a `GROUP BY`, a join, a window-function rank, and two statistical patterns) as a starting point to edit rather than a runnable query as-is, since they reference a placeholder table named `data` rather than any specific dataset.

Two Ways to Keep a Result: Apply to Dataset vs. Create New Dataset

Running a query only displays its result; nothing is kept until one of two remaining toolbar buttons is used, and the choice between them matters. Apply to Dataset records the query as a genuine step on the currently active dataset's Applied Steps pipeline — it shows up in the audit trail described in Chapter 2, and it replays automatically the next time that dataset's source file refreshes. This only makes sense for a query against a single table, the active dataset itself, since the pipeline step re-runs the query against a fresh single-table connection each time. Create New Dataset instead materializes the query's full result set as a brand new dataset in the project, the right choice for a Model Query join or union whose result doesn't semantically belong to any one source dataset's own pipeline. The new dataset records which of the registered tables its defining query actually referenced, so its lineage remains visible from the Dependencies view described in Chapter 30, even though it isn't a literal pipeline step on any single source.

Reusable Query Parameters

A query can reference `{{parameter_name}}` placeholders, resolved at run time against a set of named values managed at the project level through Query Parameters.... A materiality

threshold, a reporting-period start and end date, or any other value a recorded audit program needs to change between engagements can be defined once as a parameter and referenced by name in as many SQL steps as needed; updating the parameter's value updates every step that references it, without editing the query text itself. This is what lets a SQL-based test recorded this period become a genuinely reusable audit program next period, in the sense Chapter 29 describes for the pipeline as a whole — only the parameter values change, not the query.

Worked Example: Suspected Duplicate Payments via Self-Join

Chapter 5's profile of the AP Transactions dataset reported zero duplicate rows, and that's correct as far as it goes — no two rows in the file are identical across every column. But a duplicate *payment* doesn't require identical rows; it requires the same vendor paid the same amount more than once, typically under a different invoice number and a different date. That's a self-join, not a row-equality check, and it's exactly the kind of question the Applied Steps pipeline has no single operation for.

```
SELECT a.invoice_id AS invoice_a, b.invoice_id AS invoice_b,
       a.vendor, a.amount, a.invoice_date AS date_a, b.invoice_date AS date_b
FROM AP_Transactions a
JOIN AP_Transactions b
  ON a.vendor = b.vendor
 AND a.amount = b.amount
 AND a.invoice_id < b.invoice_id
ORDER BY a.vendor, a.amount
```

The `a.invoice_id < b.invoice_id` condition is what keeps a pair from being reported twice in both directions, and keeps a row from matching itself. Run against AP Transactions, this returns exactly three rows:

Invoice A	Invoice B	Vendor	Amount	Date A	Date B
20001	20008	Anderson Paper Co	842.50	2025-01-03	2025-01-13
20001	20024	Anderson Paper Co	842.50	2025-01-03	2025-02-05
20008	20024	Anderson Paper Co	842.50	2025-01-13	2025-02-05

Figure 33.1 — All three pairings of the same underlying pattern: three separate Anderson Paper Co invoices, 20001, 20008, and 20024, each for

exactly 842.50, each described as “Copy paper and toner restock,” spaced roughly ten days to three weeks apart across January and February.

Interpreting the Result

WHEN A SELF-JOIN SURFACES A REPEATED PAYMENT

1. **Benign explanation.** A recurring, fixed-cost restocking order, copy paper and toner at a stable unit price, placed on a routine cadence, produces exactly this pattern: same vendor, same amount, same description, different dates. Many organizations genuinely do reorder standard supplies at a fixed price on a schedule.
2. **Error or judgment lapse.** The same invoice submitted, approved, and paid twice under two different invoice numbers, whether through a vendor billing error or a duplicate data entry on receipt, produces an identical pattern and is indistinguishable from the benign case by amount and vendor alone.
3. **What would indicate something more serious.** A repeated round-amount payment to a vendor without a corresponding pattern of genuine recurring deliveries, particularly one clustered just under an approval threshold, is a classic indicator explored further in Chapter 12’s segregation-of-duties material, and warrants confirming the underlying delivery or service actually recurred three separate times rather than assuming it did.

Resolving which explanation applies here needs one more fact this query alone can’t supply: whether three physical shipments of paper and toner genuinely arrived, once every few weeks, which the vendor’s own delivery records would confirm or refute in a single lookup.

Worked Example: Ranking a Row Against Its Own Group

Chapter 10’s interquartile range test flagged invoice 20027, Cascade IT Solutions’ $48,750.00 server rack cooling fan, as more than forty-one interquartile ranges beyond the upper fence of the *entire* AP Transactions population, and Chapter 11’s Monetary Unit Sample absorbed three of its eight selection points on that same invoice regardless of random seed. Both tests compare

this invoice to every other invoice in the file, across every department. A window function can ask a narrower, complementary question instead: how unusual is this invoice next to only the *other* IT invoices it competes with for the same budget.

```
SELECT invoice_id, vendor, amount,
       ROUND(AVG(amount) OVER (), 2) AS dept_avg,
       ROUND(amount / AVG(amount) OVER (), 2) AS x_dept_avg
FROM AP_Transactions
WHERE department = 'IT' AND status = 'Approved'
ORDER BY x_dept_avg DESC
```

`AVG(amount) OVER ()`, with an empty `OVER` clause, computes the average across every row the surrounding `WHERE` has already narrowed to IT's five approved invoices, and repeats that same average on every output row — which is exactly what dividing each row's own amount by it requires. A version of this query scoped to the whole table, replacing the `WHERE` clause with `PARTITION BY department` inside each `OVER (…)`, would compute a separate average per department in one pass instead of one department at a time.

Invoice	Vendor	Amount	IT Dept. Avg	× Dept. Avg
20027	**Cascade IT Solutions**	**48,750.00**	**13,335.00**	**3.66**
20010	Cascade IT Solutions	9,900.00	13,335.00	0.74
20019	Cascade IT Solutions	4,875.00	13,335.00	0.37
20036	Cascade IT Solutions	1,860.00	13,335.00	0.14
20002	Cascade IT Solutions	1,290.00	13,335.00	0.10

Figure 33.2 — The same server rack invoice this book has now flagged three separate ways: by interquartile range against the whole population in Chapter 10, by selection-point absorption under Monetary Unit Sampling in Chapter 11, and here, at more than three-and-a-half times its own department's average approved invoice.

Ask AI: Drafting a Query in Plain Language

The Ask AI button opens a dialog that accepts a plain-language description of the query wanted, such as "show me every vendor whose total approved spend exceeds ten thousand dollars," and returns generated SQL text for review. This follows the grounded AI pattern described in Chapter 23 precisely: the dialog never executes anything itself, only writes the proposed SQL

into the editor on Insert, and the query still has to be reviewed and run with the ordinary Execute button, exactly as if it had been typed by hand. Before a generated query ever reaches the editor, it is independently checked to confirm it is a single, read-only `SELECT` (or `WITH … SELECT`) statement referencing only the tables already registered — a check performed in code, not merely requested in the prompt, on the same principle Chapter 23 applies everywhere else the AI proposes something a human has to approve.

> WHY THIS MATTERS FOR REVIEW
>
> A SQL step recorded through Apply to Dataset is exactly as visible in the Applied Steps audit trail as a step created by any ribbon operation — a second reviewer sees the literal query text, not a paraphrase of it, and can re-run it against the same or updated data. The flexibility SQL adds over the built-in operations doesn't come at the cost of traceability; it uses the same pipeline every other technique in this book already relies on.

Appendix A

Operation Reference

This appendix lists every operation registered in CaatMaster's transformation engine as of this writing: one hundred forty-four in total, organized into the thirteen categories the application itself uses. For each operation, the table gives its parameters, exactly as the underlying registry defines them, and a one-line statement of what it computes. Where an operation is not reachable from the interface at all, that fact is stated plainly rather than omitted. This appendix is generated directly from the same registry the application reads at runtime, so it reflects what the code actually contains rather than what a manual might claim it contains.

Text · 18 operations

Reached by right-clicking any column header and choosing Text, or, for the operations also listed in the top menu bar's Transform → Text submenu, from there directly.

Operation	Parameters	What It Computes
Add Prefix	Column (required); Prefix Text (required)	Adds a prefix to the beginning of text
Add Suffix	Column (required); Suffix Text (required)	Adds a suffix to the end of text
Extract First N Characters	Column (required); Number of Characters (default: 1)	Extracts the first N characters from text (LEFT-style)
Extract Last N Characters	Column (required); Number of Characters (default: 1)	Extracts the last N characters from text (RIGHT-style)
Extract Middle Characters	Column (required); Start Position (1-indexed) (default: 1); Number of Characters (default: 1)	Extracts characters from the middle of text (MID-style)
Extract Numbers	Column (required)	Extracts numbers from text
Extract Text	Column (required)	Extracts alphabetic text from string
Find and Replace	Column (required); Find (required); Replace With; Replace With Null (default: False); Case Sensitive (default: True); Use Regular Expression (default: False); Whole Word Only (default: False)	Finds and replaces values in a column
Hash (MD5)	Column (required)	Creates an MD5 hash of the text
Lowercase	Column (required)	Converts all text to lowercase
Reverse Text	Column (required)	Reverses the text string

Operation	Parameters	What It Computes
Soundex Code	Column (required)	Creates a Soundex code for phonetic matching
Split Alphanumeric	Column (required); Text Output Column (default: {column}_text); Number Output Column (default: {column}_number)	Separates alphanumeric values into text and numeric components
Standardize Missing Values	Column (required); Markers to Treat as Missing (required); Case Sensitive (default: False)	Replaces ad-hoc missing-value markers (e.g. "???", "-", "N/A") with a true null, in place
Text Length	Column (required)	Returns the length of the text
Title Case	Column (required)	Converts text to title case
Trim Whitespace	Column (required)	Removes leading and trailing whitespace
Uppercase	Column (required)	Converts all text to uppercase

Math · 12 operations

Reached by right-clicking any column header and choosing Math, or from the top menu bar's Transform → Numbers submenu for the operations listed there.

Operation	Parameters	What It Computes
Absolute Value	Column (required)	Returns the absolute value
Ceiling	Column (required)	Rounds up to the nearest integer
Custom Column from Math	Expression (required); Output Column (default: custom_result); Columns to Use (required)	Creates a custom column using mathematical operations
Exponential	Column (required)	Calculates e raised to the power
Floor	Column (required)	Rounds down to the nearest integer
Logarithm (base 10)	Column (required)	Calculates the base-10 logarithm
Logarithm (ln)	Column (required)	Calculates the natural logarithm
Modulo (Remainder)	Column (required); Divisor (required)	Calculates the remainder after division
Raise to Power	Column (required); Exponent (required)	Raises the value to a specified power
Round	Column (required); Decimals (default: 2)	Rounds numbers to specified decimals
Square	Column (required)	Squares the number
Square Root	Column (required)	Calculates the square root

Date & Time · 13 operations

Reached by right-clicking any column header and choosing Date & Time, or from the top menu bar's Transform → Date & Time submenu for the operations listed there.

Operation	Parameters	What It Computes
Age (Days)	Date Column (required)	Calculates age in days from today
Age (Years)	Date Column (required)	Calculates age in years from today
Days in Month	Date Column (required)	Returns the number of days in the month
Extract Day	Date Column (required)	Extracts the day of month from a date
Extract Hour	Datetime Column (required)	Extracts the hour from a datetime
Extract Minute	Datetime Column (required)	Extracts the minute from a datetime
Extract Month	Date Column (required)	Extracts the month (1-12) from a date
Extract Quarter	Date Column (required)	Extracts the quarter (1-4) from a date
Extract Second	Datetime Column (required)	Extracts the second from a datetime
Extract Weekday	Date Column (required)	Extracts the day of week (0=Monday, 6=Sunday)
Extract Year	Date Column (required)	Extracts the year from a date
Is Leap Year	Date Column (required)	Returns True if year is a leap year
Is Weekend	Date Column (required)	Returns True if date is Saturday or Sunday

Column · 7 operations

Reached by right-clicking any column header and choosing Column, or from the top menu bar's Transform → Column Operations submenu for the operations listed there.

Operation	Parameters	What It Computes
Calculated Columns	Formula (required); Output Column (required); Formula Type (default: arithmetic)	Creates new columns using formulas or expressions
Change Data Type	Column (required); New Type (required); Format (for dates); On Invalid Values (default: coerce)	Converts column data types to ensure compatibility with operations
Choose Columns	Action (required); Columns to Keep; Columns to Remove; New Column Order	Selects, removes, and reorders columns in the dataset
Index Column	Column Name (default: index); Start Value (default: 0)	Adds a sequential row-number column
Rename Column	Column (required); New Name (required)	Renames a column

Operation	Parameters	What It Computes
Rename Columns	Renames (required); Add Prefix; Add Suffix; Convert to lowercase (default: False); Convert to uppercase (default: False)	Renames one or more columns to improve clarity
Split Columns by Delimiters	Column to Split (required); Delimiter (default:); Custom Delimiter; Max Splits (-1 for all) (default: -1)	Splits a column into multiple columns at a specified delimiter

Split · 5 operations

All five reached from the top menu bar's Transform → Text → Split Column submenu. The Split category is not one of the groups the right-click menu lists separately, so the top menu bar is the only path to any of these five.

Operation	Parameters	What It Computes
Split by Comma	Column (required)	Splits text by comma
Split by Custom Character	Column (required); Delimiter (required)	Splits text by custom delimiter
Split by Pipe	Column (required)	Splits text by pipe
Split by Semicolon	Column (required)	Splits text by semicolon
Split by Space	Column (required)	Splits text by space

Table · 20 operations

Reached by right-clicking any column header and choosing Table.

Operation	Parameters	What It Computes
Advanced Filter	Conditions (required); Combine With (default: AND)	Filters rows using one or more conditions combined with AND/OR
Append Datasets	Dataset Names (required); Auto-align Columns (default: True); Fill Missing with Null (default: True)	Append/stack multiple datasets vertically
Column AutoFilter	Column (required); Allowed Values (required); Include Blanks (default: True)	Keeps only rows whose value in a column is one of a checked set of values
Combine Debit/Credit	Debit Column (required); Credit Column (required); Output Column (default: amount); Drop Source Columns (default: True)	Collapses a Debit/Credit column pair into one signed amount column (debit positive, credit negative) - the standard GL layout, needed before any amount-based test (Benford, outliers, sampling) can run.

Operation	Parameters	What It Computes
Compare Datasets	Right Dataset Name (required); Key Column(s) (required); Columns To Compare (blank = all common columns)	Cell-by-cell comparison of two datasets by key - classifies every record as only-in-left, only-in-right, matched-identical, or matched-different.
Extract Hierarchy Level	Account/Line-Item Column (required); Level Column Name (default: level); Strip Indentation From Name (default: True); Characters Per Level (blank = auto-detect)	Extracts the leading-whitespace depth of an account/line-item column into a new integer level column - the standard way trial balances, financial statements, and charts of accounts encode a rollup hierarchy via indentation.
Filter Rows	Column (required); Operator (required); Value	Filter rows based on a column value
Forward Fill Column	Column To Fill (required)	Carries the last non-null value in a column forward to fill subsequent nulls - fixes GL/JE exports where header fields (entry number, date, preparer) are printed only on the first line of a multi-line entry.
Fuzzy Match Datasets	Right Dataset Name (required); Left Text Column (required); Right Text Column (required); Similarity Threshold (0-1) (default: 0.8); Max Matches Per Left Row (default: 1)	Approximate text matching between two datasets by similarity score, not an exact key - e.g. matching an employee's address against a vendor's to catch near-identical (but not exactly typed) matches.
Join Datasets	Right Dataset Name (required); Left Join Key (required); Right Join Key (required); Join Type (default: left); Column Suffix (default: _right)	Join two datasets using various join types (left, right, inner, outer, cross, semi, anti). Semi/anti keep only left rows that do (semi) or don't (anti) have a matching key on the right.
Map Column Values	Column To Map (required); Value Mapping (required); Output Column (blank = overwrite); Default For Unmapped Values	Maps every value in a column through an explicit dictionary - e.g. an account name to its financial-statement category. The mapping itself is a real, inspectable parameter of this step, not a black box.
Query Sheet	Filter Condition (required); Sort By; Sort Order (default: asc); Select Columns (leave empty for all)	Creates a new filtered or transformed table using formulas
Remove Rows	Row Indices (0-based) (required); Remove These Rows (uncheck to Keep Only) (default: True)	Remove or keep specific rows by their row numbers
Select N Rows	Selection Type (required); Number of Rows (required); Random Seed (for random); Step (for every_nth) (default: 1)	Creates a new table with a specified number of rows
Sort by Column	Column (required); Ascending (default: True)	Sorts all rows by a column's values
Split Into Rows	Columns to Split (must have matching delimiter counts per row) (required); Delimiter (default: pipe); Custom Delimiter	Splits delimiter-separated values in one or more aligned columns into separate rows, e.g. turning one order row with pipe-separated line items into one row per item

Operation	Parameters	What It Computes
Split Sheet	Split By Column (required); Split Datasets	Split a dataset into multiple datasets based on unique values in a column
Transpose	Columns to Transpose (required); Include Index Column (default: True); Index Column Name (default: index)	Swaps rows and columns to convert column headers into row values
Use First Row as Headers	Row Number to Use as Header (default: 0)	Promotes the first row to become column headers (removes that row from data)
Use Headers as First Row	—	Turns the current column headers into the first row of data and resets columns to generic names (column_1, column_2, ...); all columns are cast to text

Pivot and Unpivot · 2 operations

Reached from the top menu bar's Transform → Reshape submenu. Pivot is not one of the categories the right-click menu lists separately.

Operation	Parameters	What It Computes
Pivot Table	Index Column (required); Column to Pivot (required); Values Column (required); Aggregate Function (default: sum)	Creates a pivot table
Unpivot (Melt)	ID Columns (required); Value Columns; Variable Name (default: variable); Value Name (default: value)	Transforms columns into rows

Statistics · 14 operations

Reached by right-clicking any column header and choosing Statistics. Four of these fourteen, Cross-Tabulation, Sampling, Field Statistics, and Z-Score, are additionally promoted to the top menu bar's Statistics or Audit menu; the remaining ten, including Quick Stats, Group Summary, Aging Analysis, and Stratification, are reached only through the right-click menu.

Operation	Parameters	What It Computes
Add Group Statistics	Group By Column (required); Target Column (required); Calculations (default: ['mean', 'std'])	Adds aggregate statistics as columns (mean, std, zscore, etc.)

Operation	Parameters	What It Computes
Aging Analysis	Date Column (required); As Of Date (blank = today); Age Boundaries (days) (default: [0, 30, 60, 90]); Value Column (optional, to sum per bucket); Include Counts (default: True)	Classic AR/AP aging report: buckets a date column by age in days as of a reference date (e.g. Current, 1-30, 31-60, 61-90, 90+ days) and summarizes count/sum/mean per bucket
Cross-Tabulation	Row Variable (required); Column Variable (required); Value Column (optional); Aggregation (default: count); Show Percentages (default: False); Percentages By (default: total)	Generates contingency tables for categorical variables
Field Statistics	Columns (blank = every column)	Comprehensive per-field statistics report, one row per column - count, missing, unique, and (for numeric fields) min/max/range/sum/mean/median/std/variance/skewness/kurtosis/percentiles/coefficient of variation, plus the most and least frequent value for every field.
Group Summary	Group By Columns (required); Value Column (required); Aggregations (default: ['sum', 'count'])	Creates aggregated summary reports by grouping data
Min-Max Normalization	Column (required)	Scales values to range 0-1 using min-max normalization
Percent Rank	Column (required)	Calculates the percent rank within the column
Quick Stats	Column (required)	Calculates quick column statistics
Rank	Column (required)	Calculates the rank within the column
Sampling	Sampling Method (required); Sample Size (or percentage if < 1); Sample Percentage (0-100); Stratify Column (for stratified); Value Column (dollar amount, for Monetary Unit Sampling); Random Seed; Output Type (default: sample_view)	Creates a sample-based analytical view without changing original data
Stratification	Column to Stratify (required); Strata Boundaries (required); Value Column (optional); Include Counts (default: True)	Generates stratified views by dividing values into ranges
Top/Bottom Items	Column to Rank (required); Value Column (required); Number of Items (required); Rank Type (default: top); Tie Break (default: keep_all)	Shows ranked top or bottom N records based on a metric
Trend Analysis	Date/Time Column (required); Value Column (required); Period (default: month); Aggregations (default: ['sum', 'mean']); Include Trend Line (default: False); Trend Type (default: linear)	Generates time-based analytical summaries showing trends
Z-Score	Column (required)	Calculates the z-score (standard score)

Financial · 12 operations

Reached by right-clicking any column header and choosing Financial, or from the top menu bar's own Financial menu, added once early beta feedback flagged that this entire category, twelve operations covering ratio analysis and every IFRS and IAS test in Part IV, had no discoverable path other than the right-click catalog. Every operation below is reachable both ways.

Operation	Parameters	What It Computes
Financial Ratios	Column Mapping; Selected Ratios; Fixed Values (for concepts not present as a column)	Calculate standard financial ratios (Liquidity, Solvency, Profitability)
IAS 12 DTA Recoverability Check	Column Mapping; Benchmark Profit Column (optional); Benchmark Profit Value (optional); Flag Threshold (% above benchmark) (default: 20.0)	Solve for the annual taxable profit a DTA balance requires and flag it against a benchmark profit figure
IAS 2 Inventory NRV Write-Down	Column Mapping; Tolerance Above NRV (%) (default: 0.0)	Flag inventory carried at cost above net realizable value (selling price less costs to sell)
IAS 21 FX Rate Consistency Check	Column Mapping; Flag Threshold (% rate deviation) (default: 1.0)	Flag a transaction whose booked FX rate deviates from a benchmark rate, quantifying the $ impact
IAS 36 Implied Growth Rate (Reverse DCF)	Carrying Value Column; Discount Rate Column; Explicit-Period Cash Flow Columns (in order); Benchmark Growth Column (optional); Benchmark Growth Value (optional); Flag Threshold Above Benchmark (pp) (default: 2.0)	Solve for the terminal growth rate a CGU's carrying value implicitly requires, and flag it against a benchmark
IAS 37 Provision Trend Break	Column Mapping; Flag Threshold (% YoY change) (default: 30.0)	Flag a large YoY provision change, optionally distinguishing decreases with no recorded settlement
IFRS 15 Margin Shift Detection (CUSUM)	Revenue Column; Cost Column; Period Date Column; Contract/Group Column (optional); CUSUM Slack (x std dev) (default: 0.5); CUSUM Threshold (x std dev) (default: 4.0)	Detect a sustained shift in a contract's revenue-to-cost margin over time via a CUSUM control chart
IFRS 15 Revenue Timing Gap	Revenue Recognition Date Column; Delivery/Shipment Date Column; Allowed Buffer (days) (default: 0)	Flag revenue recognized materially before the delivery/shipment date
IFRS 16 Lease Liability & IBR Check	Column Mapping; IBR Flag Threshold (bps) (default: 50.0)	Calculate lease liability/ROU asset from a lease schedule and flag an IBR that deviates from a benchmark rate
IFRS 9 Scenario-Weighted ECL	Column Mapping; Scenario Weights; Variance Flag Threshold (%) (default: 20.0)	Probability-weighted ECL across base/upside/downside PD scenarios, with an optional variance check against management's booked ECL
IFRS 9 Staging	Column Mapping; Staging Thresholds	Classify loans into Stages 1, 2, 3 based on DPD and calculate ECL

Operation	Parameters	What It Computes
Reshape Statement to Ratio Inputs	Account/Line Item Column (required); Value Column(s) (required); Period Column; Account -> Concept Mapping	Map financial-statement line items (rows) to ratio concepts and pivot into wide, one-row-per-period format

Audit · 10 operations

Reached by right-clicking any column header and choosing Audit. Most of these are also promoted to the top menu bar's Audit menu, under Audit Checks or Statistical Flags.

Operation	Parameters	What It Computes
Audit Outlier Detection	Column (required); Method (default: iqr); Flag Column (default: is_outlier); Deviation Column (default: outlier_deviation)	Detects statistical outliers for audit purposes
Benford's Law Analysis	Column to Analyze (required)	Analyzes first digits for conformity with Benford's Law
COSO Risk Heatmap	Category Column; Risk Score Column; Mapping	Aggregate risk scores by COSO components
Duplicate Detection	Columns to Check (required); Group ID Column (default: duplicate_group_id); Flag Column (default: is_duplicate)	Detects duplicate rows based on specified columns
Flag Rows	Conditions (required); Combine With (default: AND); Flag Column (default: flagged)	Marks rows matching one or more conditions (AND/OR) with a new flag column, without removing any rows
Gap Analysis	ID Column (required); Missing Flag Column (default: is_missing_sequence); Gap Count Column (default: gaps_after_this)	Detects missing values in sequential ID columns
Round Number Detection	Column (required); Roundness Threshold (default: 0.9); Flag Column (default: is_round_number); Score Column (default: roundness_score)	Flags unusually round numbers (potential manipulation)
SOX Access Review	User Status Column; Status Values Meaning Terminated/Inactive; Activity Date Column; Termination Date Column	Flag activity by a terminated/inactive user (ITGC user access review)
SOX SoD Analysis	User ID Column; Action/Role Column; Conflict Rules	Scan transaction logs for Segregation of Duties conflicts
Sequence Check	Column To Check (required); Expected Ascending Order (default: True); Flag Column (default: out_of_sequence)	Flags records that break ascending/descending order relative to the row before them, in the table's actual current row order - distinct from Gap Analysis, which looks for missing values rather than out-of-order ones (e.g. a check issued out of chronological order).

Machine Learning · 21 operations

Reached by right-clicking any column header and choosing Machine Learning, or from the top menu bar's ML menu. The four quick, single-column Anomaly Detection operations here, as discussed in Chapter 10, run against one column at a time; the genuine, multivariate, trainable versions of Isolation Forest and Local Outlier Factor are reached separately, through ML → Model Tools → Train Model.

Operation	Parameters	What It Computes
ARIMA Forecast	Value Column (required); Forecast Column (default: forecast); AR Order (p) (default: 1); Differencing (d) (default: 1); MA Order (q) (default: 1); Forecast Steps (default: 12)	Time series forecasting with ARIMA
Anomaly Detection (IQR)	Column (required); IQR Multiplier (default: 1.5); Detection Mode (Outer/Extreme Bounds, doubles the multiplier)	Detects anomalies using Interquartile Range method
Anomaly Detection (Isolation Forest)	Column (required); Contamination (default: 0.1); Number of Trees (default: 100); Max Samples (default: auto)	Detects anomalies using Isolation Forest algorithm
Anomaly Detection (Local Outlier Factor)	Column (required); Number of Neighbors (default: 20); Contamination (default: 0.1)	Detects anomalies using Local Outlier Factor
Anomaly Detection (Z-Score)	Column (required); Threshold (std devs) (default: 3.0); Direction (Both/Above Mean Only/Below Mean Only, default: Both)	Detects anomalies using Z-Score method
DBSCAN Clustering	Feature Columns (required); Cluster Column (default: cluster); Epsilon (neighborhood) (default: 0.5); Min Samples (default: 5)	Density-based spatial clustering
Decision Tree	Feature Columns (required); Target Column (required); Prediction Column (default: prediction); Max Depth (default: 10); Min Samples Split (default: 2); Min Samples Leaf (default: 1)	Classification/regression using decision tree
Exponential Smoothing	Value Column (required); Forecast Column (default: forecast); Trend (default: add); Seasonal; Forecast Steps (default: 12)	Exponential smoothing state space models
GLM Regression	Feature Columns (required); Target Column (required); Prediction Column (default: y_hat); Family (default: gaussian)	Generalized Linear Models (Gaussian, Binomial, Poisson, Gamma)
Gaussian Mixture	Feature Columns (required); Cluster Column (default: cluster); Number of Components (default: 3)	Probabilistic clustering with Gaussian mixtures

Operation	Parameters	What It Computes
Gradient Boosting	Feature Columns (required); Target Column (required); Prediction Column (default: prediction); Number of Estimators (default: 100); Learning Rate (default: 0.1); Max Depth (default: 5); Min Samples Split (default: 2); Min Samples Leaf (default: 1)	Sequential ensemble using gradient boosting
K-Means Clustering	Feature Columns (required); Number of Clusters (default: 3)	Performs K-Means clustering
LightGBM	Feature Columns (required); Target Column (required); Prediction Column (default: prediction); Number of Leaves (default: 100); Learning Rate (default: 0.1); Number of Leaves (default: 31); Min Child Samples (default: 20)	Light Gradient Boosting Machine - fast on large datasets
Linear Regression	Feature Columns (required); Target Column (required); Prediction Column (default: y_hat); Fit Intercept (default: True)	Predicts continuous values using linear regression
ML Classification	Feature Columns (required); Target Column (Labels) (required); Model ID; Output Column (default: predicted_class); Probability Column (default: probability)	Classifies data using a supervised ML model
ML Predict	Model ID (required); Feature Columns (required); Output Column (default: predicted)	Applies a trained model to predict new values
ML Risk Score	Group Column (required); Feature Columns (required); Output Column (default: risk_score)	Calculates risk scores per group based on features
Outlier Detection	Column (required); Method (default: iqr)	Detects statistical outliers
Random Forest	Feature Columns (required); Target Column (required); Prediction Column (default: prediction); Number of Trees (default: 100); Max Depth (default: 10); Min Samples Split (default: 2); Min Samples Leaf (default: 1)	Ensemble learning with multiple decision trees
Train ML Model	Model Name (required); Algorithm (required); Feature Columns (required); Target Column (required); Test Split Ratio (default: 0.2); Random Seed (default: 42)	Train a machine learning model for later use in predictions
XGBoost	Feature Columns (required); Target Column (required); Prediction Column (default: prediction); Number of Trees (default: 100); Learning Rate (default: 0.1); Max Depth (default: 6); Subsample (default: 0.8); Column Sample by Tree (default: 0.8)	Extreme Gradient Boosting for high performance

AI · 8 operations

Reached by right-clicking any column header and choosing AI, or from the top menu bar's AI menu. Every operation in this category requires an AI provider to be configured first, and follows the translate-then-compute discipline described in Chapter 4.

Operation	Parameters	What It Computes
AI Enrich	Source Column (required); Enrichment Type (required); Output Column (default: enriched)	Enriches data with AI-generated information
AI Generate Column	Column Name (required); Instruction (required); Generated Code (required)	Generates a new column using AI
AI Summarize	Text Column (required); Max Length (default: 100); Output Column (default: summary)	Generates AI summaries for text content
AI Text Classification	Text Column (required); Categories (required); Category Column (default: ai_category); Confidence Column (default: ai_confidence)	Classifies text into categories using AI
Explain Column with AI	Column (required); Explanation Style (default: concise)	Generates AI explanations for column values
Extract Entities	Text Column (required); Entity Type (required)	Extracts entities from text using AI
Pattern Learning (Flash Fill)	Source Column (required); Output Column (required); Pattern / Instruction (e.g. "extract the last name")	Applies a described transformation pattern to every row using AI
Sentiment Analysis	Text Column (required)	Analyzes text sentiment

SQL · 1 operations

This entry corresponds to the dedicated SQL Query view, reached from the sidebar toggle at the top of the Data view, rather than to a dialog opened from the Transform view.

Operation	Parameters	What It Computes
SQL Query	SQL Query (required)	Transform the dataset with a DuckDB SQL query

Standalone Tools Outside the Operation Registry

The following features are not part of the operation registry described above and do not appear in the right-click menu. Each is reached directly from the sidebar and, with the exception of Financial Import Wizard and Cleaning Wizard, none of them modify the underlying dataset or add a step to the Applied Steps pipeline.

Tool	Sidebar Section	What It Does
Profile Dataset	Analyze	Runs the automatic dataset profile described in Chapter 5: row and column counts, duplicate rows, per-column completeness and outliers, a Benford screen, and correlated column pairs. Read-only.
Verify Data	Analyze	Runs one or more of the six completeness and integrity checks described in Chapter 6 and reports pass, fail, or skipped for each. Read-only.
Variables Sampling	Analyze	Opens the Classical Variables Sampling dialog described in Chapter 11: sample size planning on one tab, sample result evaluation on the other.
Audit Programs	Analyze	Manages saved, reusable sequences of steps that can be reapplied to a new dataset, covered in Chapter 29.
Join Datasets	Combine	Combines two datasets on a matching key, covered in Chapter 2.
Append Datasets	Combine	Stacks two datasets with compatible columns into one, covered in Chapter 2.
Compare Datasets	Combine	Reports differences between two datasets expected to represent the same population at two points in time.
Fuzzy Match Datasets	Combine	Joins two datasets on an approximate, rather than exact, text match, useful when the same vendor or customer is spelled inconsistently across two sources.
Import Folder Listing	Import	Imports the file and folder metadata of a directory as a dataset, useful for testing file-retention or access-control policies against what is actually present on a file share.
Project Lineage	Lineage & Export	Visualizes how every dataset in the current project was derived, including joins, appends, and pivots.
Export for Review	Lineage & Export	Packages a dataset together with its Applied Steps pipeline for a second reviewer.

Appendix B

Dataset Reference

This book follows one fictional company, Meridian Office Supply Co., through fifteen small working files, none exceeding thirty-four rows, so that every worked example could be followed in full rather than taken on faith. This appendix lists every one of them, its columns, and every chapter that uses it, in case a reader wants to locate the exact file behind a specific figure cited earlier. All fifteen sit alongside this book's own source file, in the same repository, and none contains real client data of any kind.

File	Rows	Columns	Used In
ap_transactions.csv	34	invoice_id, invoice_date, vendor, department, account, description, amount, payment_method, approver, status	Ch. 2, 3, 5, 6, 7, 8, 10, 11, 14, 27, 29, 31
quarterly_financials.csv	4	period, revenue, cogs, gross_profit, net_income, ebitda, current_assets, current_liabilities, cash, inventory, total_assets, total_liabilities, total_equity, interest_expense	Ch. 9
user_activity_log.csv	21	user_id, user_name, department, action, action_date, status, termination_date	Ch. 12
process_risk_register.csv	8	process_area, finding_count, risk_score	Ch. 13
contract_revenue_schedule.csv	14	contract_id, period, revenue, cost, recognition_date, delivery_date	Ch. 15
loan_portfolio.csv	10	loan_id, exposure, dpd, pd_current, pd_origination, lgd, pd_base, pd_upside, pd_downside, management_ecl	Ch. 16, 25
lease_schedule.csv	6	lease_id, description, payment, periods, management_ibr, benchmark_ibr, initial_direct_costs, lease_incentives	Ch. 17
cgu_impairment_test.csv	4	cgu_id, carrying_value, discount_rate, cf_y1, cf_y2, cf_y3, cf_y4, cf_y5	Ch. 18

File	Rows	Columns	Used In
provisions_register.csv	5	category, prior_year, current_year, settlement_flag	Ch. 19
inventory_valuation.csv	5	sku, description, cost, selling_price, costs_to_sell, quantity	Ch. 19
deferred_tax_register.csv	4	jurisdiction, dta_balance, tax_rate, utilization_years, benchmark_profit	Ch. 20
fx_transactions.csv	5	transaction_id, currency_pair, booked_rate, benchmark_rate, local_amount	Ch. 20
meridian_trial_balance_q1.csv	12	period, account, balance	Ch. 21
income_statement.csv	1	period, revenue, cogs, gross_profit, net_income, ebitda, interest_expense	Ch. 21
balance_sheet.csv	1	period, current_assets, current_liabilities, cash, inventory, total_assets, total_liabilities, total_equity	Ch. 21

Table B.1 — Every Meridian Office Supply Co. working file used in this book, generated directly from the files themselves rather than transcribed by hand.

Chapter 22 draws on a separate set of files, real XBRL and inline XBRL fixtures rather than Meridian business data, since that chapter demonstrates a file format rather than a company's figures.

File	Format	Purpose
test-taxonomy.xsd	XBRL taxonomy schema	The custom, self-contained taxonomy every other file in this set references.
test-instance.xml	XBRL instance document	A single-period Revenue and Assets filing used to confirm plain XBRL parses correctly.
test-multi-period.xml	XBRL instance document	Two years of Revenue plus one segment-dimensional fact; this book's primary Chapter 22 working file.
test-inline.xhtml	Inline XBRL (iXBRL)	The same Revenue fact embedded inside a human-readable HTML narrative, confirming inline filings parse identically to plain XBRL.

File	Format	Purpose
remote-ref-instance.xml	XBRL instance document	References its taxonomy by an unresolvable network URL, used to demonstrate the local-first taxonomy package fallback.
test-package/test-taxonomy-package.zip	XBRL Taxonomy Package (spec-compliant)	A real, minimal Taxonomy Package registered to resolve remote-ref-instance.xml's schema reference entirely from disk.

Table B.2 — Every XBRL fixture used in Chapter 22.

Appendix C

Default Thresholds Reference

Nearly every deterministic test in this book, from a statistical outlier fence to an IFRS impairment flag, ships with a configurable default rather than a fixed, hidden constant. Every default in this appendix is quoted directly from the application's own operation registry or service code, not reconstructed from memory, and every one of them carries the same caveat stated repeatedly throughout Parts II through IV: a default is a documented starting point for follow-up audit judgment, never a standard-defined bright line, and the human sanity check this book has deferred to a qualified auditor throughout belongs here most of all, before any of these defaults are relied upon in a real engagement.

Classic CAAT Defaults

Technique	Parameter	Default	Chapter
Profile Dataset – Benford screen	Minimum population; Nigrini MAD conformity threshold	100 non-zero values; 0.012	5, 8
Benford's Law Analysis	Minimum population, enforced by the dialog itself	100 valid, non-zero values	8
Audit Outlier Detection	IQR fence; z-score threshold (both fixed, not configurable in this dialog)	1.5× IQR; 3.0 std dev	10
Anomaly Detection (IQR) – ML menu	IQR Multiplier; Detection Mode	1.5; Outer Bounds (Extreme Bounds doubles the multiplier)	10
Anomaly Detection (Z-Score) – ML menu	Threshold (std devs); Direction	3.0; Both	10
Anomaly Detection (Isolation Forest) – ML menu	Contamination; Number of Trees; Max Samples	0.1; 100; auto	10
Anomaly Detection (Local Outlier Factor) – ML menu	Number of Neighbors; Contamination	20; 0.1	10
Round Number Detection	Roundness Threshold	0.9	10
Fuzzy Match Datasets	Similarity Threshold (0–1)	0.8	2
Classical Variables Sampling	Confidence Level (and implied z-score)	90/95/97.5/99% → 1.645/1.960/2.240/2.576	11
Pareto Chart	Max categories rendered	20	27

Technique	Parameter	Default	Chapter
Control Chart	Control limits (fixed, not configurable)	mean ± 3 std dev	27

IFRS and IAS Defaults

Standard	Parameter	Default	Chapter
IFRS 15 Margin Shift (CUSUM)	Slack (k); Threshold (h)	0.5× scale; 4.0× scale	15
IFRS 15 Revenue Timing Gap	Allowed Buffer	0 days	15
IFRS 9 Staging	Stage 2 / Stage 3 threshold; SICR PD Multiplier	30 / 90 days past due; 2.0×	16
IFRS 9 Scenario-Weighted ECL	Variance Flag Threshold	20%	16
IFRS 16 Lease Liability & IBR Check	IBR Flag Threshold	50 bps	17
IAS 36 Implied Growth Rate	Flag Threshold Above Benchmark	2.0 pp	18
IAS 37 Provision Trend Break	Flag Threshold (% YoY change)	30%	19
IAS 2 Inventory NRV Write-Down	Tolerance Above NRV	0%	19
IAS 21 FX Rate Consistency Check	Flag Threshold (% rate deviation)	1%	20
IAS 12 DTA Recoverability Check	Flag Threshold (% above benchmark)	20%	20

AI-Assisted Feature Defaults

Feature	Parameter	Default	Chapter
AI Manager	Maximum output tokens per call (configurable, persisted)	4,000	23
AI Manager	Completion temperature: general/structured call; batched column operation; grounded chat answer	0.1; 0.2; 0.3	23, 24, 26
Column-level AI operations	Rows per batched call	25	23, 24
Numeric Disclosure Reconciliation	Absolute tolerance; relative tolerance	0.005; 10%	25

One entry deserves a direct note rather than a bare number. Chapter 16's walkthrough of the SICR PD trigger sets the multiplier to 3.00x for that specific worked example; the application's

own default, shown whenever the IFRS 9 Staging dialog first opens, is 2.00x. The chapter says so explicitly at the point it changes the value, and this appendix repeats it here since a threshold reference that silently mismatched its own book would defeat the purpose of compiling one.

Appendix D

A Quick Reference for Polars and SQL

This book teaches CaatMaster almost entirely through its dialogs, without asking a reader to write a line of code. Two exceptions exist: three transformation dialogs accept a Polars expression directly, and the SQL Query workspace covered in Chapter 33 accepts genuine SQL. Neither demands real programming experience to use productively for the kind of one-off calculation or filter this book's built-in operations don't happen to cover, and this appendix collects just enough of each to get a reader started. It is a launching point, not a substitute for either language's own documentation.

D.1 — Polars Expressions

Three dialogs accept a raw expression instead of a fixed set of parameters: Custom Column from Math and Calculated Columns, both reached by right-clicking a column header and choosing Calculate, and Query Sheet, reached from the sidebar, which filters and reshapes a table using an expression instead of the Filter dialog's point-and-click conditions. All three placeholder hints show the same shorthand, and as of this printing all three actually accept it identically.

The one syntax rule that matters: refer to a column by writing `col('Column Name')` exactly as the column appears in the dataset, including spaces and case. Everything else is ordinary Polars — comparison operators (`>` , `<` , `==` , `!=`), arithmetic (`+` , `-` , `*` , `/`), and a small set of methods chained after a column reference for text, null, and rounding work.

Task	Expression	Used In
Compute a difference	`col('Revenue') - col('COGS')`	Custom Column from Math
Round to two decimals	`col('Amount').round(2)`	Custom Column from Math
Flag values over a threshold	`col('Amount') > 10000`	Calculated Columns, Query Sheet
Text contains a substring	`col('Vendor').str.contains('Consulting')`	Query Sheet
Row has a missing value	`col('Approver').is_null()`	Query Sheet
Three-way conditional label	`pl.when(col('Amount') > 10000).then(pl.lit('High')).otherwise(pl.lit('Normal'))`	Calculated Columns

Table D.1 — Every expression above was run against a real dataframe before being printed here, not composed from memory of Polars' API.

> A GOTCHA WORTH KNOWING
>
> Combining two conditions with Python's own `and` / `or`, as in `col('Amount') > 1000 and col('Approver').is_not_null()`, fails with `the truth value of an Expr is ambiguous` — a Polars expression is a description of a calculation, not yet a value, so it cannot be True or False the way `and` / `or` require. Use `&` and `|` instead, and parenthesize each side: `(col('Amount') > 1000) & (col('Approver').is_not_null())`. The parentheses are not optional style — `&` binds tighter than `>` in Python, so `col('Amount') > 1000 & col('Approver').is_not_null()` without them fails with a different, equally real error, `cannot unpack series of type `bool` into `i32``, because `&` is evaluated before `>` gets to it.

D.2 — SQL for the Query Workspace

Chapter 33 covers reaching the SQL Query workspace, its DuckDB internals, Model Query cross-dataset joins, and worked audit examples in full. This section is a minimal starting point for a reader who has never written SQL, using the same AP_Transactions table Chapter 33 works with throughout.

```
-- Every column, every row
SELECT * FROM AP_Transactions

-- Only rows meeting a condition
SELECT * FROM AP_Transactions WHERE amount > 5000

-- Combine conditions with AND / OR
SELECT * FROM AP_Transactions
WHERE department = 'IT' AND amount > 1000

-- Sort results, largest first
SELECT * FROM AP_Transactions ORDER BY amount DESC

-- One row per group, with an aggregate per group
SELECT department, SUM(amount) AS total, COUNT(*) AS n
FROM AP_Transactions
WHERE status = 'Approved'
GROUP BY department
ORDER BY total DESC
```

Every clause above is standard SQL, not a CaatMaster-specific dialect — DuckDB, the SQL engine described in Chapter 33, accepts all of it directly. The self-join, window-function, and Model Query join patterns that come up specifically in audit work are covered where they belong, worked through against real figures, in Chapter 33 itself.

Audit Analytics with CaatMaster is complete: all nine parts, thirty-three chapters, and all four appendices.